Learning to Build Common Ground in Multi-Turn Interaction

Saujas Srinivasa Vaduguru

Language Technologies Institute
School of Computer Science
Carnegie Mellon University

April 2026

Thesis Committee

Daniel Fried	*Carnegie Mellon University*
Graham Neubig	*Carnegie Mellon University*
Yonatan Bisk	*Carnegie Mellon University*
Yoav Artzi	*Cornell University*

Submitted in partial fulfillment of
the requirements for the degree of Doctor of Philosophy.

Abstract

People are increasingly using digital agents in a variety of tasks like software development, information seeking, and knowledge work. These agents need to infer the users' goals from what they communicate, and take turns acting in the environment and interacting with the user until the goal is achieved. Effective communication in these settings involves *common ground* between the user and the agent – mutual knowledge, beliefs, and suppositions. In this thesis, I propose ways to build and evaluate agents that leverage common ground to communicate well with people.

In Part I, I look at reasoning about user intent. Users and agents have a mutual expectation of cooperativity that is part of their common ground. A cooperative user communicates in a way that lets the agent infer their intent. This assumption allows for disambiguating messages that may have multiple literal interpretations. I study this in the context of programming-by-example, where a user chooses (possibly ambiguous) examples of program behavior to illustrate an intended program to a synthesizer. I train a neural program synthesizer that efficiently resolves ambiguity in examples using high-quality synthetic training data from interactions between a simulated, cooperative user and a syntheizer. I propose further investigating methods to train agents to communicate better with people using synthetic data by examining the role of privileged access to the solution available to the simulated user. I hypothesize that training with or without privileged access to the solution leads to different communicative behavior, and aim to characterize these differences.

In Part II, I turn to reasoning about observations shared by the user and agent. I study collaborative design, where a user instructs an agent to create a design on a shared canvas over multiple turns using drawing and natural language. I show that instructions to generate a design on a blank canvas differ from instructions to refine an existing design in the user and agent's shared context. I show that multimodal models are worse at refinement than generation, highlighting room for improvement in multi-turn interaction.

In Part III, I focus on building common ground over time. I develop a method to train agents to leverage shared knowledge of past interactions to communicate more efficiently with people in repeated interactions. I show that training an agent in simulated games to communicate successfully while minimizing cost elicits convention formation, which enables efficient communication. I find that neither success nor cost alone is sufficient to elicit this behavior. I also propose developing a method to train browser use agents to seek help that considers the shared context of prior interactions to guide the agent's behavior. By selectively seeking help, and learning from the help provided, agents can become more capable while reducing the user effort to complete tasks.

Contents

1 Introduction **1**

1.1 Thesis overview . . . 2

Part I Reasoning about Intent . . . 2

Part II Reasoning about Shared Observations . . . 3

Part III Reasoning about Common Ground Over Time . . . 4

1.2 Summary of progress . . . 5

2 Background **7**

2.1 Common ground . . . 7

2.2 Games . . . 9

I Reasoning about Intent **11**

3 Inferring Intent by Reasoning about Pragmatic Demonstrations **12**

3.1 Background . . . 15

3.2 Method . . . 16

3.2.1 Speaker and listener Models . . . 17

3.2.2 Training base models . . . 17

3.2.3 Generating informative examples . . . 17

3.2.4 Model updates . . . 18

3.3 Experiments . . . 18

3.3.1 Regular expressions . . . 18

3.3.2 Models for comparison . . . 19

3.3.3 Procedure . . . 20

3.3.4 Measurement . . . 20

3.3.5 Human-provided specifications . . . 21

3.3.6 Results . . . 21

3.4 Analysis of training . . . 23

3.5 Related work . . . 23

3.6 Conclusion . . . 24

4 Learning from Information Asymmetries in Simulated Interaction *(proposed)* **26**

4.1 Proposed setup . . . 28

II Reasoning about Shared Observations **29**

5 Multimodal Interaction for Collaborative Construction **30**
5.1 The mrCAD Task . . . 32
5.1.1 mrCAD environment . . . 32
5.1.2 Playing the Game . . . 33
5.2 Effects of Multimodality and Refinement on Communication . . . 34
5.3 The mrCAD Dataset . . . 35
5.3.1 Collection . . . 36
5.3.2 Dataset statistics . . . 36
5.3.3 Analyzing instructions in mrCAD . . . 38
5.4 Evaluation of frontier models . . . 39
5.4.1 Benchmark . . . 39
5.4.2 Evaluating vision-language models . . . 40
5.4.3 Performance of vision-language models . . . 41
5.5 Related Work . . . 42
5.6 Discussion . . . 43

III Reasoning about Common Ground Over Time **45**

6 Learning to Form Conventions from Simulated Interactions **46**
6.1 Related work . . . 48
6.2 Learning in simulated repeated reference games . . . 49
6.3 Experiments . . . 52
6.3.1 Image domains . . . 52
6.3.2 Models . . . 52
6.3.3 Training . . . 53
6.3.4 Evaluation . . . 54
6.3.5 Human listener experiments . . . 54
6.3.6 Simulated listener experiments . . . 57
6.4 Discussion . . . 58

7 Agents that Learn from Help they Seek *(proposed)* **60**
7.1 Proposed setup . . . 60
7.2 Experiments . . . 61
7.2.1 Illustrative results . . . 62

A Appendix: Inferring Intent by Reasoning about Pragmatic Demonstrations **64**
A.1 Program sampling . . . 64
A.2 Training base models . . . 64
A.3 Pragmatic model training . . . 65
A.4 Dataset collection . . . 67
A.5 User study . . . 69
A.6 Prompting an LLM . . . 70
A.7 Speaker quality . . . 71
A.8 Connections to iterated bootstrapped training . . . 71

B Appendix: Multimodal Interaction for Collaborative Construction **72**
B.1 Analysis . . . 72
B.1.1 Statistical methods . . . 72
B.1.2 Distance metric . . . 72
B.2 Multimodality and Refinement Ablations . . . 73
B.2.1 Experimental methods . . . 73
B.2.2 Results . . . 73
B.3 Data Collection Procedure . . . 74
B.3.1 mrCAD annotation task . . . 74
B.3.2 Dataset collection interface . . . 74
B.3.3 Dataset collection parameters . . . 74
B.3.4 Sampling CAD tasks . . . 76
B.3.5 Prioritizing motivated annotators . . . 76
B.3.6 Prioritizing refinement instructions . . . 77
B.3.7 Exclusion criteria . . . 78
B.4 Vision-language model experiments . . . 78
B.4.1 Hardware and software details . . . 78
B.4.2 Prompt . . . 78
B.5 Analyzing the number of actions . . . 80

C Appendix: Learning to Form Conventions from Simulated Interactions **82**
C.1 Base models . . . 82
C.1.1 COCO base models . . . 82
C.1.2 Tangram base models . . . 83
C.2 Sampling contexts of images . . . 84
C.3 Preference optimization . . . 85
C.4 Metrics . . . 86
C.5 Human study . . . 86

C.5.1 Interface 88
C.5.2 Questions 89
C.5.3 Characterizing the consistency of human speakers 90
C.6 Tangrams model-as-a-listener results 91

Bibliography **92**

List of Figures

3.1 Diagrammatic overview of PRAX, depicting different stages of the process . 13
3.2 An illustration of how the Rational Speech Acts framework is used to select an informative example . . . 14
3.3 Performance of PRAX compared to baseline methods as a function of turns as measured by different metrics . . . 22
3.4 Example specifications for two programs provided during the user study, along with the highest ranked guess from the LITERAL and the PRAX models. 22
3.5 Progression of the TOP-1 metric over the course of rounds of training of the PRAX model, with comparisons to training on last round PRAX data and to training in a single round . . . 23

5.1 A depiction of the mrCAD task . . . 31
5.2 Illustration of the Chamfer distance metric . . . 33
5.3 Results of multimodality and refinement ablations . . . 35
5.4 Dataset statistics for mrCAD . . . 36
5.5 Example rollouts from the mrCAD dataset . . . 37
5.6 Analysis of instructions in mrCAD, showing the difference in modality and language use in generation and refinement . . . 38
5.7 Comparison of human and model improvement towards target showing proportional improvement over rounds and examples . . . 40

6.1 Example utterances from a model trained with our proposed method . . . 47
6.2 Diagrammatic overview of the proposed method to train models to form conventions . . . 50
6.3 Results of repeated reference game experiments with human listeners . . . 53
6.4 Examples of adaptation over the course of a game by the SUCCESS+COST speaker model . . . 54
6.5 Word novelty rate (WNR) results for human listener experiments . . . 56
6.6 Results of simulated listener experiments with a GPT-5-mini listener . . . 57
6.7 Word class distribution of SUCCESS+COST model utterances over time . . . 58

A.1 Comparing validation performance of a base model trained from a pretrained checkpoint to one trained from a randomly initialized checkpoint . . . 65
A.2 Comparison between the L0 model from the original experiments and the L0+ model trained for longer . . . 67

A.3 Interface for providing examples for a regular expression 68
A.4 Interface for verifying a specification written by another human 69
A.5 Screenshot of the regular expressionuser study UI with an example interaction 70
A.6 Speaker loss on the validation set computed over rounds of training 71

B.1 Screenshots of the user interfaces used for mrCAD data collection 75
B.2 Dynamic submission threshold for mrCAD dataset collection 77
B.3 Structure of the prompt used for API calls and SFT models. 79

C.1 The listener's interface for repeated reference games 89
C.2 The speaker's view of images for repeated reference games 90
C.3 Questions asked to participants about the model with which they interacted. 90
C.4 Questions asked to participants comparing different models. 91
C.5 Results of tangram experiments with a model listener 92

List of Tables

3.1 Success metrics at the end of 10 turns of interaction with each model 20
3.2 Average success metric over 10 turns 21

5.1 Proportional improvement results for vision-language models 42

B.1 Parameters for ablation studies and primary dataset collection 76
B.2 Prompting hyperparameters 80
B.3 Fine-tuning hyperparameters for all runs 80
B.4 Average number of actions taken by models in generation and refinement stages 81

C.1 Mapping from POS tags to part-of-speech types used in Figure 6.7. 87
C.2 The number of games with each speaker. 88

Chapter 1

Introduction

AI agents are becoming more capable, and carrying out increasingly complex tasks in realistic computer environments like code repositories (OpenAI, 2024), web browsers (OpenAI, 2025c), and administrative software (Drouin et al., 2024a). As agents improve, they become capable of working without intervention to perform increasingly complex tasks that would take people a long time to complete (Kwa et al., 2025).

However, the ability to execute long-horizon plans does not necessarily make agents better collaborators (Shen et al., 2025). Even the most powerful models tend to overstep bounds of user requests (Chen et al., 2025a). They fail to incorporate information introduced in later turns of interaction (Laban et al., 2025). They also fail to adapt to communicate more efficiently (Hua and Artzi, 2024). These issues highlight that while the ability to act in digital environments has improved, there is still room to improve models' ability to communicate and interact with people efficiently and effectively.

Collaboration requires that an agent participate in *joint actions* with a user, and coordinate on shared goals, as opposed to just taking *individual actions*. An agent that works with a user to build software has to coordinate with the user to understand what the software should do, and ensure the user understands how the agent's work meets their requirements. This coordination requires *common ground* between the user and the agent – mutual knowledge of beliefs, goals, and suppositions (Clark, 1996).

Depending on the kind of interaction between the user and the agent, the common ground they share may have different elements. The roles of the user and agent, and the associated responsibilities are a part of common ground. In interactions that span multiple turns, prior turns are part of the common ground. If the interaction is in a conventional natural language like English, the common ground includes that language. If the interaction is in a setting like an integrated development environment (IDE),[1] the state of the IDE, including files in the workspace, different tools that are available, etc. are part of the common ground.

Common ground is not static. As a pair of people interact, they constantly update their common ground based on things that are said, and done, and observed. A typical interaction pattern between a user and an agent is for the user to instruct the agent to perform some task. Here, the user aims to bring their intent (which is initially known only to them) into the common ground between them and the agent. However, these updates

[1] www.cursor.com

may not always happen in the same way, leading to discrepancies in each participant's belief about what is in common ground. When necessary, these discrepancies can be resolved with further communication. For agents to successfully coordinate with users, they need to be able to engage with and drive changes in common ground shared with a user (Shaikh et al., 2024).

Agents that collaborate effectively with people need to be able to use common ground. They need to effectively deploy prior knowledge of the agent's role in the collaboration. They need to be aware of the environment they share with the user, and what is observable by the agent, the user, or both. They also need to be able to track and effect changes in common ground over the course of one or more interactions. This motivates the focus of this thesis on building common ground in multi-turn interaction.

1.1 Thesis overview

In this thesis, I propose ways to take advantage of the assumption that agents are engaged in grounded, multi-turn interaction with users to improve how efficiently and effectively they communicate. I model grounded, multi-turn interactions as cooperative games played by a user and an agent that communicate with each other, and train agents to be better partners to human users in these games. This allows for training agents in simulated games, without relying on demonstrations of how people communicate. I also propose resources to characterize new challenges in multi-turn interaction, and ways to evaluate agents in these settings. I consider challenges in reasoning about common ground, and building common ground in interactions over time.

First, in Chapter 2, I present background on common ground and cooperative games that will be used throughout the thesis. Then, I present the proposed thesis in three parts.

Part I Reasoning about Intent

When a user and agent interact, both are aware that the user intends to accomplish some task, and both the user and the agent share the goal of accomplishing that task with as little effort as possible. The user's communication is to be interpreted against this backdrop. When the user says something, the agent can reason about why the user believed that specific utterance was the best way to communicate their intent. When a customer walks up to a barista at a coffee shop and says *"Cappuccino."*, the barista infers that the customer intends to order a cappuccino, and they're not simply stating that a cappuccino is available, or asking whether a cappuccino is available. The barista might reason that if they wanted to do either of these, they would have said so. This reasoning can allow the agent to infer more than the literal meaning of the user's utterances.

In Chapter 3 I consider the task of programming-by-example. Here, a user specifies their intent to a program synthesis agent (synthesizer) using input-output examples – pairs of program input and the output produced by the intended program. I propose a method for simulating a cooperative user who produces pragmatic examples – those that are specifically designed to contrast their intended program against plausible alternatives, and use interactions between this simulated user and the synthesizer to generate synthetic training data. I show that training on this synthetic data yields a synthesizer that infers the user's intent with higher accuracy and with fewer examples (less user effort) than a synthesizer that does not reason about the user providing pragmatic examples.

Approaches to training agents to interact better with people in simulated interactions (including some discussed in this thesis) typically simulate a human user with an agent that has privileged access to the target solution, or a fully-specified version of the user's intent. In Chapter 4, I propose investigating the implications of such training-time access for how models interact with users who are uncertain about their intent. It is often the case that a human user does not have have a complete picture of the output they intend the agent to produce for a variety of reasons. A user booking flights with an agent's assistance may not know different flight options available to them, but have preferences when shown these options. A user making slides with agentic assistance may not have a clear picture of the slide template, but be able to iteratively tweak the slides based on their aesthetic sensibility. In this chapter, I propose looking at agents that assist a user in solving math problems. Here, a user may not be able to solve the problem themselves, but nevertheless be able to verify whether a proposed solution is correct, and provide feedback on where the assistant might have gone wrong. I hypothesize that simulated users that have privileged access to the solution would communicate differently from those who are uncertain about the solution. I propose conducting a controlled investigation to understand how agents trained to communicate with such simulated users communicate differently from those that don't have privileged access to the solution.

Part II Reasoning about Shared Observations

Collaboration often occurs in a setting where multiple parties work alongside each other in a shared environment. These settings include from collaborative document editing when writing a paper, pair programming, multiplayer video games, and even high-stakes settings like a team of medical specialists working together on the same surgical procedure. In such settings, in addition to all participants being aware of the shared goal, they also have shared observation of the environment, and consequences of actions others take on this shared environment. In contrast, an agent acting autonomously to fix a software bug or gather information by browsing the web does not interact with the in the same way, and interacts

for multiple turns with the environment in a way that may not be visible to a user.

In Chapter 5, I look at how people collaborate over multiple turns of interaction to create an object. I aim to understand how people issue instructions to iteratively refine an object, and evaluate how well agents interpret these instructions. Specifically, I look at the collaborative creation of a *design* – a complex object composed of parametric curves. In this setting, a designer and a maker start with an empty shared canvas, and iteratively recreate a target design known only to the designer. At each turn, the designer uses freeform text and freehand drawing on the canvas to instruct the maker. The maker responds by adding, modifying, and removing curves on the shared canvas. Crucially, at each turn, the designer's instruction – their drawings and language – are grounded in the current state of the canvas visible to both agents. Additionally, the designer's language may be grounded in the drawings. Interpreting the designer's instructions and executing the refinements requires understanding language in context of the state of the canvas, the drawings, and the shared goal of creating a design.

I detail *mrCAD*, a collaborative design game to collect a large-scale dataset of human-human interactions. I analyze the dataset and identify the ways in which generation instructions – instructions to produce a design on an empty canvas – differ from refinement instructions – instructions grounded in the existing state of the canvas. I also describe how we use this dataset to benchmark multimodal models at this task, and identify how models are much worse at refinement than generation.

Part III Reasoning about Common Ground Over Time

As collaborators work together for a period of time, they adapt to work more efficiently. They leverage the fact that prior interactions are now part of shared context in their communication to get more done by saying less. Over time, they are able to build a better understanding of each other's preferences and the shared environment through interaction. If a first-time vegan customer at a coffee shop order's a cappuccino, the barista might ask whether they want cow's milk or plant-based milk. If the customer visits regularly, they might remember the customer's preference, and use plant-based milk even if the customer just asks for a *"Cappuccino"*. The barista here has adapted their interpretation of this customer's cappuccino order, and enriched it with information about the customer's preferences by asking suitable questions. This allows the barista to communicate more efficiently with the customer over time.

In Chapter 6, I study the formation of conventions of reference in repeated interactions. When people repeatedly refer to objects in the same shared context over time, they converge on efficient conventions of reference – ways to describe these objects with few words by exploiting shared knowledge of the context and prior interactions. In repeated reference

games, an experimental setting used to study this kind of convention formation, multimodal models have been shown to be inefficient communicators that fail to adapt to shared context in human-like ways. I build on the approach in Chapter 3 and propose a method for training agents in simulated interactions to be efficient communicators, with no explicit demonstrations of how people form conventions. Using the controlled experimental setting, I show that models incentivized to succeed while minimizing the cost of their utterances learn to form conventions. Neither success not cost alone is enough to elicit this behavior. I also show that these models trained with no human demonstrations of convention formation produce messages that people respond to more quickly over time, suggesting that people are sensitive to this efficient communication.

In Chapter 7, I propose developing agents that improve over time by seeking help, rather than passively waiting to be given the right kind of guidance. Browser use agents navigate complex web environments and execute complex, multi-step plans to perform tasks considered routine. Satisfactorily achieving goals in these environments requires precise instruction. Agents may encounter states where there are multiple viable ways forward, and choosing the right one requires context-dependent guidance. However, having to provide this guidance at every use reduces the utility of agents. Agents that retain relevant guidance and apply it in future tasks reduce the burden of specification on the user. Additionally, agents that seek help before making a mistake reduce the burden of correction on the user. I propose a method to teach agents to seek help from users, and learn from this help to make interaction over time more efficient.

1.2 Summary of progress

Much of the work in this thesis has been conducted in collaboration with a number of researchers. In the chapters where I describe work that also appears in a papers jointly authored with collaborators, I use the first person pronoun *we* in the chapter. In other chapters, I use the first person pronoun *I*.

Chapter 3 describes our work on PraX that appeared at ICLR 2024. Chapter 5 describes our work on mrCAD that appeared in the Findings of EMNLP 2025. Chapter 6 describes completed work on conventions that is currently under submission to ACL 2026. Chapter 7 describes ongoing work, and Chapter 4 is work that I plan to undertake in the future.

Papers

- **Saujas Vaduguru**, Daniel Fried, and Yewen Pu. 2024. Generating pragmatic examples to train neural program synthesizers. In *The Twelfth International Conference on Learning Representations*

- William P McCarthy*, **Saujas Vaduguru***, Karl D.D. Willis, Justin Matejka, Judith E Fan, Daniel Fried, and Yewen Pu. 2025. mrCAD: Multimodal communication to refine computer-aided designs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*
- **Saujas Vaduguru**, Yilun Hua, Yoav Artzi, and Daniel Fried. 2026. Success and cost elicit convention formation for efficient communication. To appear at the *64th Annual Meeting of the Association for Computational Linguistics*

Chapter 2

Background

2.1 Common ground

Common ground is mutual knowledge of beliefs, goals, and suppositions held by participants in an interaction. Clark (1996) argues that common ground provides basis for agents to coordinate joint actions. Without common ground, agents would not be able to act in a coordinated manner. This basis for coordination may come from explicit agreement, precedent, or convention. Language is a type of convention, and a powerful basis for coordinating actions.

As agents interact, and take joint actions and share in perceptual experiences, they update their common ground. Since common ground is part of an agent's private belief, two agents may have discrepencies in common ground that need to be resolved to successfully coordinate. Agents may do this by asking for clarification, issuing corrections, or expressing uncertainty.

Here, I present discuss divisions and types of common ground relevant to work that follows in the rest of the thesis. The account I present is based on Clark (1996).

Divisions of common ground Common ground may be divided into three parts: initial common ground, the current state of the activity, and public events so far.

Initial common ground is background knowledge shared by participants before they enter into a joint activity. This may include various kinds of knowledge. Participants interacting in a conventional natural language like English share knowledge of the language. They may also share knowledge of the world, ranging from intuitive physics to factual knowledge. This may also include knowledge inferred from an participants' identity or role. In an interaction between a doctor and a patient, the doctor is assumed to be an expert in medicine, and (without prior interaction) unaware of what ails the patient. The patient is expected to be forthcoming with relevant information about their condition, and to cooperate with the doctor's efforts to diagnose and treat them.

These expectations also apply to interactions between people and AI agents. Consider an agent operating within an integrated development environment (IDE). The user presupposes some agent abilities, such as the ability to read files and run code. The user also expects the agent to be knowledgable about code syntax, and have general knowledge of programming concepts and practices. The user also assumes that the agent is cooperative,

and will act in accordance with the user's instructions. Similarly, the agent can also presuppose that the user is cooperative, and communicates their intent clearly and faithfully. This assumption is the foundation of Chapter 3.

Participants also assume shared access to the current state of the activity, and the state of the world that allows them to keep track of progress. They reason about an external representation of the state that is assumed to be accessible to all participants. In an interaction to make a purchase at a drugstore, the cashier and customer can both see items available on shelves, and items that the customer has selected that may have been placed on the counter. Similarly, a user interacting with the IDE agent can see the current state of the code. Additionally, they may be able to observe other parts of the state, like the position of the user's cursor, or a piece of code they have highlighted (Chi et al., 2026).

This ability to observe the state of the activity is very useful when iteratively refining an object like a program, a piece of writing, or a design. In Chapter 5 I explore how participants iteratively refine a design. In later turns of the interaction, participants issue instructions that need to be interpreted in the context of the agent's work so far. This idea of refinement instructions being grounded in the current state of the activity also shows up in a variety of other settings such as editing text (Chen et al., 2026) or images (Zhang et al., 2023a).

Finally, participants also share knowledge of public events so far. This includes actions that participants have taken and things they have said. For example, a cashier might keep track of the things a customer wants to buy and whether they have paid. When iteratively constructing an object, participants are aware of the sequence of changes that led to the current state of the object chapter 5. When learning how to perform a task from a teacher's demonstration, the teacher is takes actions that make it easier for the student to follow along (Ho et al., 2016), and the student keeps track of these actions to learn the general procedure.

In addition to sharing knowledge of public events in the current interaction, participants also share knowledge of previous interactions between (or involving) the same participants. A barista might know a regular customer's preferred order, and understand that from the customer just saying "the usual." This shared knowledge of public events is also useful to agents that adapt over time. For example, agents may infer user preferences based on their prior choices (Lin et al., 2022), and how they use the agent's output (Gao et al., 2024). In Chapter 6, I enable agents to use shared knowledge of previous interactions to communicate more efficiently.

Communal and personal common ground Common ground may derive from participants' shared membership in different communities, or from their shared experiences. Communal common ground is shared by members of the same community. These communities may be of speakers of the same language, nationals of the same country, employees

of the same company, etc. For example, two students at Carnegie Mellon University are likely to know that Gates-Hillman Center is acrosss the street from the Tepper School of Business, and that the other student is likely to know this fact too.

Personal common ground on the other hand derives from participants' shared experiences. These may be shared perceptual experiences or joint actions. Shared perceptual experiences are things that participants jointly perceive. Drivers on the road perceive the cars, pedestrians, and traffic devices around them. Joint actions also become part of the common ground of all participants, as in the example of a customer telling a casher what they want to buy. The focus of this thesis is mostly on personal common ground – which accumulates over the course of interactions between users and agents. This involves common ground due to joint actions (like in Chapter 6) or shared perceptual experiences (like in Chapter 5).

While I don't consider agents having common ground due to membership in a community, there is knowledge I consider mutual to the agent and user prior to their interaction. This varies depending on the setting. In Chapter 3, agents and users have mutual knowledge of the semantics of regular expression programs. In Chapter 5 and Chapter 6, agents and users have mutual knowledge of English words, and how they may be be used to describe images. At least part of this knowledge is imparted through pretraining the models. In Chapter 3, I pretrain base models on regular expression data. In Chapter 5 and Chapter 6, I build on multimodal pretrained models (Gemma Team, 2025; Bai et al., 2025).

2.2 Games

A useful formalism for reasoning about communication is to view it as a cooperative game (Lewis, 1969), typically involving two players. The players in this game have a shared goal, and both players win if the goal is achieved. If the goal is not achieved, the players lose. This is in contrast to competitive games like chess or Go, where one player wins and the other loses. By allowing players to send messages to each other to coordinate their actions, we can study how goal-oriented agents with shared interests communicate. This formalism can be applied to a variety of settings where users and agents collaborate to solve a task, and I draw on it extensively throughout the thesis. However, not all uses of language are fully cooperative (Lewis et al., 2017; FAIR et al., 2022).

One type of cooperative game is a speaker-listener game. Here, a speaker intends to instruct a listener to take a specific action. The speaker is allowed to send a message to the listener, who acts upon the message. If the listener takes the action the speaker intended, both players win. In this type of game, the players have asymmetric access to the speaker's intent, and have asymmetric abilities – the speaker can send messages but not act, and the

listener can act but not send messages.

For example, in a reference game, the speaker intends the listener to select one of a number of images, and produces a description of the target image. If the listener selects the target image, both players win. This type of game has been used extensively in natural language processing and computational linguistics. It has been used to model contextual image descriptions (Andreas and Klein, 2016), navigation instruction following (Fried et al., 2018a), code generation from natural language (Zhang et al., 2023b) and more. I use variants of this game (called by different names) in Chapter 3, Chapter 5, and Chapter 6.

Haber et al. (2019) and Udagawa and Aizawa (2019) study a different kind of cooperative game where two players are given overlapping but different views, and they have to coordinate on objects in their shared view. In this case, both players have symmetric abilities, and are free to send messages to each other, but have asymmetric observations.

The third type of game I will discuss here is an assistance game (Laidlaw et al., 2025). In this game, a user and an assistant are present in the same environment, and have the same abilities to act in the environment. The user has privileged access to a reward function, which specifies a goal. The assistant has to help the user achieve their goal. This involves observing the user's actions and inferring the user's reward function, and then acting according to the inferred reward function to assist the user. I consider a similar (but not identical) game in Chapter 7, where both a user and agent can act in the environment, and the agent has to learn from the user's actions.

Part I

Reasoning about Intent

Chapter 3

Inferring Intent by Reasoning about Pragmatic Demonstrations

In program synthesis – specifically programming-by-example (PBE) – a user describes a target program using input-output examples (i.e. test cases) and the synthesizer finds a program that is consistent with these input-output examples. In PBE, the users directly express the semantics of the intended program (what it *should do*) without having to understand the syntax of the program (what it *should look like*). Such systems have found real-world use in a variety of scenarios such as spreadsheet formulas (Chen et al., 2021; Gulwani, 2011) and data wrangling (Feng et al., 2018).

An important aspect of inferring programs from examples is dealing with *ambiguity*. Given a set of examples, there can be many spurious programs consistent with the set, and picking out the right one the user has in mind is a long-standing challenge. For example, when describing the regular expression `a+b*`,[1] an informative user might provide the example $(ab, \checkmark)$ indicating that the string ab matches the target regular expression. However, to the program synthesizer, both `a+b*` and `a*b+c?`[2] would be among many acceptable answers based on this example.

To handle ambiguity in examples provided by users, Pu et al. (2020) frame program synthesis as a joint action between a user and a synthesizer. The user participates in this joint action by choosing an example that can be used by the synthesizer to infer their intended program. In their choice of examples, they coordinate with the synthesizer, choosing examples that are informative to the synthesizer. These examples are interpreted against shared knowledge of the semantics of the programming language (i.e., what programs should do) and the shared goal of finding a program that matches the user's intent, which are part of the common ground between the user and the synthesizer.

This framing allows Pu et al. (2020) to use *pragmatic inference*, specifically, the Rational Speech Acts (RSA) framework (Frank and Goodman, 2012a), to build a program synthesizer that can resolve ambiguity via recursive Bayesian inference. The RSA framework allows the synthesizer to reason about what program a user intended, given that they chose that particular set of examples rather than a different one. For example, the synthesizer could reason that a user that wanted to describe `a*b+c?` would have chosen an example containing the character c. However, this approach requires the synthesizer to perform expensive counterfactual inference over the space of all programs and examples to resolve ambiguity,

[1] 1 or more a s followed by 0 or more b s

[2] `c?` means optionally having a c at the end

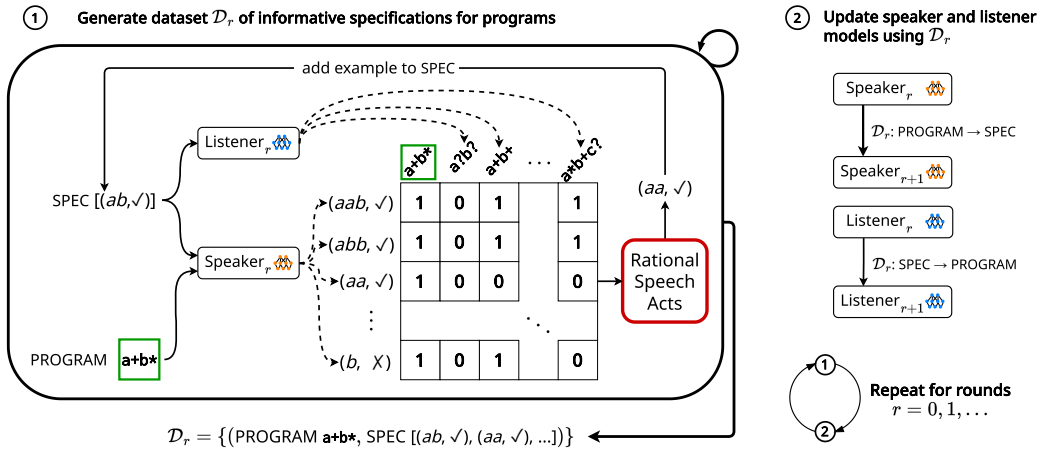


Figure 3.1: PRAX iteratively generates datasets containing increasingly informative program specifications (lists of examples consistent with the program), and updates models on the generated datasets. ① We use a Speaker model — that generates an example consistent with a target PROGRAM — to propose a set of candidate specifications. Using the Rational Speech Acts model of pragmatic reasoning (red box; described in in Figure 3.2), we choose the example that is most informative to a Listener model that synthesizes programs consistent with a given specification. In this manner, we incrementally build the list of examples SPEC for the PROGRAM. We repeat this for different programs to create a dataset of informative PROGRAM-SPEC pairs. ② We use the dataset to update the Speaker and Listener models. We train the speaker to generate the selected pragmatic examples, and the listener to synthesize the target program given the generated examples.

making it difficult to scale to realistic programming domains.

To scale to realistic program spaces, modern approaches of PBE systems have relied on training neural networks to efficiently search through large program spaces for consistent programs given examples (Balog et al., 2017; Devlin et al., 2017). In this chapter, we explore whether we can use simulated reasoning in communication games using the RSA framework as a way to generate training data consisting of pragmatic examples. The generated data is then used to train neural networks to enable scaleable pragmatic program synthesis. We dub this approach PRAX. We hypothesize that since the RSA framework computationally models how a human chooses examples to communicate a program, end users would succeed more often when communicating with a neural synthesizer trained on pragmatic data (our work) as compared to a neural synthesizer trained on non-pragmatic data (Balog et al., 2017; Devlin et al., 2017).

An overview of PRAX is shown in Figure 3.1. We start with a neural *literal listener* — a synthesizer trained in the style of Devlin et al. (2017) — and a neural *literal speaker* that generates examples consistent with a given program. We generate a sequence of pragmatic examples incrementally to obtain a training pair (`program`, examples). This pair is then

added to an aggregate training set, which is used to finetune both the speaker model — making it more likely to generate pragmatic examples — and the listener model — making it more likely to recover the intended program given pragmatic examples.

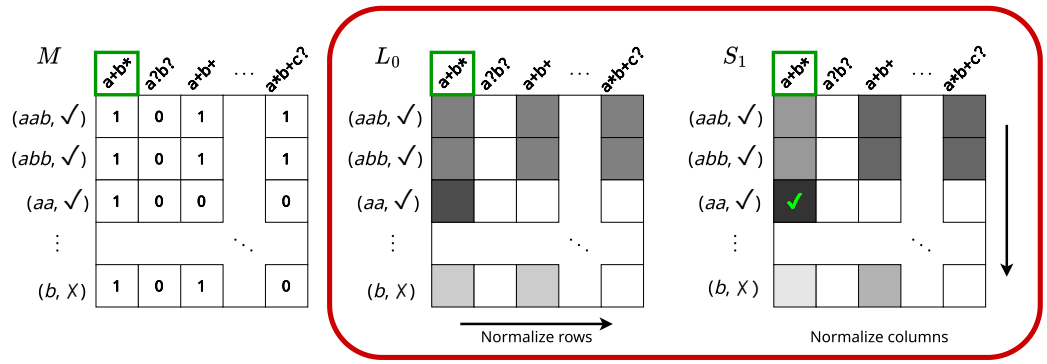


Figure 3.2: An illustration of how the Rational Speech Acts framework is used to select an informative example for a given program. We start with the matrix corresponding to the consistency relation between the sample of programs and examples shown in Figure 3.1. We obtain a literal listener distribution L_0 over programs for each example by normalizing the rows of this matrix. Since the M matrix is binary, each row in L_0 is a uniform distribution over consistent programs in the sample — any of the consistent programs is equally likely to be the intended program. We then obtain a pragmatic speaker distribution S_1 by normalizing the columns of the L_0 matrix: modeling the probability an informative speaker might have for choosing each example when communicating a program to a literal listener. RSA outputs the highest-probability example in S_1 (e.g., (*aa*, ✓)) in the column corresponding to the target program (e.g., a+b*).

We validate the effectiveness of PRAX on the well-studied PBE task of inferring regular expressions from a set of examples. Each example is a pair (*string*, **bool**), indicating whether a particular string matches the regex. To compare our training algorithm to standard supervised learning from human-annotated data, we collect a novel dataset of human annotations, consisting of (`program`, examples) where the examples were given by a person for a total of 440 regular expressions. We find that with only a small number (40) of human annotations – used only for model selection – our method is able to outperform a system that is fine-tuned using 400 annotated regexes from this dataset.

We conduct human evaluation of PRAX by giving a user a target regex to communicate interactively to the synthesizer using examples, and find that the informative examples generated by our procedure substantially improve the performance of a regular expression synthesizer, with improvements of 22.8% absolute (51.4% relative) in accuracy (11 participants, 340 regexes total). PRAX, despite not using human-provided data during training, matches the performance of of a model fine-tuned on a large dataset of human-written pragmatic examples.

3.1 Background

Programming-by-Example In this chapter, we tackle the task of finding a $\texttt{program} \in P$, where P is a space of possible programs. As a *specification* of intent, a user provides a sequence of input-output $\textsf{examples} \in E^+$,[3] where $E = \mathcal{X} \times \mathcal{Y}$ is the space of all possible input-output pairs that programs in P operate over. For example, P may be a space of regular expression programs, $\mathcal{X}$ the space of all strings in the alphabet that the regular expressions are defined over, and $\mathcal{Y} \in \{\checkmark, \times\}$ where output ✓ indicates whether the input string matches the regular expression. For simplicity of explanation, in this section we consider cases where the specification consists of *a single example*, deferring the cases of multiple examples to the next section.

The semantics of programs are captured by the *consistency relation* M between P and E:

$$M = \{(\texttt{program}, \textsf{example}) \mid \textsf{example} = (x, y) \in \mathcal{X} \times \mathcal{Y}, \texttt{program}(x) = y\}$$

A program is consistent with an example iff executing the program on the input produces the intended output. We can view M as a *consistency matrix* where each row corresponds to an example, each column corresponds to a program, and an element is 1 if the program is consistent with the example and 0 otherwise (Figure 3.1 and **??**, left).

Literal model of program synthesis A minimal requirement of a program synthesizer is that it finds *any* program that is consistent with the given specification. We refer to such a synthesizer as the *literal listener* L_0, which naively assigns equal probability to any consistent program.

$$L_0(\texttt{program}|\textsf{example}) \propto M(\textsf{example}, \texttt{program})P(\texttt{program}) \tag{3.1}$$

This literal listener distribution is given by normalizing the rows of the consistency matrix to produce uniform probability distributions (L_0 in Figure **??**). However, this literal listener cannot resolve ambiguity when interacting with users, as it places equal probability on all consistent programs. A literal speaker – one that generates *any* consistent examples for a given program – is defined analogously as $S_0(\textsf{example}|\texttt{program}) \propto M(\textsf{example}, \texttt{program})P(\textsf{example})$.

Pragmatic model of program synthesis When interacting with a synthesizer, users choose examples that are *informative* – those that distinguish the program they desire from others. For example, given the specification $[(\textit{ab}, \checkmark)]$, a user is likely wants the regular expression

[3] Here the notation X^+ indicates a sequence of 1 or more elements belonging to X

`a+b+` than `a+b+c?`, since they probably would have included the character `c` if they wanted the latter expression.

To leverage the informativity of examples, Pu et al. (2020) use the Rational Speech Acts (RSA; Frank and Goodman 2012a) framework to derive a *pragmatic program synthesizer* that resolves ambiguity by modeling how people choose examples informatively. First, they construct a pragmatic speaker S_1 that chooses an example in proportion to the likelihood that it would make the literal listener infer the intended program:

$$S_1(\textsf{example}|\texttt{program}) \propto L_0(\texttt{program}|\textsf{example})P(\textsf{example}) \tag{3.2}$$

Using a uniform prior $P(\textsf{example})$, the S_1 distribution is given by normalizing the columns of the L_0 matrix (S_1 in Figure **??**). As we can see, given a program S_1 selects an example in proportion to the likelihood of L_0 recovering the program given the example, choosing examples that are informative to the listener.[4]

Finally, a pragmatic listener (program synthesizer) L_1 is built on top of S_1:

$$L_1(\texttt{program}|\textsf{example}) \propto S_1(\textsf{example}|\texttt{program})P(\texttt{program}) \tag{3.3}$$

using a prior $P(\texttt{program})$ over programs. This listener resolves ambiguity by choosing that program that an informative speaker (modeled by S_1) would have described using the chosen example.

Pu et al. (2020) demonstrated that building a pragmatic synthesizer L_1 in this way allows for users to communicate the target program to the synthesizer using fewer examples without training a model on human-produced examples or explicitly defining a prior over the space of programs. However, in realistic domains, enumerating large numbers of programs and examples is intractable, preventing the application of this framework to a broader range of tasks.

3.2 Method

In this section, we describe the iterative process by which we bootstrap a pragmatic neural program synthesizer by generating informative specifications, without human supervision (Figure 3.1).

[4]For a sequence of examples, Pu et al. (2020) propose factoring the pragmatic speaker distribution autoregressively as $S_1(\textsf{examples}|\texttt{program}) = \prod_{i=1}^{N_{examples}} S_1(\textsf{example}_i|\texttt{program}, \textsf{examples}_{:i})$

3.2.1 Speaker and listener Models

We build on past work that uses neural models as specification-conditioned proposal distributions over programs (Balog et al., 2017; Devlin et al., 2017). Our listener (synthesizer) models represent distributions over programs $L_\theta(\texttt{program}|\textsf{examples})$. Our speaker (specification generation) models generate the sequence of examples in a specification autoregressively: $S_\phi(\textsf{example}_i|\texttt{program}, \textsf{examples}_{1:i-1})$.[5] While all our listener and speaker models share the same architecture and initialization, we vary their training data, as described below.

3.2.2 Training base models

As a foundation for our approach, we train *base* listener and speaker models to approximate literal speakers and listeners S_0 and L_0 (Equation (3.1)). Since we cannot enumerate the consistency matrix completely and normalize rows, we obtain these approximate models by training on data obtained by randomly sampling an input from the space of inputs $\mathcal{X}$, and executing the program on the input to obtain the output (e.g., by checking if an sampled example string is matched by a sampled regular expression). This lets us generate as many samples from M as we can, which we can use to train a base listener to approximate L_0 (Equation (3.1)), and a base listener to approximate an analogous S_0, using standard maximum likelihood training. This is essentially the method proposed by Devlin et al. (2017). We denote the resulting base listener model as L_{θ_0} and the base speaker model as S_{ϕ_0}, and use these as the initial models in our iterative model bootstrapping procedure.

3.2.3 Generating informative examples

The crux of our algorithm is using the existing S_ϕ and L_θ to approximate S_1, which can then be used to generate training data to improve S_ϕ and L_θ over rounds of training. At each round r of our approach, we use the current speaker and listener models, together with the RSA procedure, to create a dataset of informative examples specifying programs (① of Figure 3.1). We incrementally generate examples to create a specification. Given a partial specification of i examples $\textsf{examples}_{1:i}$, we sample a set of additional candidate examples: $S_{\phi_r}(\textsf{example}_{i+1}|\texttt{program}, \textsf{examples}_{1:i})$. Similarly, we sample a set of alternative programs: $L_{\theta_r}(\texttt{program}|\textsf{examples}_{1:i})$ from the partial specification.[6]

We can then compute the sampled consistency matrix over the generated examples and programs, and use RSA inference as shown in Figure 3.2 to choose the highest scoring

[5] We train the speaker to predict the input-output pair to encourage the model to capture aspects of the domain semantics

[6] We follow prior work (Pu et al., 2020) and impose a *uniform*, rather than learned, prior over this sample.

example from the approximate S_1 distribution.[7] This example is added to the partial specification, and we repeat until a maximum number of examples are reached.[8] The completed program-specification pair is then added to a dataset $\mathcal{D}_r$ of examples from that round of training. This process amounts to choosing an example proposed by the current speaker model that minimizes ambiguity among programs that the current listener infers to be likely.

3.2.4 Model updates

We use the dataset $\mathcal{D}_r$ to update both the speaker and listener models as sketched in part ② of Figure 3.1 using standard maximum likelihood training. In each round r we further fine-tune the speaker and listener models on the generated data to obtain the updated parameters θ_{r+1} and ϕ_{r+1}. To select the model that works best with human-provided examples, we choose the model that maximizes a model selection metric computed over a small set of programs paired with human-provided examples.[9] Note that this validation set is never used to update the model parameters, and only is used to choose a model.

3.3 Experiments

3.3.1 Regular expressions

We validate the training algorithm we propose on the task of synthesizing regular expressions ('regexes') as formally defined in Section 3.1. We use the regular expression domain-specific language presented by Ye et al. (2020). In addition to defining a regular expression specification language, they also define a sampling distribution over the space of regular expressions that we use to sample programs for training and evaluating our model. This distribution uses templates that generalize types of regular expressions that people ask about on fora such as StackOverflow. Further details are provided in Section A.1.

[7] Ideally, one could perform exact inference to draw samples from S_1 directly. As stated earlier, this is intractable. Therefore, we first sample a subset of the rows and columns in the consistency matrix, then perform the RSA inference over this much smaller and denser sampled matrix. However, the consistency matrix M is sparse (mostly 0s) – most programs are inconsistent with any non-trivial set of examples – allowing for reasoning about a sample of the matrix.

[8] Since the models are used only to generate the programs and utterances that are used to create the lexicon, we can draw examples from other sources, including models other than $S_{\phi_r}(\mathsf{example}_i | \texttt{program}, \mathsf{examples}_{:i})$ and $L_{\theta_r}(\texttt{program} | \mathsf{examples}_{:i})$.

[9] This amounts to performing early stopping on the validation metric.

3.3.2 Models for comparison

Base models We use ByT5-small models (Xue et al., 2022) as the backbone for all speaker and listener models. To obtain the base speaker S_{θ_0} and listener L_{ϕ_0} models that approximate the literal speaker and listener respectively, we use a set of 300,000 randomly-generated program–specification pairs (with varying numbers of examples in each specification) and finetune the pretrained ByT5 checkpoint. Full details of training are provided in Section A.2. The L_{θ_0} model acts as the LITERAL model in our experiments.

PRAX We start with the base models S_{θ_0} and L_{ϕ_0} and use the iterative data generation and finetuning algorithm to obtain a sequence of synthesis models L_{ϕ_r} for rounds $r = 0, \ldots, R_{max}$. We use the TOP-1 metric evaluated on a small validation set to choose the best model which we refer to as the PRAX model.

Finetuning on human-provided specifications We obtain an HFT model by fine-tuning L_{ϕ_0} on a curated high-quality human-provided specifications (Section 3.3.5). This model allows us to compare how well our approach of using model generated informative examples compares with sourcing more expensive human annotations.

GPT-3.5 We evaluate GPT-3.5 by using the program-specification pairs we obtain as users interact with the other three models, and revealing each specification to the GPT-3.5 model in the order the user provided them, one example at a time. We stop when the model guesses the correct regular expression, or when all the examples are presented.We can think of this as a form of interaction where the human doesn't observe the outputs of this model while giving examples. Further details of how the model is prompted are in Section A.6.

Inference Crucial to our approach is the ability to generate programs from examples, and vice versa. To generate programs from a specification (sequence of examples from either self-play or human), we present it to the listener, and sample 500 programs using top-p sampling (Holtzman et al., 2020) with $p = 0.9$. We then deduplicate the set of sampled programs, and filter out programs inconsistent with the given specification. We can then sort the remaining programs by their score under the model to obtain a ranked list of consistent programs. Similarly, to generate specifications from a program, we sample 500 examples using top-p sampling with $p = 1$ and check that the examples are consistent with the program.

Model	TOP-1@10 (SE)	TOP-10@10 (SE)	EDIT DISTANCE $\leq$ 1@10 (SE)
LITERAL	0.434 (0.047)	0.522 (0.047)	0.513 (0.047)
GPT-3.5*	0.074 (0.014)	0.189 (0.021)	0.082 (0.015)
HFT	0.587 (0.047)	0.623 (0.047)	0.614 (0.047)
PRAX	**0.661** (0.043)	**0.703** (0.042)	**0.694** (0.042)

Table 3.1: Success metrics at the end of 10 turns of interaction with each model, with standard errors computed using bootstrap sampling. * indicates that the results are in replay.

3.3.3 Procedure

We evaluate the model on the basis of successful communications on 11 human participants. A sampled regex p is given to a human participant, whom describes it using a sequence of examples, providing one example in each turn for upto a maximum of 10 turns. The synthesizer takes the examples provided and generates a ranked list of inferred programs, of which the top-1 regex p' is shown to the participant. The communication is successful when $p = p'$, at which point the interaction ends.[10] A total three synthesizers were considered – the LITERAL model, the human fine-tuned HFT model, and the PRAX model. The identity of the models is referred to the users only as differently colored robots. The study yielded communication history over a total of 340 regexes (109 to the LITERAL model, 113 to the HFT model, and 118 to the PRAX model).[11]. Further details about how the user study is conducted are provided in Section A.5.

3.3.4 Measurement

We consider the following metrics. TOP-1@t measures whether the model's top-1 matches intended regular expression at any point at turn t of the interaction. We can also consider the average value of TOP-1 by aggregating across the turns $t \in \{1, \ldots, 10\}$. Averaging across turns rewards models that pass success criteria given fewer turns — a model that can infer the target regex at turn 4 on average is better than a model that can only infer the target at turn 10. We use TOP-1 over a validation set as the model selection criterion for our proposed method. TOP-10@t and TOP-10 are similarly defined. EDIT DISTANCE $\leq$ 1@t measures whether the model's highest scoring prediction in any of the turns up to t is at most a 1 token edit from the intended program, and EDIT DISTANCE $\leq$ 1 is the average of this value over $t \in \{1, \ldots, 10\}$.

[10] We used the `greenery` Python library to identify regex matches in terms of semantic similarity, and not just surface form match.

[11] a small bug caused us to collect few extra interactions for some of the models, which does not change the measurements on the models' relative performances

Model	TOP-1 (SE)	TOP-10 (SE)	EDIT DISTANCE ≤ 1 (SE)
LITERAL	0.233 (0.028)	0.333 (0.034)	0.296 (0.031)
GPT-3.5*	0.048 (0.010)	0.122 (0.014)	0.056 (0.011)
HFT	0.349 (0.032)	0.424 (0.035)	0.390 (0.034)
PRAX	**0.373** (0.028)	**0.430** (0.030)	**0.432** (0.031)

Table 3.2: Average success metric over 10 turns. We see that the PRAX training method we propose is on par with HFT, and outperforms other baselines significantly for all success criteria. * indicates that the results are in replay.

3.3.5 Human-provided specifications

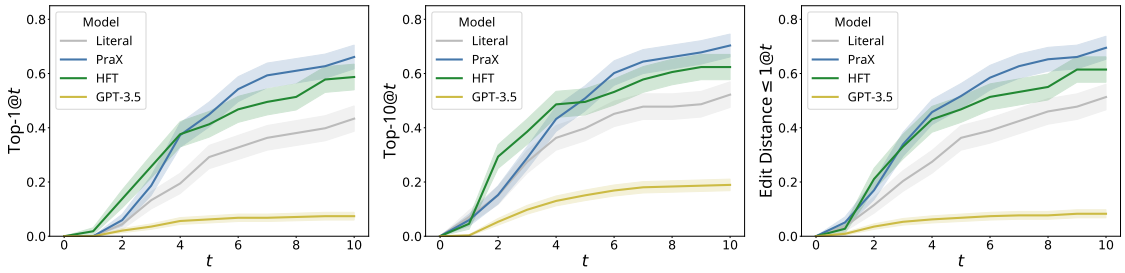
An alternative to generating informative examples using the method we propose is to have human annotators provide examples. We collect a new dataset of high-quality program-specification pairs.

Procedure We present a participant with a sampled regular expression, and instruct them to provide examples that they might use to illustrate the regular expression to another person. Participants are asked to provide at least 5-7 examples. We verify whether the examples are informative by checking whether a different annotator is able to identify the program which the given set of examples describes. Further details about the data collection process are presented in Section A.4.

Usage of data We collect a total of 440 program-specification pairs. We sample a small subset of 40 pairs that received 2 "correct" verifications as a validation set for model selection. We use the other 400 pairs as a training set to finetune the L_{θ_0} models on human-provided informative examples, obtaining HFT (see Section A.2).

3.3.6 Results

Table 3.1 shows the rate of success for different models at the end of 10 turns of interaction. We see that training on informatively examples results in large gains in performance. Both the PRAX and HFT models significantly outperform the literal model for all three criteria of success. Looking at the aggregate success rate across turns in Table 3.2 reveals that it is not just that the PRAX synthesizer eventually catches up to the HFT model, but also performs on par with it over the course of the interaction. Figure 3.3 shows the progression of each metric over the course of interaction. In contrast, we see that GPT-3.5 performs worse than the literal model. One reason for this could be that the distribution of regular expressions that the GPT-3.5 encounters in its training data could be quite different, leading



(a) (b) (c)

Figure 3.3: Performance of various models as a function of turns, measured in (a) TOP-1@t, (b) TOP-1@t, and (c) EDIT DISTANCE $\leq$ 1@t. Lines show averages, and bands are standard errors. Our model PRAX, trained entirely from self-play and RSA inference without using human-provided data performs better than the non-pragmatic LITERAL model across all turns and metrics, and matches the performance of HFT tuned on a human-provided examples.

Target program	Examples	LITERAL	PRAX
`4A{2,}`	*4AAAAAAAAAAAAAAAAA* ✓ *4AA* ✓	`4A{1,}`	`4A{2,}`
`[A-Z]{1,}i{2,4}`	*Aiii* ✓ *Bii* ✓ *BBiiii* ✓ *AAAAAAAAAii* ✓	`(A{1,}|B{1,})i{2,4}`	`[A-Z]{1,}i{2,4}`

Figure 3.4: Example specifications for two programs provided during the user study, along with the highest ranked guess from the LITERAL and the PRAX models.

to worse performance. In conclusion, the experiments validate our hypothesis that humans communicate more effectively with the model trained on informative examples (the PRAX and HFT models) than with a model trained on randomly chosen examples (the LITERAL model).

Examples of synthesized programs Figure 3.4 shows examples of guesses by the LITERAL and PRAX models given the same sequence of examples. In the first case, we see that the PRAX model is able to infer that if a user wanted a regular expression that accepted *4A*, they would have specified that, and instead correctly guesses that the user wanted at least two *A* s in the string. The second example also shows how the LITERAL model synthesizes a regular expression that is correct, but is too specific, while the PRAX model recovers the correct generalization.

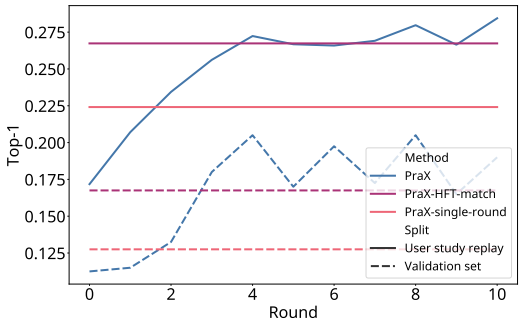


Figure 3.5: TOP-1 metric over the course of rounds of training of the PRAX model. We report the metric on the validaton set as well evaluating on all interactions from the user study in the replay setting (similar to how we evaluated GPT-3.5). We compare the accuracy over rounds of training to generating specifications and updating the models only once, amounting to a single round of the procedure with more programs (PRAX-single-round). We also compare to fine-tuning the base model on 400 pairs (same number as HFT) generated by the speaker in the 5th round of training (PRAX-HFT-match) to assess the quality of our speaker-generated examples.

3.4 Analysis of training

Figure 3.5 shows the progression of the TOP-1 metric over the course of different rounds of training. We see that as we train the model for more rounds, the performance of the model generally increases, and then tapers off. This shows that as the model is trained for more rounds, it gets increasingly pragmatic. Since the model we choose for the user study is trained for 5 rounds, on $5 \times 1024 = 5120$ programs, we also compare to training the model for only a single round on the same number of programs. In a replay study (similar to how we evaluated GPT-3.5; Figure 3.5), we find that iteratively generating data and updating the model performs better. We also see that finetuning the base model on 400 examples (to match the HFT setting) from a later round of training also results in a strong model, suggesting that as the speaker is trained more, it generates examples that are useful to finetune a listener model.

3.5 Related work

Neural network models of pragmatic reasoning Prior work has applied the the RSA pragmatic reasoning framework to improve neural models at inference time for tasks including image captioning (Andreas and Klein, 2016; Cohn-Gordon et al., 2018), instruction generation (Fried et al., 2018a), vision-and-language navigation (Fried et al., 2018b), and machine translation (Cohn-Gordon and Goodman, 2019). RSA is used at inference time to

re-rank multiple outputs from the neural models. PRAX has two advantages over these works: (1) it requires no human-provided data during training; (2) it uses RSA at training time via data generation, amortizing expensive RSA computaion.

Other approaches have used pragmatically-motivated training procedures. The closest works to ours are White et al. (2020) and Lazaridou et al. (2020), who use reinforcement learning approaches to fine-tune a speaker model using reward from a fixed listener model. Monroe and Potts (2015) and McDowell and Goodman (2019) backpropagate through the RSA procedure at training time to reason counterfactually about pragmatically produced utterances from humans. Again, PRAX is unique in that it does not require human-provided training data.

Finally, Andreas and Klein (2016) find that amortizing pragmatic reasoning during training does *not* perform as well as explicit pragmatic reasoning during inference in the domain of image captioning. We demonstrate that amortization is in fact effective for the domain programming-by-example.

Pragmatic reasoning for program synthesis Similar to our work, Vaithilingam et al. (2023) conduct a study of how users interact with an exact RSA pragmatic regular expression synthesizer over a toy domain of ∼1000 regexes total over strings of only 0s and 1s. Pu et al. (2023) propose a way to make pragmatic PBE more efficient by inferring a global ranking function, but still relies on the expensive exact RSA during training. Our approach is different in that by using neural models for speakers and listeners *at training time*, we are able to scale to a realistic regex domain. Pertseva et al. (2022) also present an approach to version space algebra-based approach to regular expression synthesis from examples by explicitly modeling the probability of examples describing programs (as in our speaker models), but they work with only positive examples (a subset of our example space with examples that have the output ✓, excluding those with the output ✗). Ferreira et al. (2021) present an SMT-based method that reasons about distinguishing inputs to synthesize regular expressions.

3.6 Conclusion

We present PRAX, a novel algorithm that bootstraps pragmatic program synthesizers by (1) generating datasets using self-play between a speaker (`program` → examples) and a listener model (examples → `programs`), and (2) training on the generated data. Crucial to our approach is the use of *pragmatic inference* to make the generated data more informative. PRAX produces pragmatic program synthesizers with minimal supervision: in a challenging regular expression domain, matching the performance of synthesizers fine-tuned on human-

produced examples, despite not using any human-provided data during training. Future work might explore scaling pragmatic program synthesis to open-ended Python code generation, and application to multimodal specifications — e.g. with natural language and examples (Ye et al., 2020).

Chapter 4

Learning from Information Asymmetries in Simulated Interaction *(proposed)*

One way to improve how agents backed by large language models interact with people is to train them on interactions with people. This could involve training agents to imitate desirable responses to user queries (Chung et al., 2024), providing agents with feedback on their responses that can be used for reinforcement learning (Ouyang et al., 2022), or extracting naturally occurring feedback from interactions with people like expressions of sentiment (Chen et al., 2025b). However, each of these settings requires getting agents to interact with people, which is resource intensive, and may be infeasible in some settings.

An alternative that has emerged is to train agents in simulated interactions with another agent. This might involve having a model provide feedback on specific aspects like response harmlessness (Bai et al., 2022) for single-turn interactions. For multi-turn interactions where the agent and user add to common ground over the course of the interaction, it involves having another agent simulate a user that interacts with the agent.

The model simulating the user is given access to some privileged information, and conditions its responses on this information. In task-oriented settings, this may be the solution to the task (Cook et al., 2026; Zhou et al., 2025a; Wu et al., 2025), or a fully-specified version of an initially underspecified instruction (Sun et al., 2025). The speaker model from Chapter 3 followed the same pattern by conditioning on the target program. Agents are trained to maximize rewards in multi-turn interactions with this user simulator, allowing them to engage in collaborative interactions with people.

However, giving a user simulator access to a complete specification or reference solution reduces the uncertainty that people engaging in multi-turn interactions have to reason about. The default tendency of language models used as simulators is to regurgitate all the information available up front, since it is available in the prompt. Approaches that train with a user simulator often counter this by prompting the simulator to be intentionally vague, and withhold information from the agent until it is specifically requested. These approaches may not reflect how people communicate under uncertain knowledge of the world or their preferences.

As an illustration, consider a user simulator tasked with collaboratively creating the following target code for calculating the n^{th} Fibonacci number:

```
def fibonacci(n):
    if n <= 1:
```

```
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

The simulator instructs the agent in the first turn to *"Write a Python function to calculate the nth Fibonacci number"*, and receives the following subtly incorrect implementation:

```
def fibonacci(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(1, n + 1):
        a, b = b, a + b
    return b
```

A simulator that is prompted with the target code snippet can immediately recognize the error in the bounds of the loop without additional inference or reasoning. However, without access to the code, it might initially be uncertain about the source of the error – could the initial conditions of `a, b = 0, 1` be incorrect? Could the issue be returning `b` instead of `a`? Identifying the source of the error and giving correct feedback would require effort on behalf of the user simulator to reason through the execution of the code and identify where and why it might fail, without the source of the error being immediately obvious from comparing the code to a reference solution.

A user may also refine their intent based on the agent's initial attempt. For example, a user may have expected a recursive implementation of the Fibonacci function when they issued the instruction in the example above, but only seek to fix the off-by-one error in the loop bounds upon seeing the agent's iterative implementation.[1] This gap only widens as the complexity of the task increases. A real user trying to implement a complex piece of software does not operate as a teacher trying to guide the agent to a particular solution that they have completely fleshed out in advance.

I hypothesize that agents trained in simulations with a model that has access to privileged information will communicate differently than agents trained in simulations with a model that does not have privileged information. I propose systematically investigating this hypothesis. The setting where the user simulator does not have access to the solution resembles that in Ni et al. (2025). In their work, they propose an algorithm that incentivizes producing utterances that increase the other agent's belief in the correct answer. Their

[1] Note that this example is only illustrative. A different approach to presenting privileged information (such as using test cases) may not run into this specific issue, but still run into the same broader problem.

training algorithm does lead to improved collaborative reasoning performance, but they don't systematically investigate the influence of information asymmetry.

4.1 Proposed setup

I will create a framework to compare the communicative behaviors that emerge from training in simulated interactions. I will choose a particular algorithm proposed for training interactive agents (Cook et al., 2026). As a domain, I propose using math tasks (like Cook et al. (2026)). These tasks have verifiable solutions that can be checked against a reference solution, or by a sufficiently capable model.

With the framework in place, I plan to conduct controlled experiments. To illustrate, consider training an agent based on the Gemma-3-12B model (Gemma Team, 2025). I plan to train variants of the Gemma-3-12B-based agent with different user simulators that vary in the model used to simulate the user (Gemma-3-12B, Gemma-3-27B) and whether the user has access to the solution (yes, no). I will then evaluate each of these variants in interactions with a third model that is not used during training, and is more capable, such as Gemini-3-Flash (Gemini Team, Google, 2025). At evaluation time, I will vary whether the user simulator has access to the solution.

First, I hypothesize that agents that rely on feedback from a user simulator that accesses privileged information will not generalize to situations where the user or user simulator does not have access to the solution. I aim to test this by studying how well agents trained to interact with users that have privileged information work with users that do not have access to the solution, for users of various capability levels. Second, I aim to characterize how agents trained in different settings communicate. For this, I propose performing analyses like Zhou et al. (2026), who analyze user simulator language by measuring attributes like the front loading of information in earlier turns, the amount of repetition, clarification seeking, expressions of uncertainty, etc. I hypothesize that agents trained in different settings will exhibit different patterns of these metrics. Note that Zhou et al. (2026) propose analyses for τ-bench user simulators, and I will have to choose appropriate ones for the math setting I aim to investigate.

By conducting this investigation, I hope to shed light on how assumptions baked into training procedures affect how agents engage in multi-turn interactions, and navigate uncertainty held by the agent and the user. This would pave the way to training procedures that allow us to leverage the flexibility and scale of simulated interactions, while being faithful to the constraints of communicating with real people.

Part II

Reasoning about Shared Observations

Chapter 5

Multimodal Interaction for Collaborative Construction

Recent advances in generative AI allow people to ask for the creation of artifacts in natural, conversational language. However, while many models generate high-quality output in response, they often struggle to process complex, multi-turn requests (Laban et al., 2025). This challenge is especially pronounced in tasks such as image editing (Fu et al., 2024) and code generation (Cassano et al., 2023), resulting in frustration with AI-generated content that is "nearly there" but difficult to *refine* to precisely match user intent.

On the other hand, the ability of humans to iteratively refine communicated concepts is a core mechanism used to ensure robustness of communication (Dingemanse and Enfield, 2024), as well as a crucial part of the design process Lawson (2006); Williams and Cowdroy (2002). Central to the ability to iteratively refine an artifact is the ability to reason about common ground. Refinement instructions are given in the context of the current state of the artifact. These instructions may also be *multimodal*, using means like gesture or drawing to make their meaning precise. Agents that collaborate in these settings need to be able to reason about common ground to understand the refinement instructions and execute them correctly.

We present *mrCAD*, a resource for studying and benchmarking multimodal refinement communication in a computer-aided design (CAD) setting. Our task (Figure 5.1) elicits the following phenomena: (1) contextual dependence, where the refinement instructions must be interpreted in the context of the current CAD (Section 5.3); (2) interactions between modalities, where drawings and texts must be interpreted together (Section 5.3); and (3) monotonicity, where participants communicate to iteratively shrink the distance between the current CAD and the target CAD (Section 5.3 and Section 5.4). These phenomena capture how humans can communicate *flexibly* to accomplish *precise* tasks such as producing CAD designs.

We highlight two main uses of mrCAD: (1) as a **benchmark** for evaluating multimodal, multi-turn instruction following systems such as those powered by vision-language models, and (2) as a resource for **understanding** the nature of human refinement communication.

Benchmarking CAD reconstruction is particularly attractive as an instruction following benchmark for the following reasons: (1) The CAD reconstruction accuracy can be programmatically defined, rather than relying on learned metrics such as CLIP Radford et al. (2021). (2) CAD programs can be easily manipulated using clicks and drags (similar to a slide

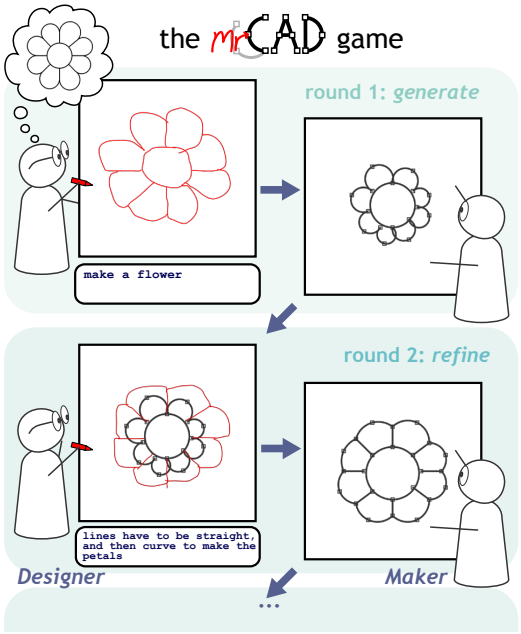


Figure 5.1: We present mrCAD, a dataset of humans playing a multi-turn, multimodal communication game in a 2D CAD environment. A pair of participants collaborated to recreate a target CAD over multiple rounds. The target design is known only to the *Designer*, who instruct the *Maker* using drawing and text. The *Maker* manipulate the current CAD based on these instructions.

design software), making it easy to evaluate both tool-using agents and human participants. (3) The target CAD designs in mrCAD are sourced from SketchGraph Seff et al. (2020), consisting of naturalistic 2D designs created by human designers, rather than synthetically generated from a domain-specific language that the authors have crafted by hand.

Understanding As a resource for understanding the nature of multimodal refinement in human-human collaboration, mrCAD is large scale, containing 6,082 human-to-human plays of the game, with a total of 15,163 rounds of instruction. A large number of CADs in our dataset are recreated *multiple* times by different dyads, ranging from 2 to 30 games per design, capturing the natural variation in refinement strategies across people. A notable feature of CAD is that while it only contains simple curves, they combine form rich semantic objects and sub-parts (like the "flower" and "petals" in Figure 5.1), which must be parsed contextually Ji et al. (2022a). This is in contrast to works such as Zitnick and Parikh (2013), where each object in the scene has a canonical, pre-defined name.

Our work makes the following contributions:

- A novel dataset of multimodal refinement of CAD designs.
- A detailed procedure for collecting a dataset of multimodal refinement communication

using crowd-sourcing.

- The dataset and benchmark, consisting of 15,163 instructions and executions, wrapped in an accessible gym environment.
- Analyses of human-human communication that reveal differences in the multimodal languages of generation and refinement.
- Evaluation of existing VLM models, revealing a severe gap in their refinement ability, particularly compared to their generation ability.

5.1 The mrCAD Task

The mrCAD task (Fig. 5.1) is a two player, multi-turn communication game. Following McCarthy et al. (2024), a *Designer* issues multimodal instructions to guide a *Maker* to take actions in a mutually observable CAD environment to construct a target CAD, known only to the *Designer*.

5.1.1 mrCAD environment

State The state D is a CAD data structure.

```
D -> {Curve, ...}
Curve  -> Line | Circle | Arc
Line   -> l(P,P) // end points
Circle -> c(P,P) // points on diameter
Arc    -> a(P,P,P) // start, mid, end
```

Each curve is defined by its *control points* that the *Maker* selects. A `render` function allows players to view the state by rendering the design as an image.

Action The *Maker* can alter the CAD by making, removing, and moving curves via translation. They can also change the shape of a curve by moving its control points. If multiple curves share the same control point, moving the point would modify all the curves, and deleting the point would delete all the adjacent curves.

```
Action -> make_curve Curve
        | remove_curve Curve
        | move_curve Curve Vxy
        | move_point P P
        | delete_point P
```

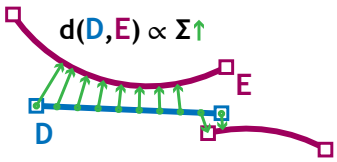


Figure 5.2: The **asymmetric Chamfer distance** from CAD D to CAD E is calculated by sampling 10 points on every curve of D, and calculating (symbolically) the minimum distance from each point to E. Each distance is then normalized by multiplying $\frac{1}{4}$ of the maximum size of the canvas, making it invariant to the canvas size. These distances are summed, making the asymmetric Chamfer distance. The symmetric Chamfer distance we use is the average of both directions.

Transition Actions $\vec{a} = [a_1, a_2, \ldots, a_n]$ can be applied to a design, which results in a new design.

$$D' = \vec{a}(D) = [a_1, a_2, \ldots, a_n](D) = (a_n \circ \cdots \circ a_1)(D)$$

Distance metric CADs are programmatic entities that exist in a visual space, where very different elements could be used to create two CADs of similar appearance. For example, a series of straight lines can be used to represent a curve. Therefore, we devise a distance metric Δ based on Chamfer distance Butt and Maragos (1998) that respects the vector-like nature of CADs while also accounting for geometric similarity (Fig. 5.2).

Instructions The *Designer* can instruct the *Maker* using multimodal messages.

```
Message -> ⟨Text, Drawing⟩
Text    -> [char, ...] | empty
Drawing -> [stroke, ...] | empty
```

A text is a sequence of characters and a drawing is a sequence of strokes, encoded in SVG format.

5.1.2 Playing the Game

The shared goal of the *Designer* and *Maker* is to collaboratively reconstruct a target design, which is only given to the *Designer*.

Round and rollout A round is a tuple of: a design D, the message from the *Designer* m, the actions generated from the maker $\vec{a}$, and the resulting updated design $D' = \vec{a}(D)$. A

rollout is a sequence of rounds.

$$r_i = (D_i, m_i, \vec{a}_i, D'_i)$$
$$R = [r_1, \ldots, r_n]$$

Each rollout has $D_1 = \{\}$ and $D_i = D'_{i-1}$. A game is "won" if the final design is within a certain threshold θ of distance from the target D^*.

$$\Delta(D'_n, D^*) < \theta$$

Designer At round i, the *Designer* is given both the target and current designs, rendered as images of D^* and D_i, along with the history of the (rendered) interaction of previous rounds $\texttt{render}(R_{1:i-1})$, and generates a message m from

$$P_{\text{designer}}(m \mid \texttt{render}(D^*), \texttt{render}(D_i), \texttt{render}(R_{1:i-1}))$$

Presenting the *Designer* only the rendered CADs encourages them to focus on the geometric properties and communicate the refinements in a naturalistic manner, instead of communicating about the underlying programmatic representation itself.

Maker The *Maker* takes in the message, the current design, the interaction history, and generates a sequence of actions $\vec{a}$ from

$$P_{\text{maker}}(\vec{a} \mid m, D_i, R_{1:i-1})$$

Play Playing the game starts with a target design D^* and involves a dyad (a *Designer-Maker* pair) acting in turns to generate a rollout.

$$R = \texttt{play}(D^*, P_{\text{designer}}, P_{\text{maker}})$$

5.2 Effects of Multimodality and Refinement on Communication

We first confirm that multimodality and refinement do indeed play important roles in communication of designs. We conduct an ablation study by manipulating the kind of communication schemes available to the participants (details in Section B.2.1):

1. **multimodal + refinement:** (original condition): 3 rounds total. Each round consists of instruction (30s) and execution (120s).
2. **drawing-only + refinement:** the *Designer* can only draw but not use text.

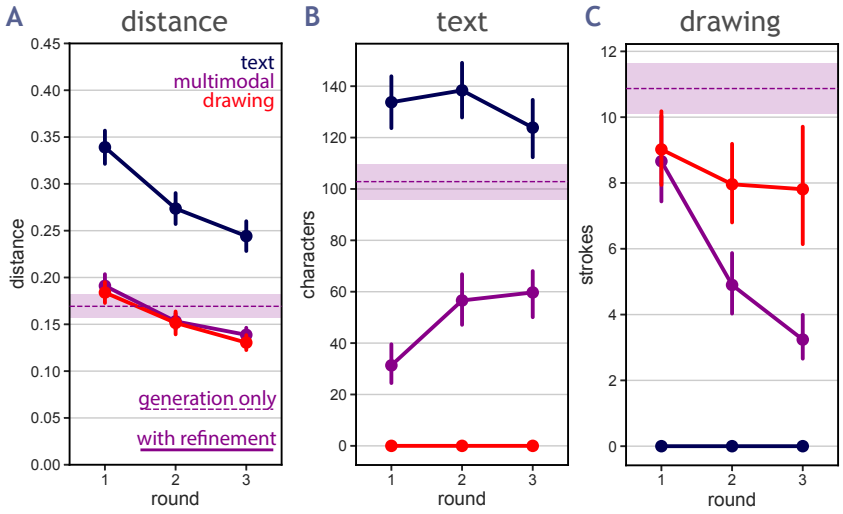


Figure 5.3: **A**: reconstruction accuracy for the 4 communication conditions — multimodal+refinement, text only + refinement, drawing only + refinement, and multimodal + generation only. Using text only was less effective. Generation-only was less effective. Drawing only and multimodal are comparable in performance. **B**: usage of text across rounds — in the multimodal condition, participants used more texts in the later refinement rounds, suggesting a usage of text in conjunction with drawings to communicate refinements. **C**: usage of drawing across rounds — in the multimodal condition, participants used more drawing in the generation round, and less in the refinement rounds.

3. **text-only + refinement:** the *Designer* can only use text but not draw.

4. **multimodal + generation-only:** the pair of participants had 1 big round of instruction (90s) and execution (360s), so they can only generate at once but not refine.

Results are summarized in Figure 5.3. In conclusion: (1) drawing is important, and (2) refinement is important. Note that while drawing-only performed as well as multimodal in this study (Figure 5.3A), in the multimodal condition participants still used more text and less drawing over time (Figure 5.3B,C). This points to a preference to use more drawing in communicating generation and more text in communicating refinements. Statistical analysis of results are given in Section B.2.2.

5.3 The mrCAD Dataset

We detail the collection of the dataset, present the main statistics of our dataset, and a set of basic analyses that reveals the striking effects of multimodality and refinement on communication. We also include a gallery of multimodal refinement rounds (Figure 5.5), highlighting the intriguing yet unexplored aspects of the dataset for future research.

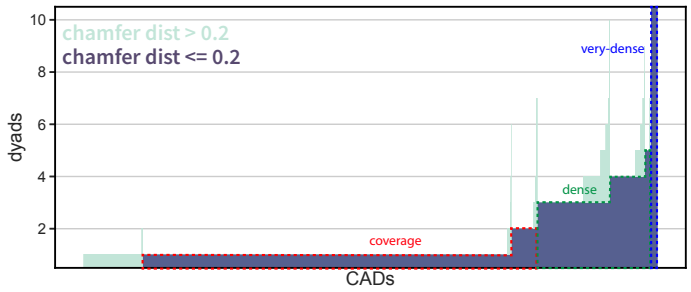


Figure 5.4: The mrCAD dataset contains three subsets: the **coverage set** of 2249 CADs with 1-2 successful rollouts, **dense set** of 698 CADs with 3+ successful reconstruction, and the **very-dense set** of 27 CADs with 30+ successful reconstruction.

5.3.1 Collection

Our goal is to collect a large number of high-quality, multimodal refinement instructions. We designed an online platform for crowd workers to play the mrCAD game (Section B.3.1) and recruited players on Prolific. We seeded the mrCAD communication games with target designs created by people (Seff et al., 2020; Section B.3.4). We then presented these games in sequence to crowd workers, taking measures to allow skilled players to play longer while limiting the annotations from unskilled players (Section B.3.5). We also designed a dynamic submission threshold to encourage more refinement instructions (Section B.3.6) as well as a fixed threshold on final reconstructions to exclude poor reconstructions (Section B.3.7).

5.3.2 Dataset statistics

mrCAD contains 3166 unique CADs recreated by 1092 dyads. This resulted in a total of 6082 rollouts (for a sample, see Figure 5.5A) with a total of 15163 rounds of instructions and corresponding executions. On average, each rollout has 2.49 rounds, and there are a total of 6078 generation rounds (round 1) and 9085 refinement rounds (rounds 2+). To achieve coverage over diverse CADs while also capturing variance in human communication Ji et al. (2022a), our dataset contains three distinct subsets: a **coverage set**, containing 2249 unique CADs each successfully reconstructed by 1–2 dyads; a **dense set**, containing 698 unique CADs each successfully reconstructed by at 3–6 dyads; and a **very-dense set**, containing 27 unique CADs successfully reconstructed by at least 30 dyads (Figure 5.4). In the following analyses we combine data from the coverage and dense sets, and leave the very-dense set for future work.

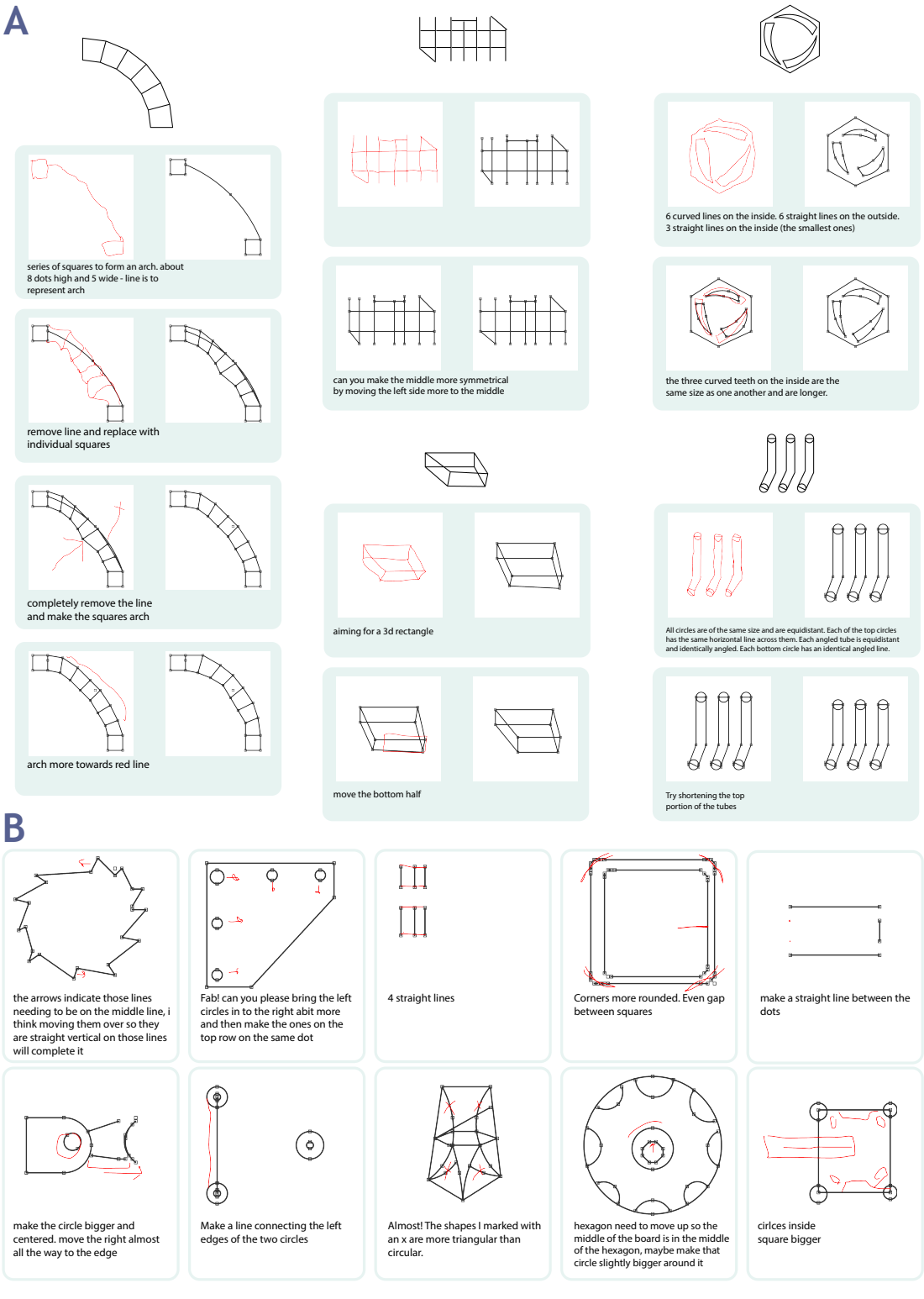


Figure 5.5: **A** Example rollouts from the dataset. Target CADs (top-center) were shown to *Designers*, who created instructions (left columns) that *Makers* followed (right columns). Dyads iteratively refined their CADs across a series of rounds (rows). **B** Examples of multimodal refinement instructions. Language and drawing mutually constrain and inform the others' semantics. Many instructions don't make sense without the accompanying drawings, and vice-versa.

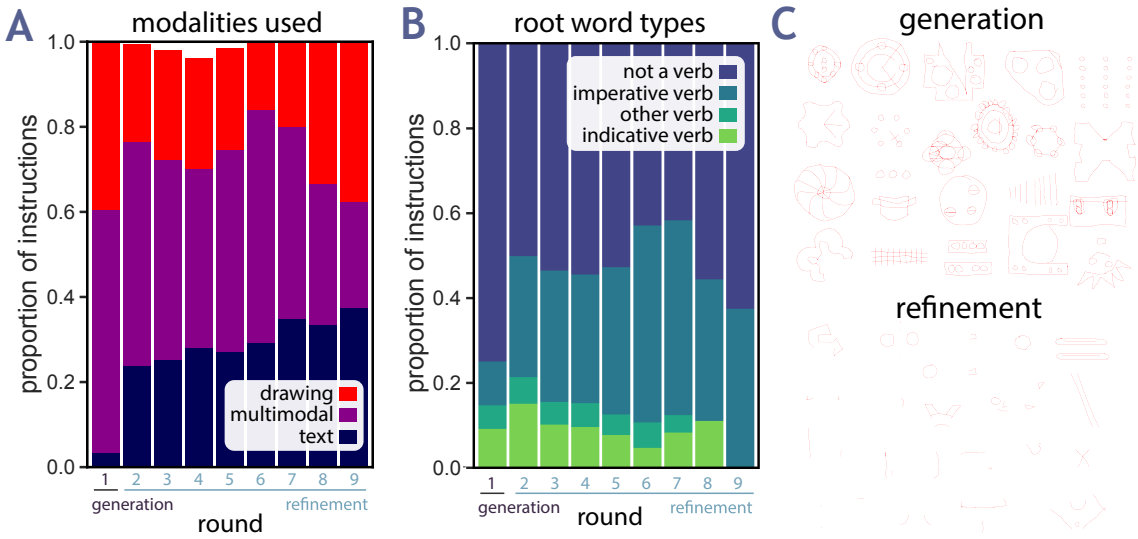


Figure 5.6: **A** Designers' instructions to *generate* CADs (round 1) involved lots of drawing and little text, whereas instructions to *refine* CADs (rounds 2+) used a balance of modalities. **B** The proportions of the types of root words in the dependency parse tree of instruction text. More verbs are used over rounds, and these verbs become more imperative. **C** Samples of 20 generation drawings and 20 refinement drawings highlights the rich detail in generation instructions, and more targeted modifications in refinement.

5.3.3 Analyzing instructions in mrCAD

Refinements increased accuracy of CADs After the initial round, distance to the target continued to decrease ($b = -0.0512$, $t = -16.5$, $p \leq 0.001$; Figure 5.7). These improvements got smaller as rounds progressed ($b = 0.0094$, $t = 8.84$, $p \leq 0.001$), suggesting that refinements became more fine-grained in later rounds.

Modality use differs in generation and refinement Across the 4946 rollouts analyzed, 83.4% are multimodal, 14.5% are drawing-only, and only 2.1% are text-only. Across the individual rounds analyzed, 53.5% are multimodal, 30.2% are drawing-only, and 16.3% are text-only. The distribution of text, drawing, and multimodal instruction was strikingly different in generation (round 1) refinement (rounds 2+) ($\chi^2(2) = 1095$, $p = 0$); almost all generation instructions involved drawings (96.6%), whereas refinement rounds had a roughly equal split of unimodal text and drawing (Figure 5.6A).

Refinement drawings are more sparse than generation drawings Most strikingly, compared to generation, *Designer*s drew *less* in refinement rounds, as measured by the number of strokes ($b = -5.64$, $t = -72.8$, $p \leq 0.001$) and amount of digital "ink" ($b = 20.9$, $t = 2.23$, $p = 0.026$) used. Rendering these drawings (Figure 5.6C) suggests that *Designers*' drawings changed from mostly complete drawings of target CADs in generation to smaller

modifications of sub-parts of the current CAD in refinements.

Refinement text expresses actions and directives The text of instructions contained an average of 50.2 characters, or 9.66 words. We use the English pipeline of the Stanza NLP package Qi et al. (2020) to parse the sentences and extract the root words. We tag the root word based on whether it is a verb, and if so whether the mood of the verb is indicative, imperative, or otherwise. We see in Figure 5.6B that generation instructions don't have a verb at the head a majority of the time, while refinement instructions do. The proportion of imperatives – verbs typically used to express directives issued to a listener – also increases over the course of refinement. This suggests refinement and generation instructions use different languages.

Multimodal refinement messages There are a total of 3723 multimodal refinement messages. Of these, many contained drawings and text that worked together to convey meaning, such that either modality alone would not be sufficient to convey the same intent (Fig. 5.5B). This dataset, along with the *very-dense* subset, provide a resource for future work to investigate how language and drawing are used *together* to communicate precisely.

5.4 Evaluation of frontier models

We evaluate both API access models and open-weights models (which we finetune) on the mrCAD dataset. We are interested in the following.

RQ1 What is the difference in performance between humans and models in following the instructions in mrCAD?
RQ2 What is the effect of generation vs refinement in model performance ?
RQ3 What is the effect of multimodal vs single-modal in model performance?

5.4.1 Benchmark

We select CADs from the **dense set** that have 3 or more successful ($\Delta(D_n, D^*) < 0.2$) rollouts. This results in 682 unique CADs with 2324 rollouts, and a total of 5751 rounds of instruction-execution pairs. Then, for these selected rounds, we can evaluate VLM agents by having them assume the role of a human maker P_{maker}.

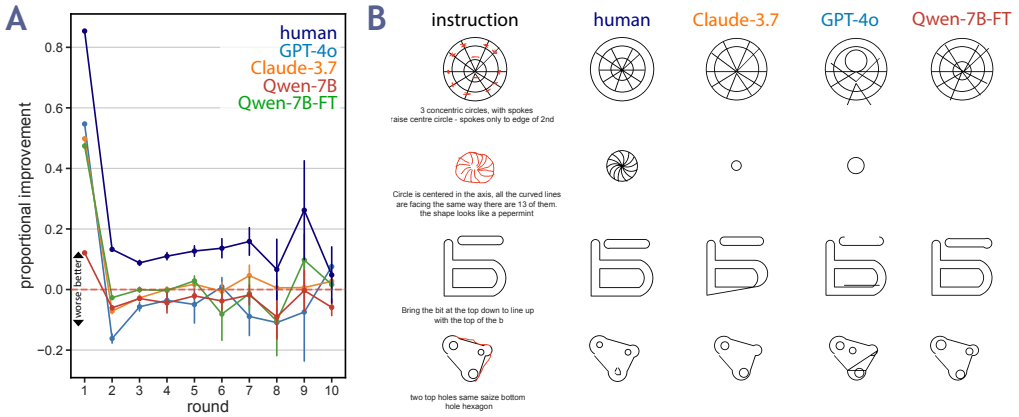


Figure 5.7: **A** Comparison of human and model movement towards target following instructions, normalized by distance at start of round. Only humans make reliably positive changes in responses to refinement instructions. Models made positive steps in generation but largely destructive changes when refining. **B** Comparison of human and model responses.

Metric We evaluate the instruction following abilities by proportional improvement **PI**: how much the distance to the target D^* shrinks as a consequence of an agent's actions.

$$\text{PI}(A, R_{:i}, D^*) = \frac{\Delta(D_i, D^*) - \Delta(A(D_i), D^*)}{\Delta(D_i, D^*)}$$

This metric is a proportion of the remaining distance to target at every round. This accounts for the fact that distances to the target decrease significantly as rounds progress, making late-stage refinements less noticeable without normalization.

5.4.2 Evaluating vision-language models

Gym environment We build a gym framework for standardized evaluations across models. The gym framework allows for instantiating agents that interface with a standardized representation of CADs, and allows for handing the mrCAD dataset, evaluating models, and simulating interactions between designer and maker agents. We evaluate multiple vision-language chat models by having these models assume the role of a human maker.

$$P_{\text{model-maker}}(A \mid m, D_i, R_{1:i-1})$$

API access models Since we present the rendered interaction history and drawing instructions to the model, we require models that are accept interleaved image-text inputs. We choose the following models off-the-shelf based on this criterion: GPT-4o OpenAI (2024),

GPT-4o-mini OpenAI (2025a), Claude-3.7-Sonnet Anthropic (2025), and Qwen2.5-VL-7B-Instruct Bai et al. (2025). To generate editing actions, we prompt the models to generate editing actions as tool calls.

Finetuned models We also fine-tune a Qwen2.5-VL-7B-Instruct model in the same setting to generate editing actions with supervised fine-tuning. The models are given all the previous rounds of interaction between the pair of humans that played that game, the most recent instruction, and trained to predict the editing actions. The actions are formatted as a sequence of tool calls, and the model is trained to produce tool calls. We train LoRA Hu et al. (2022) adapters for all linear modules.

We use the 2684 rollouts from the coverage set as a training set. We also include rollouts that don't pass the evaluation threshold. We create a small held-out validation set by choosing 303 successful rollouts from the dense set for 187 CADs that have fewer than 3 successful rollouts for model selection.

Ablation on modality We ablate one modality (if present) from all the instructions and evaluate them to create −drawing and −text evaluation sets. We also perform these ablations on the training data and finetune separate Qwen2.5-VL-7B-Instruct models to obtain Qwen-7B $\text{FT}_{-\text{drawing}}$ and Qwen-7B $\text{FT}_{-\text{text}}$ models to evaluate on the −drawing and −text sets respectively. We use the same LoRA finetuning setup for these experiments.

5.4.3 Performance of vision-language models

For the results, we group rounds as either generation (round 1) and refinement (round 2+). Note that this grouping is imprecise and can be improved in future works, as later rounds can contain generations as well. We show some examples of model actions in Figure 5.7B.

RQ1, RQ2 We found (Table 5.1) that models are generally able to decrease the distance to the target during *generation*, with more capable models (GPT-4o, Claude-3.7-Sonnet) and fine-tuned models doing better. However, we find that models perform quite poorly in *refinement* turns. We see that models often make changes that actually *increase* the distance to the target – in opposition to the intention of the designer's instruction. We also see that while an approach like supervised fine-tuning leads to significant improvements in generation turns, the gains don't transfer to refinement turns. This suggests that more sophisticated training approaches might be needed to tackle refinement.

We also investigated the influence of context. People are able to interpret a refinement instruction in the context of all instructions that came before it and improve the design. To study whether models leverage previous turns of the interaction to improve performance, we presented models with only the most recent state and instruction as opposed to the

	generation	refinement
Human	0.854	0.119
GPT-4o	0.547	−0.119
−context	–	−0.036
−drawing	0.508	−0.159
−text	0.540	−0.124
Qwen-7B FT	0.474	−0.017
−drawing	0.437	−0.038
−text	0.470	−0.035
GPT-4o-mini	0.308	−0.129
−context	–	−0.049
Claude-3.7	0.520	−0.112
Qwen-7B	0.121	−0.050

Table 5.1: Proportional improvement results for vision-language chat models, including results on ablated instructions. All models performed worse than humans at following instructions, especially refinement instructions, where the models' outputs made designs worse.

entire trajectory. In refinement rounds (where this results in a different context; −context rows in Table 5.1) we observed better performance for both GPT-4o and GPT-4o-mini which we tested. We further investigate the connections between context and the number of actions taken in Section B.5. However, the proportional improvement metric is still negative for these models, highlighting the room for improvement.

RQ3 We also see that models are sensitive to both instruction modalities (Table 5.1), with performance descreasing when a multimodal instruction is ablated to have only a single modality. This effect is more pronounced in generation rounds.

5.5 Related Work

Multimodality The instructions in mrCAD are multimodal, consisting of both texts and drawings. This is in contrast to other works such as Pejsa et al. (2016) and Ku et al. (2020), where the context of the agent is multimodal, yet the instructions themselves only contain language. Our drawings efficiently convey spatial information and reference, and even short utterances – "delete this", "move to here" – can hugely constrain the meaning of drawings. Our work highlights the capacity of drawing to act, not only as a depictive medium, but as a versatile tool for *communication* Goodman (1976); Huey et al. (2021); Fan et al. (2023), and provides a resource for studying how language is used in conjunction with

another medium of communication in a grounded way Lachmy et al. (2022).

Generating Designs Prior work has explored AI agents that assist with computer aided design tasks, including those that leverage various kinds of input modality Sanghi et al. (2022); Chen et al. (2024), including drawings Seff et al. (2021). Beyond CAD, other work has explored how agents can support various kinds of designs given multimodal inputs, including generating HTML and CSS code Si et al. (2024) and slide-shows Ge et al. (2025).

Refinement The interactions between people in mrCAD take multiple turns, as a dyad collaboratively refines a current CAD toward a specific target. Lachmy et al. (2022) study complex instructions, but these are presented all at once by the speaker and verified with interpretation by another agent. mrCAD prioritizes refinement, where people must elaborate on and repair previous instructions. In this regard, our work is similar to Kim et al. (2019). Zhou et al. (2025b) present an algorithm to train LLM agents to engage in multi-turn collaborative design interactions with a simulated user.

Our work is also similar to work by Narayan-Chen et al. (2019) that presents a dataset of collaborative construction interactions in a Minecraft environment. Both works focus on a similar setting of an "architect" or "designer" agent that knows the target instructing a "builder" or "maker" to create it while not being able to act in that environment. We also introduce the dimension of multimodal instructions, which provide an additional facet of context in which language is grounded. Our lighter-weight environment also allows us to collect a substantially larger number of interactions, while also having more complex targets and reconstructions (perfect reconstruction is very difficult for complex targets in our setting, while it is not the case in theirs).

5.6 Discussion

We present mrCAD, a large-scale dataset of humans playing multi-turn, multimodal communication games in a CAD environment. Analysis revealed significant differences between instructions to *generate* and instructions to *refine*, notably a trend towards the complementary use of drawing and text when refining designs. This distinction is mirrored by a severe performance gap in frontier VLMs ability to generate vs to refine CADs.

We posit that this is due to the lack of multimodal refinement data on the internet, from which these models are trained: (i) *Lack of drawing-as-instruction data:* While the internet is a rich source of textual and image data, data of drawings used as instructions is much sparser – particularly the ephemeral drawings that accompany language. (ii) *Lack of refinement trajectories:* Furthermore, people typically upload only finished artifacts to the internet, often

with accompanying contextual text data, making these datasets well-suited for generation tasks. On the other hand, the process of editing artifacts based on further instructions is rarely uploaded, which is why these models struggle with interactive refinements. As a multimodal refinement dataset, mrCAD serves as a valuable resource for investigating these phenomena.

Part III

Reasoning about Common Ground Over Time

Chapter 6

Learning to Form Conventions from Simulated Interactions

As people interact with each other in natural language, they adapt to shared conversational context to communicate more efficiently. One way people adapt is by forming *ad hoc* conventions, where a pair of agents enrich the standing meaning of words or phrases with information drawn from the history of their interaction. You might direct a first-time visitor to meet you *"on the eighth floor patio of the computer science building"* but on the second visit, you might just agree to meet them *"on the patio."*

In this chapter, we make models adapt to communicate more efficiently with people in natural language by forming conventions. These *ad hoc* conventions are part of the common ground between a specific pair of agents. Agents can affect changes to the common ground, and introduce conventions that can be understood in the context of the prior interactions between this pair of agents. This allows agents to converge of efficient conventions, and communicate with less effort over time.

We finetune large multimodal models (LMMs) to adapt in context to prior interactions with a person. Even though our approach uses simulated games between models to produce training data and relies on *no* additional human-produced data, the models we train adapt to communicate with people with increasing success and efficiency (Lazaridou et al., 2020).

We demonstrate our method by training models to play repeated reference games over images (Figure 6.1). Repeated reference games (Krauss and Weinheimer, 1964; Clark and Wilkes-Gibbs, 1986; Hawkins et al., 2020b) have been used to investigate such adaptive behavior in humans. In these games, a *speaker* is repeatedly directed to describe one image in the context of a set of images to a *listener* that guesses the image to which the speaker is referring. Experiments with people playing games over abstract images (Hawkins et al., 2020b) and photographs (Hawkins et al., 2020a) reveal that people rapidly adapt to their conversational partner to use increasingly short utterances while improving communicative success. Prior work has found that LMMs (Hua and Artzi, 2024) and large language models (LLMs; Hua et al., 2025) do not exhibit this behavior, and maintain similar utterance cost over the course of the interaction.

We start with pretrained speaker and listener models and simulate games between them. We apply simple notions of communicative success and cost to create preference pairs for DPO-style policy optimization (Rafailov et al., 2023). We apply our procedure to train speaker models, and evaluate them in repeated reference games over MS COCO photographs and tangram images. In games with human listeners, we find that our model

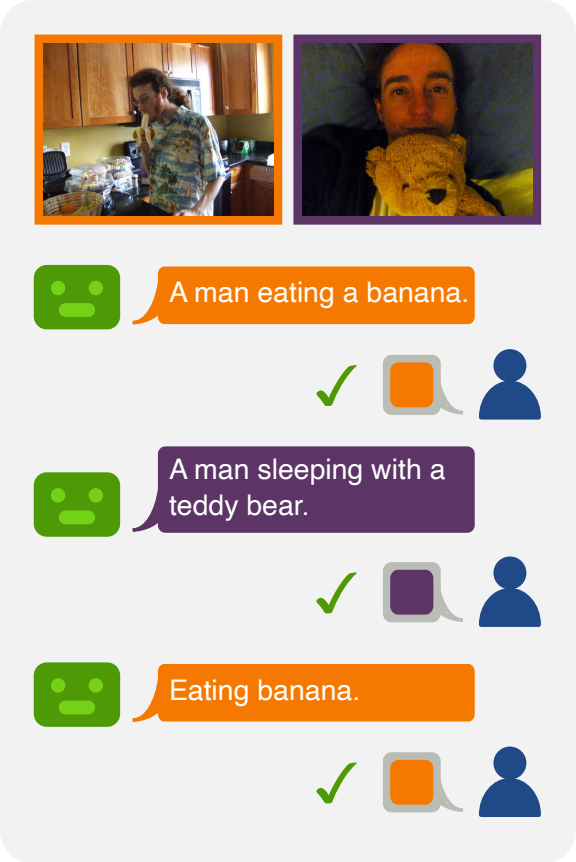


Figure 6.1: A real example interaction with our model playing a repeated reference game. In each turn, the speaker model has to describe a single image in the context of a set of images and previous interactions. A listener tries to guess the image being described. We train speaker models that adapt to communicate efficiently (using fewer words) with people by forming *ad hoc* conventions.

reduces message length by 41% while increasing communicative success by 16% over the course of games over COCO images. Over the course of games over tangram images, our speaker model reduces message length by 29% while increasing success by 27%. We also find that human listeners adapt to respond faster to our models' utterances compared to those from a model that does not adapt. By the 4th repetition of an image, people respond 52% faster to our COCO model's utterances, and 35% faster to our tangram model's utterances.

We find that both components of our utility function – communicative success and cost – are essential. Ablations that remove one or the other do not elicit the same behavior. We find that while a utility function based on success alone leads to an increase in success over the interaction, it also leads to 7% *increase* in message length over the course of the interaction. On the other hand, defining utility based on cost alone hinders the model's

ability to communicate successfully, even though the messages are shorter. In contrast, our approach that defines communicative utility based on both these factors yields models that communicate with people with increasing success *and efficiency*.

6.1 Related work

Convention formation in humans and models Extensive prior work has shown how people adapt their language to their partners to communicate efficiently. Krauss and Weinheimer (1964) find that speakers shorten their utterances in repeated interactions. Clark and Wilkes-Gibbs (1986) study the collaborative aspects of reference, including minimization of effort for both partners. Hawkins et al. (2020b) collect a large-scale dataset of repeated reference games over tangrams. They characterize the changes in utterances as speakers adapt, finding that semantic meaning stabilizes within a pair of interlocutors, even as it diverges across pairs.

Hua and Artzi (2024) study in-context conversational adaptation in off-the-shelf multimodal models using repeated reference games over COCO images, and formalize the notion of stable meaning within a pair using the notion of word novelty. They find that absent heavy-handed prompting, even highly capable multimodal models don't leverage context to communicate more efficiently, continuing to generate detailed descriptions even when they have diminishing communicative utility. In our work, we train models to adapt in context to communicate more efficiently over time, and elicit convention formation in multimodal models.

Machine learning for convention formation Takmaz et al. (2020) present an approach to generating referring expressions by training on reference chains produced by people in the PhotoBook game (Haber et al., 2019), which exhibit linguistic traits such as decreasing cost and consistent use of words in subsequent mentions. Hua et al. (2025) propose a post-training approach for LLMs to generate references in conversation. They curate preference training data from reference chains in TV show scripts, and show that this induces convention formation when models play repeated reference games over sets of words. These approaches adapt human-produced reference chains to train models to exhibit convention formation behaviors. In contrast, the approach we propose does not rely on any additional data from extended interactions (beyond pretraining) produced by people.

Hawkins et al. (2020a) frame conversational adaptation as domain adaptation, and propose updating model weights online during an interaction. They use syntactic structure to determine reusable parts of a referring expression, and induce convention forming behavior in models that play image reference games. In comparison, our approach learns

from simulated games between models, and doesn't require updating model weights at inference time. We also create training data using simple measures of success and cost, without relying on explicit syntactic structure.

Zhu et al. (2021) present an approach to adapt to a listener by modeling their mental state. They maintain an explicit neural model of the listener that is trained with meta-learning to guess the listener's response based on prior interactions. They find that this leads to utterances that are increasingly successful due to the speaker adapting to the listener. However, their approach does not focus on improving communicative efficiency. In contrast, our approach improves communicative efficiency, and does not require an explicit listener model.

Communicative cost in large language models Training language models with reinforcement learning from human feedback (RLHF) has been shown to increase response length (Singhal et al., 2024; Dubois et al., 2024). To inhibit length increase due to RLHF, Wu et al. (2024) include pairs that prefer shorter responses between similarly successful ones in preference data. We use a similar mechanism to penalize message cost, and show that this leads to models that communicate increasingly efficiently with people over time.

Learning from simulated games We extend the research program proposed by Lazaridou et al. (2020), who argue for using multi-agent simulations to make models trained on generic language data functional, utility-driven agents that communicate with people in natural language. We use simulations to train models to adapt to communicate efficiently. Konyushkova et al. (2025) present a self-play method to train multimodal models to play image identification games, and find that it improves performance on tasks like visual question answering. We also train models that play image identification games, but look at repeated games and adaptation over time rather than a single multi-turn interaction. Horst et al. (2025) present an environment with games (Chalamalasetti et al., 2023) that can be used to compare algorithms for training models to interact with people.

6.2 Learning in simulated repeated reference games

We consider repeated reference games defined by a context $C = \{c_1, c_2, \ldots, c_k\} \subset \mathcal{C}$ of k images and a sequence of trials $t_j = (c_j, u_j, l_j)$ where $c_j \in C$ is the target, $u_j \in \mathcal{U}$ is the speaker's utterance, and $l_j \in C$ is the listener's guess in that trial. In the i^{th} trial, the speaker observes C, a sequence of previous trials $T = [t_1, t_2, \ldots, t_{i-1}]$, and the next target $c_i \in C$, and produces an utterance u_i. The listener observes the same context C, the same sequence of trials T and utterance u_i, and guesses a referent $l_i \in C$. The pair of agents is successful if

Figure 6.2: ① We simulate interactions between speaker and listener models, sampling multiple descriptions for a target image at the same stage of the interaction. ② We create preference pairs based on the communicative utility of descriptions. ③ We train the speaker with preference optimization using the pairs created from simulated games.

$l_i = c_i$. The game proceeds in *blocks* of trials, where each block includes each image being presented exactly once as the target, in a random order.

Simulating repeated reference games We simulate repeated reference games between a speaker model and a listener model. The speaker is modeled as a distribution $S_\theta(u|C, T_{:i}, c_i)$ with parameters θ. The listener is modeled as a distribution $L_\phi(l|C, T_{:i}, u_i)$ parameterized by ϕ. We simulate games (Algorithm 1) by choosing a context of images, and simulating a sequence of trials. We start with an empty sequence of trials $T = [\,]$ and incrementally add trials to it. For each trial, we choose the next target c_i and sample n utterances $u_1, u_2, \ldots, u_n \sim S_\theta(\cdot|C, T, c_i)$ from the speaker. We present each utterance to the listener and obtain a guess $l_j = \arg\max_l L_\phi(l|C, T, u_j)$ for $j = 1, 2, \ldots, n$, and construct a set of candidate trials $V = \{(c_i, u_j, l_j)|1 \leq j \leq n\}$. We sample a trial from V to add to the running list of trials T, and repeat for N trials.

Communicative utility Speakers may be viewed as rational actors (Grice, 1975; Frank and Goodman, 2012b) that choose an utterance according to its utility. An utterance is useful if it successfully communicates the speaker's intent. It is also useful if it is easy for the speaker to produce, or easy for the listener to comprehend. A short, easy to produce utterance might not lead to the listener inferring the speaker's intent, but a detailed utterance that would succeed might be expensive to produce. The speaker has to trade off these notions of utility. In defining utility, we consider success to be whether the listener guesses the intended referent, and cost of an utterance u to be the number of words $|u|$ in the utterance (Bergen et al., 2016).

Algorithm 1: Simulating repeated reference games between models

Input: Speaker S_θ, listener L_ϕ, context C, number of trials N,
Output: Samples G, sequence of trials T
$T \leftarrow [\,], G \leftarrow \emptyset$
while $|T| < N$ **do**
  `// Choose next target so each image appears once in a block`
  $c \leftarrow \texttt{NextTarget}(C, T)$
  `// Sample` n `speaker utterances`
  $u_1, u_2, \ldots, u_n \sim S_\theta(\cdot|C, T, c)$
  `// Obtain a listener guess for each utterance`
  for $i = 1, 2, \ldots, n$ **do**
    $l_i \leftarrow \arg\max_l L_\phi(l|C, T, u_i)$
  $V \leftarrow \{(c, u_i, l_i)|1 \leq i \leq n\}$
  $G \leftarrow G \cup \{(C, T, V)\}$
  `// Sample a trial to continue`
  $t_{\text{next}} \sim V$
  `// Extend conversation history`
  $T \leftarrow T + t_{\text{next}}$
return G, T

Preference pairs One method to optimize communicative utility would be to create a scalar utility value that can be optimized with policy gradient methods. However, trading off communicative success and cost in a single value involves searching over a large space of functions that combine these terms, with some combinations being difficult to optimize (Lee et al., 2019). We instead build on the success of learning from pairwise preferences (Rafailov et al., 2023), and create preference pairs where one sample has higher utility than the other.

For a given game state, defined by the context C and the sequence of previous trials T, Algorithm 1 gives us a set of candidate trials V. To create preference pairs, we consider the shortest utterance in V that succeeds, $t^* = \arg\min_{\{t \in V | l(t) = c(t)\}} |u(t)|$. We then create preference pairs $t_w, t_l = (t^*, t)$ for each trial $t \in V$ where $u(t)$ is longer than $u(t^*)$, or leads to an incorrect guess

$$\{(t^*, t)|t \in V, |u(t)| > |u(t^*)| \vee l(t) \neq c(t)\}$$

We refer to this instantiation of communicative utility as SUCCESS+COST. This results in a dataset

$$\mathcal{D} = \{(C, T, t_w, t_l)|t_w, t_l \in V\}$$

where we prefer that the speaker produce $u(t_w)$ over $u(t_l)$ in the context of images C and previous trials T. Note that in constructing preference pairs, we do not explicitly compare with previous trials in T. The comparisons we use are *local* and between different utterances sampled for the same trial.

Alternative notions of communicative utility We also experiment with constructing preference pairs based on other notions of communicative utility. Specifically, we consider the effects of optimizing only for success or only for cost, rather than a combination. In the SUCCESS condition, we create pairs (t_w, t_l) where the listener guesses correctly for t_w but not t_l. In the COST condition, we create pairs (t_w, t_l) where $|u(t_w)| < |u(t_l)|$, without regard to whether the listener guesses correctly.

6.3 Experiments

6.3.1 Image domains

We experiment with two image domains: photographs and tangrams. We source photographs from the COCO dataset Lin et al. (2014). Multimodal models are trained on extensive photograph data, and have access to canonical associations between these images and words. However, in this task, models have to pick associations that are useful in the given context, enabled by conventions. We also experiment with tangrams (Ji et al., 2022b), which lack strong associations between images and canonical language descriptions. This makes descriptions of tangram shapes more arbitrary and challenging to interpret. However, over multiple trials, people can form conventions that coordinate on these associations and allow them to describe these images efficiently. Tangrams have been used extensively to study convention formation Clark and Wilkes-Gibbs (1986); Hawkins et al. (2020b).

6.3.2 Models

We primarily use models based on Gemma-3-12B Gemma Team (2025). We use the pretrained version of the model that has not been instruction-tuned. The speaker is prompted with the context of images, the previous trials of the game, and the label for the target image in the current trial. We prompt the model with a demonstration game to show the format of game responses, but include at most one reference to an image to avoid biasing how the model handles repeated reference. For experiments with tangram images, we initially found that models were not able to generate distinct descriptions for different images. To address this, we finetuned the *only the vision encoder* of the model on synthetic repeated reference games to obtain the Gemma-Tangrams-Base model. We use human-produced

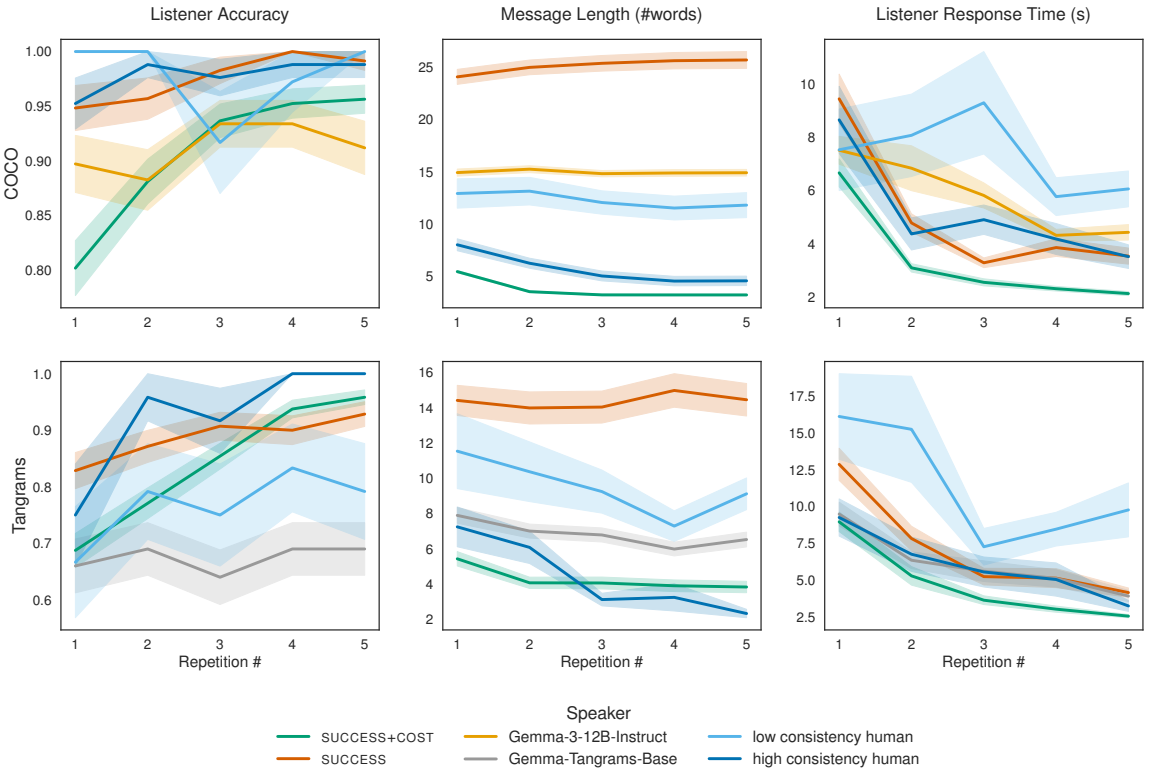


Figure 6.3: The SUCCESS+COST speaker communicates increasingly successfully and efficiently over the course of the interaction with human listeners. The model achieves increasing accuracy while decreasing message length. This efficiency is also reflected in people responding faster to utterances produced by the SUCCESS+COST speaker compared to other models. Error bars show standard error.

descriptions of tangrams collected by Gul and Artzi (2024) to create synthetic games, and use supervised finetuning to adapt the model to the visual domain. In creating data for base models, we ensure we don't use any data from real repeated interactions to avoid biasing the model to specific convention formation behavior.

6.3.3 Training

We simulate games between base speaker and listener models using Algorithm 1. We present contexts with 4 images (Section C.2) and simulate $N = 20$ trials (5 blocks) for each game. For each trial, we sample $n = 4$ speaker utterances. We create preference pairs as described in Section 6.2. We then finetune the speaker with Identity Preference Optimization (IPO; Gheshlaghi Azar et al., 2024). Training details are provided in Section C.3.

6.3.4 Evaluation

We evaluate our speaker models in new games with a listener, with images not seen during training. For Gemma speakers that play games over COCO images, we use greedy decoding, which we found led to the highest listener accuracy in validation games. For tangram images, while greedy decoding resulted in the highest validation accuracy, we found that it caused all models, including the Gemma-Tangrams-Base, to degenerate to repeating the same description for every repetition of an image. To mitigate this lack of diversity in the tangram speaker models, we used nucleus sampling. Additional details are provided in Section C.3.

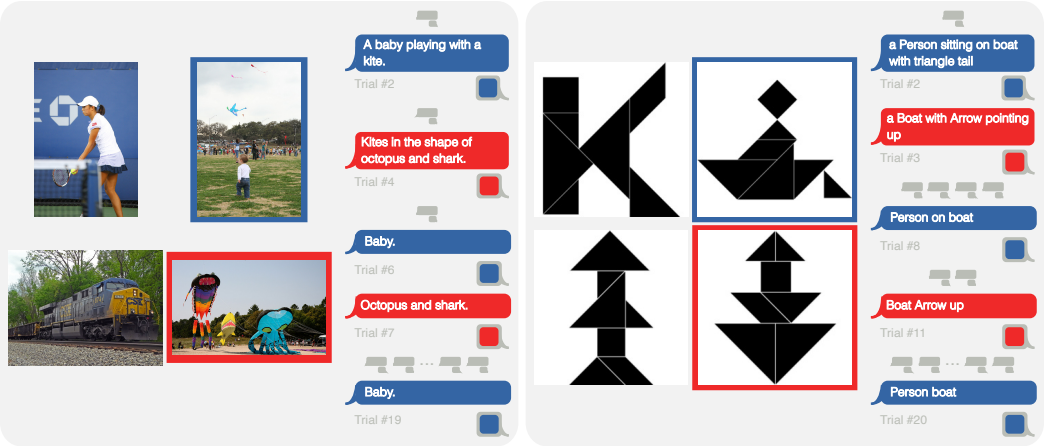


Figure 6.4: Examples of adaptation over the course of a game by the SUCCESS+COST speaker model. In the COCO example, we see an example of the model converge to a convention that doesn't reference 'kites', since it doesn't distinguish the two images. In the tangrams example, we see how the model uses a detailed description (*"with triangle tail"*) in an initial turn, but in later turns drops the detail without losing success.

6.3.5 Human listener experiments

We evaluate our speaker model in games with a human listener, and compare the performance to baseline models that play games with the same listener. As a reference, we also collect data from games between human speakers and human listeners on the same distribution of image contexts.

For experiments with a model speaker, we have human listeners play 2 games. One of these games is with our SUCCESS+COST speaker model. The second game is with one of two baseline models. The first baseline is the SUCCESS speaker. The second baseline for COCO games is the Gemma-3-12B instruction-tuned model, a model that has been instruction-tuned without a focus on communicative efficiency. For tangram games, we

use the Gemma-Tangrams-Base model as the second baseline since it has been adapted to tangram images.

The sequence of the two games is randomized. Each game consists of 20 trials. In each trial, the listener is presented with the context of images and the speaker's descripition, and asked to click on the image they believe the speaker to be describing. Once they guess, the listener is shown the target image, and told whether they were correct. For each trial, we also record the listener's response time – the time between their being shown the description and their clicking on a guess. The 2 games that the listener plays are for different image contexts. At the end of the game, the listener is given some questions to answer about the game. After playing both games, the listener answers questions that compare both the speakers with which they interacted. For experiments with a human speaker, we have listeners play a single game on the same interface, and answer questions at the end of the game.

We recruit 65 paritipants for COCO games with a model speaker, 60 participants for tangram games with a model speaker, 74 participants for COCO games with a human speaker, and 31 participants for tangram games with a human speaker. After excluding incomplete games, we are left with 126 model-as-a-speaker games for COCO and 120 for tangrams. We have 30 human-as-a-speaker games for COCO, and 12 for tangrams. Details of participant recruitment, filtering, compensation, and experiment interface are provided in Section C.5.

Training with SUCCESS+COST makes models communicate efficiently We find that our SUCCESS+COST models communicate efficiently with people, despite not being trained on any additional human-produced data. We observe increasing communicative success over repetitions of an image, while also seeing decreasing message length (Figure 6.3, left and middle). While the SUCCESS models do achieve better accuracy, they do so with much longer messages, and don't improve over time.

We look to the listener's response time (Figure 6.3, right) to evaluate the how easy the utterances are for a person to interpret. Specifically, we consider the response time on the last repetition of an image. At the start of the game, the listener hasn't seen how the speaker describes a particular image (the convention they use), but is also not familiar with the images. Evaluating response time on the last repetition allows us to see the effect of convention formation on a listener that has had the chance to get familiar with the context and develop convention with the speaker. We find that people who played with the SUCCESS+COST speaker and the SUCCESS speaker responded faster to the SUCCESS+COST speaker (Welch's t-test, $p < 2 \times 10^{-5}$ for COCO and tangrams). Similarly, those who played COCO games with the SUCCESS+COST speaker and the Gemma-3-12B-Instruct

speaker responded faster to the SUCCESS+COST speaker (Welch's t-test, $p < 10^{-11}$). Those who played tangram games with the SUCCESS+COST speaker and Gemma-Tangrams-Base responded faster to the SUCCESS+COST speaker (Welch's t-test, $p = 0.001$).

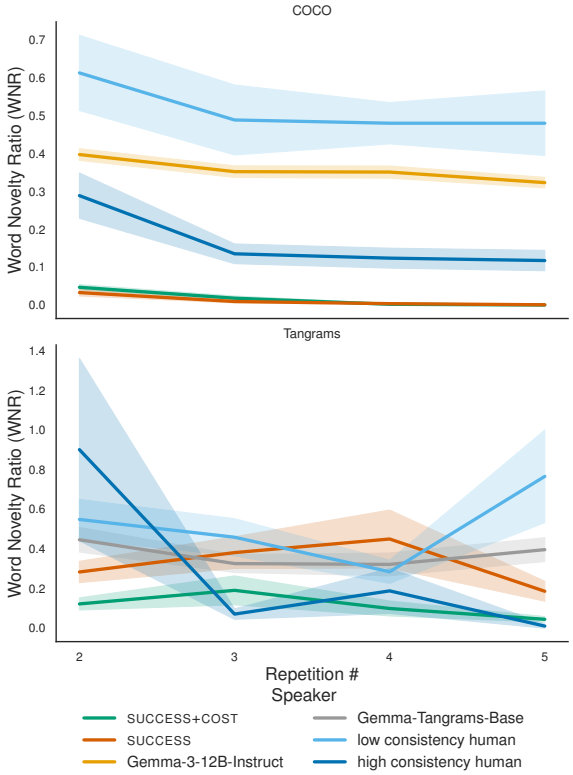


Figure 6.5: SUCCESS+COST speakers achieve low and decreasing WNR. Along with decreasing message length, this suggests convention formation. SUCCESS models achieve low WNR, but repeat entire messages without growing more efficient. Error bars show standard error.

SUCCESS+COST elicits convention formation We find that models trained with SUCCESS+COST communicate efficiently by forming conventions. To evaluate this, we use word novelty rate (WNR; Hua and Artzi, 2024, Section C.4) which measures the rate at which new words are introduced in an utterance. It is calculated as thef word error rate between the utterance for the previous repetition of the image to the current utterance, without penalties for deletions. A low and decreasing WNR (Figure 6.5) suggests that the SUCCESS+COST speaker forms conventions by reusing words from previous utterances. In contrast, we see that the other models we evaluate do not exhibit this behavior. The SUCCESS speaker produces successful utterances, and learns that reusing words enables success, as reflected in the low WNR. However, the messages do not decrease in length, failing to exploit convention formation to be more efficient. The Gemma-3-12B-Instruct

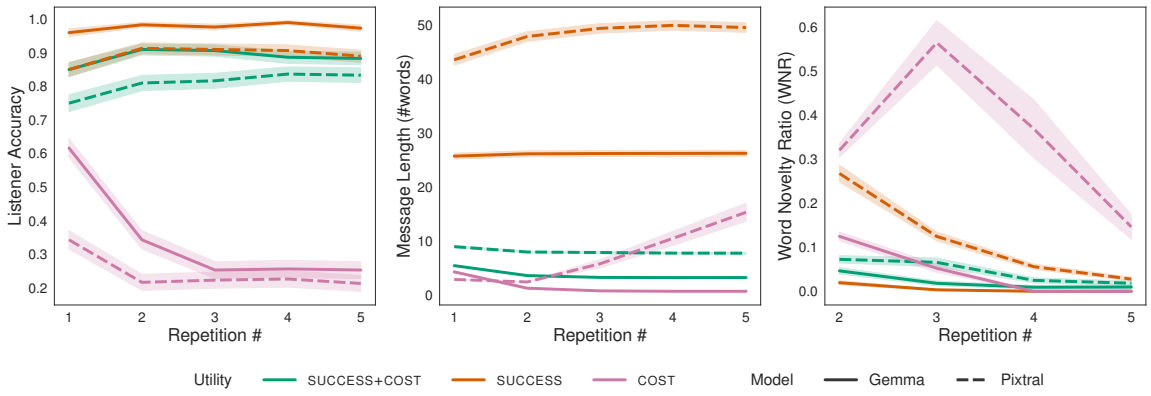


Figure 6.6: In COCO experiments with a GPT-5-mini listener, the SUCCESS+COST speaker shows similar trends to human listener experiments for Gemma and Pixtral-based models. For Pixtral models, message length decreases by 13.4% (8.95 → 7.74) by repetition 4. When training with COST alone, listener accuracy drops to near chance levels.

speaker produces successful utterances, but does not show decreasing message length and consistently innovates in its utterances, suggesting that the model doesn't form conventions.

We find two patterns of language use by human speakers, similar to Hua et al. (2025). Some adapt their language towards shorter, more consistent utterances, as has been found in extensive prior work. We term these "high consistency" speakers. Others don't adapt, choosing instead to use more verbose descriptions that continually innovate, who we term "low consistency" speakers. We use a similar method to Hua et al. (2025) (Section C.5) to classify speakers as high consistency or low consistency. Notably, we find that human listeners are also sensitive to lack of adaptation by low consistency speakers, taking longer to respond to their utterances (Figure 6.3, right).

Models increasingly use open class words over time We tag utterances using the spaCy POS tagger (Honnibal et al., 2020) and find (Figure 6.7) that the SUCCESS+COST model's utterances contain more open class words (nouns, verbs, adjectives) over time relative to closed class words (pronouns, determiners, prepositions, etc.). Even though models aren't trained on demonstrations of human convention formation behavior, these trends are similar to what Hawkins et al. (2020b) find with people.

6.3.6 Simulated listener experiments

We conduct additional experiments to evaluate speakers trained with COST alone, and speakers based on another pretrained model. For these experiments, we use a GPT-5-mini listener (Hua and Artzi (2024) find that listeners based on large multimodal models perform

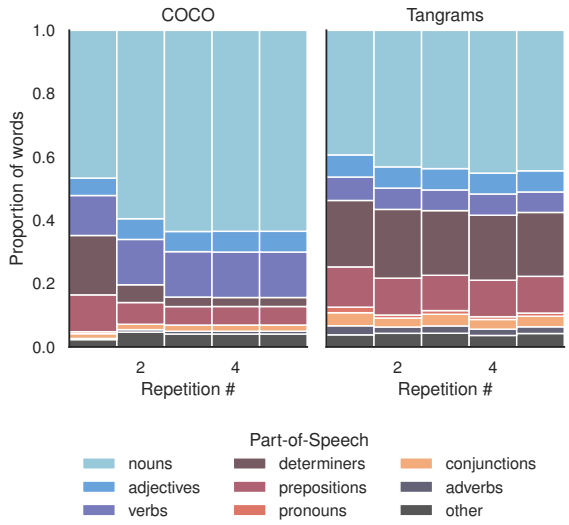


Figure 6.7: SUCCESS+COST model uses more open class words (nouns, verbs, and adjectives; in cooler hues) relative to closed class words (pronouns, determiners, prepositions, etc.; in warmer hues) in its utterances over time by preferentially dropping function words.

on par with human listeners at this task). In these experiments, we evaluate only listener accuracy, message length, and WNR.

We find similar trends for SUCCESS+COST and SUCCESS for Pixtral-based models as for Gemma-based models. Details of training for the Pixtral-based models are provided in Section C.1.

Training models with COST alone is insufficient In experiments with a simulated listener, we find that while training with SUCCESS+COST and SUCCESS show similar trends to experiments with human listeners, training with COST alone is insufficient, as it (unsurprisingly) results in low communicative success. Results for tangram speaker models trained with COST alone are in Section C.6.

6.4 Discussion

In this chapter we present a method to train models that adapt their language to communicate more efficiently over time in repeated interactions. We apply our method to training models that play repeated reference games over images. We find that our method elicits linguistic conventions that are legible to human listeners, even though models receive no explicit demonstrations of human convention formation behavior during training. The use of convention by the models we train also leads to faster responses from human listeners compared to models that have not been trained for communicative efficiency. This

highlights how training for communicative efficiency can improve the ability of people to respond to what a model generates.

The rewards we use to elicit this behavior are broadly applicable. We rely on general notions of communicative success (whether the speaker's intent was recovered by the listener) and cost (the number of words). In future work, these rewards might be instantiated in other settings to allow agents to communicate more efficiently with people when carrying out complex tasks.

Chapter 7

Agents that Learn from Help they Seek *(proposed)*

Agents often have to perform multiple tasks in similar environments. Those that apply knowledge gleaned from prior experience to new tasks have the potential to be more capable and efficient. These agents can persist knowledge of useful abstractions and procedures, preferences, and other information to perform new tasks more reliably and with less effort wasted in repeatedly exploring solutions to a problem. They can also grow increasingly capable by composing these capabilities to perform increasingly complex tasks reliably.

However, these agents work alone and adapt based on exploration and environment feedback. Approaches to guiding and adapting agents such as authoring `SKILLS.md` files or writing rules rely on user initiative. These agents don't proactively seek help when they need it, and need to wait for the user to identify the need for help. For people, the agents need more work to monitor and guide, and don't naturally adapt from experience and guidance. I propose developing a method that has agents seek help when needed, and learn from the help provided.

Additionally, agents should be easy to help. While language is a powerful way to provide feedback, when guiding an agent through multi-step routines, describing each step in detail might be cumbersome. Demonstrations, where the user takes the actions for the agent to observe and learn from, are more efficient.

Agents that learn from the help they seek have to adapt their queries to the context of prior interactions. An agent that repeatedly asks for help to overcome the same challenge might be successful, but will have diminished utility.

7.1 Proposed setup

I propose training agents that collaborate with people on browser use tasks. In this setting, two agents act in the same browser environment. The agent can observe what the user is doing, and vice versa. The user gives the agent a task to complete, typically specified in natural language. The agent can act in the environment to complete the task. If the agent chooses to, it can hand over control to the user. The user can then act in the environment to assist the agent, and hand control back when they choose. Since the agent can observe the user's actions, it can learn how how to do the task from the user's demonstrations. The agents are also free to exchange natural language messages in a chat.

An agent-user pair perform a sequence of tasks. Over time, as the user helps the agent

improve, the agent should adapt to the shared context of prior interactions, and learn to rely on previously provided help when it is applicable in new situations. I propose enabling this using existing non-parametric memory mechanisms developed for browser use agents, such as reusable workflows (Wang et al., 2025b), programmatic macros (Wang et al., 2025a; Yu et al., 2026; Zheng et al., 2025), or free-form insights (Ouyang et al., 2026). Rather than inducing memory content on the agent's own actions, the agent would induce memories based on its own actions and the user's actions, allowing it to learn from help.

The challenge that remains is teaching the agent to ask for help selectively. Prior work has considered agents that ask for clarification when uncertain (Eisenstein et al., 2025; Yu et al., 2020) or when faced with an ambiguous instruction (Zhang and Choi, 2025; Zhang et al., 2025; Rao and Daumé III, 2018). Berant et al. (2025) consider clarifications that have a cost, requiring the agent to trade off the cost of asking for help against the cost of making a mistake. I propose adapting these methods to train agents to ask for help selectively. By gathering interactions between agents and a simulated user, and identifying those where the agent succeeds with the least user effort, we can train an agent to ask for help selectively.

I propose combining these methods by simulating an agent-user pair solving a sequence of tasks. By simulating interactions over multiple tasks, we can identify interactions where the agent leverages help provided in prior tasks to reduce reliance on user help in subsequent tasks. This would resemble meta-learning, where an agent learns how to learn effectively from user assistance within one sequence of tasks.

7.2 Experiments

I propose evaluating these agents on environments such as WebArena (Zhou et al., 2024) and WorkArena(++) (Drouin et al., 2024b; Boisvert et al., 2024). These benchmarks are designed to evaluate browser use agents on a variety of realistic, but controlled websites. These benchmarks also have sets of tasks that have shared structure, allowing us to evaluate whether an agent learns from help provided in prior tasks to perform subsequent tasks more efficiently. The aim is to reduce the fraction of actions taken by the user simulator, both overall, and particularly over a sequence of tasks.

As baselines, we consider the following settings:

- **Base agent**: The agent solves tasks alone, without help from the user, and does not persist knowledge from prior tasks.
- **Memory agent**: The agent solves tasks alone, without help from the user, but persists knowledge from prior tasks in memory. This should improve performance over the base agent, but be limited in the improvement by the agent's inability to discover

some solutions.

- **Prompted help-seeking agent**: The agent is allowed to ask the user for help and maintain memory of prior interactions, but is not trained to ask for help. In this case, the agent is not trained to ask for help selectively. This may result in worse performance because the agent didn't ask for help when needed, or higher user effort because the agent asked for redundant help.

I aim to develop agents backed by open-weights models that I can train, such as Gemma 3 (Gemma Team, 2025), Qwen3 (Yang et al., 2025) or GPT-OSS (OpenAI, 2025b). I will evaluate them in interactions with users simulated by stronger computer use models such as Gemini-3-Flash (Gemini Team, Google, 2025).

7.2.1 Illustrative results

As a preliminary experiment, we look at whether a Gemma 3-based agent (henceforth *student*) can benefit from help provided by a stronger user simulator model like Gemini-3-Flash with Agent Workflow Memory (henceforth *teacher*). We consider the setting where the student observes entire trajectories from the teacher agent, and persists knowledge from these demonstrations using Agent Workflow Memory (Wang et al., 2025b). This is an unrealistic interaction scenario – the teacher has to solve the same task as the student every time, duplicating effort – but allows us to see whether the student can benefit from teacher demonstrations and persist knowledge for subsequent tasks.

We consider a sequence of 195 WebArena tasks.[1] For each task, the student attempts the task first. The teacher then attempts the task (with no knowledge of the student's actions). Then, rather than the student inducing workflows from its own trajectories (as in AWM), the student induces workflows from the teacher's trajectories. Note that the teacher's workflows only influence its trajectories, and are not directly available to the student. As baselines, we consider the base agent and memory agent (with AWM) as described above. As an upper bound on performance, we also consider the teacher solving the tasks alone.

Model	Success rate (%)	Number of steps
Base agent	13.8	5.57
Memory agent	20.5	9.15
Memory agent with teacher demonstrations	23.1	9.18
Teacher	50.0	13.13

[1]Where available, the corresponding task from WebArena-Lite (Liu et al., 2024) is used, since these tasks have corrected reward functions.

We see that teacher demonstrations give the student stronger basis for workflow induction, resulting in greater success. These results are preliminary, but suggest that agents may benefit from guidance. The challenge that remains is achieving these gains while minimizing teacher effort.

Appendix A

Inferring Intent by Reasoning about Pragmatic Demonstrations

A.1 Program sampling

We sample programs from the CONCATENATION and SEPARATION templates from Ye et al. (2020) since the INTERSECTION template is not supported by some popular regular expression libraries like Python's standard `re` module. We sample programs with a maximum concatentation depth of 3, and sample programs from the CONCATENATION and SEPARATION templates in a 5:1 ratio using the sampling tools provided by Ye et al. (2020).

Despite controlling the complexity of programs during sampling, many sampled regular expressions are quite long and difficult for a human to reason about easily. To make the task easier for annotators and user study participants, we use the number of tokens in the regular expression program as a proxy for complexity, and cut off programs that are shown to humans at a maximum length of 30 tokens.

A.2 Training base models

We use ByT5 pretrained models (Xue et al., 2022) as the backbone to build all our speaker and listener models. We sample programs as described in Section A.1 and generate uninformatively chosen examples for the programs. We then train the models on a sequence-to-sequence task. The input to the listener is a linearized sequence of examples, and the output is the program. The input to the speaker is a program followed by a linearized sequence of prior examples, and the output is the next example in the sequence.

We generate random examples by first choosing whether the example is a positive or negative examples with equal probability. If the example is positive, we use the `rstr` Python package to sample strings that match the target regular expression. If the example is negative, we use the same package to sample a string from the regular expression `.*` and check that it doesn't match the target program.

We train the base models on 100,000 programs. For each program, we randomly sample 3 specifications. The length of each specification is chosen to be an integer between 0 and 15, uniformly at random. The models are trained for 1 epoch, using the AdamW optimizer with a learning rate of 5×10^{-5}, with the learning rate warmed up over the first 10% of training steps, and then decayed linearly. The batch size is set to 32.

Additionally, we found that training the base model (that we dub L0) for 3 epochs

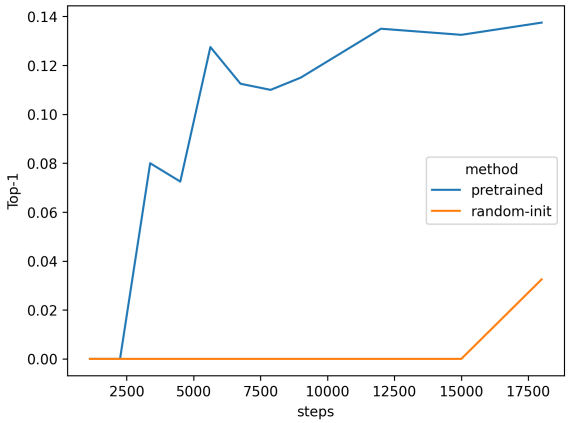


Figure A.1: Comparing validation performance of a base model trained from a pretrained checkpoint to one trained from a randomly initialized checkpoint. One epoch of training is 9375 steps.

achieved higher performance. To ensure a fair comparison, we trained from the higher performing base model (that we dub L0+) using our PRAX algorithm. We see in Figure A.2 that we get significant improvements over the L0+-based literal model even when continuing from an improved base model.

We also compare to training the base model from a randomly-initalized checkpoint Figure A.1, and find that training from the pretrained checkpoint is far more sample-efficient.

A.3 Pragmatic model training

Algorithm 2: Outer training loop

Input: Base listener L_{θ_0}, base speaker S_{ϕ_0}, hypotheses $\mathcal{H}$, validation pairs $\mathcal{V}$, metric METRIC

Output: Trained listener model L_θ

for $r = 0$ **to** R_{max} **do**

- Sample set of k hypotheses $H \sim \mathcal{H}$
- $\mathcal{D}_r \leftarrow \{(h, \text{GENERATEPRAGMATICSPECIFICATION}(h)) \mid h \in H\}$
- $L_{\theta_{r+1}} \leftarrow \text{TRAINLISTENER}(L_{\theta_0}, \bigcup_{j=0}^{r} \mathcal{D}_j)$
- $S_{\phi_{r+1}} \leftarrow \text{TRAINSPEAKER}(S_{\phi_0}, \bigcup_{j=0}^{r} \mathcal{D}_j)$
- $\mathcal{H} \leftarrow \mathcal{H} \setminus H$

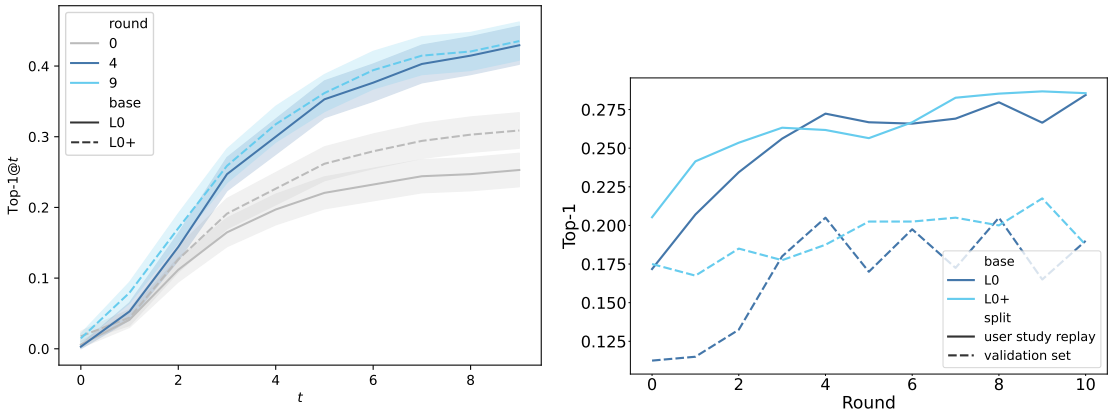
return $\arg\max_r$ METRIC$(L_{\theta_r}, \mathcal{V})$

Finally, we finetune the the base models we train by iteratively using the speaker and listener models to generate data, and finetuning on the generated data using the algorithm

Algorithm 3: Approximate RSA inference

Input: Listener L_θ, speaker S_ϕ, target program h
Output: Specification s

$s \leftarrow [\,]$
for $i = 1$ **to** $N_{examples}$ **do**
  `// Sample examples from` S_ϕ `conditioned on` s`, filtered for consistency with` h
  $E \leftarrow \textsc{GetExampleCandidates}(S_\phi, h, s)$
  `// Sample programs from` L_θ `conditioned on` s`, filtered for consistency with` s
  $D \leftarrow \textsc{GetDistractors}(L_\theta, s)$
  foreach $p \in D \cup \{h\}$ **do**
    foreach $e \in E$ **do**
      $M[e, p] \leftarrow \mathbf{1}[p \vdash e]$
  `// Compute literal listener distribution`
  foreach $p \in D \cup \{h\}$ **do**
    foreach $e \in E$ **do**
      $$L_{\text{literal}}[e, p] \leftarrow \frac{M[e, p]}{\sum_{p'} M[e, p']}$$
  `// Compute RSA speaker distribution`
  foreach $p \in D \cup \{h\}$ **do**
    foreach $e \in E$ **do**
      $$S_{\text{RSA}}[e, p] \leftarrow \frac{L_{\text{literal}}[e, p]}{\sum_{e'} L_{\text{literal}}[e', p]}$$
  $e^* \leftarrow \arg\max_e \; S_{\text{RSA}}[e, h]$
  $s \leftarrow s \oplus [e^*]$
return s



(a) TOP-1 over turns

(b) TOP-1 over rounds of training

Figure A.2: Comparison between the L0 model from the original experiments and the L0+ model trained for longer. Round 0 corresponds to the LITERAL model.

shown above.

To generate the candidates at a particular round r, we use both the base models and the models from round r. Sampling from multiple models with different biases can help us draw a more diverse sample of programs and examples, and get a better approximation of the consistency matrix to perform RSA reasoning over. We draw 250 samples from each of the models we are sampling – the base listener, the listener from round r, the base speaker, and the speaker from round r.

As we note in the algorithm, we update the models by training from the initial parameters (of the base speaker or listener model) on all the datasets generated up to and including that round. We train for for $R_{max} = 20$ rounds with $k = 1024$ programs per round, generating specifications with up to $N_{utterances} = 10$ examples. We choose the model trained for 4 rounds based on the TOP-1 metric on the validation set of 40 examples. At each round we update the models using the AdamW optimizer for 1 epoch, using a learning rate of 5×10^{-5}. The batch size is set to 32.

A.4 Dataset collection

We recruited 18 computer science graduate students. The data collection task proceeded in two phases – specification creation and specification verification. Some participants did the both tasks, while some only did the verification task. Participants who did both tasks spent approximately 3 hours on the task and were compensated with $45, while those who did the verification task spent approximately 2 hours and were compensated with $30. Participants were allowed to perform the task online at a location of their choice, and were

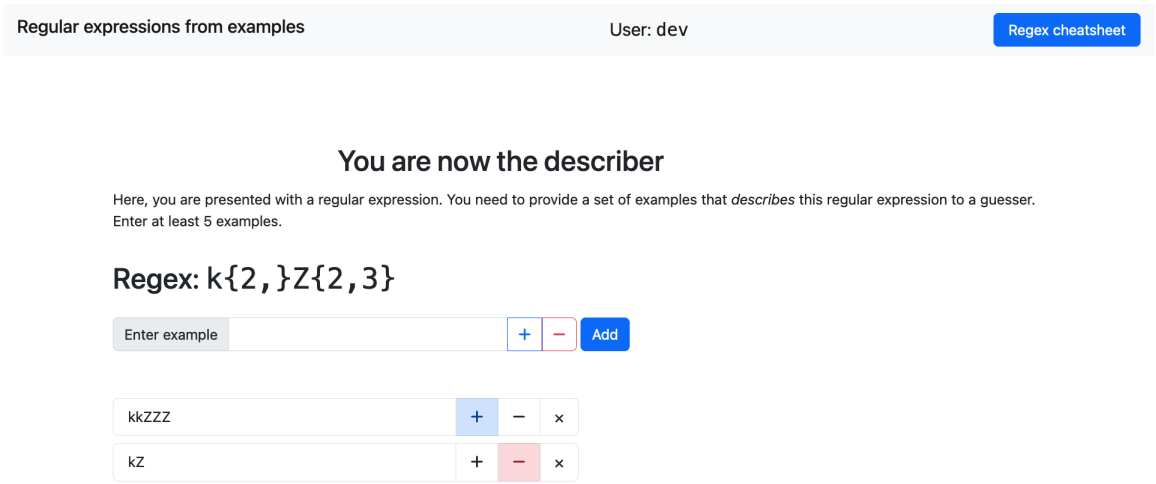


Figure A.3: Interface for providing examples for a regular expression

allowed to perform the task across multiple sittings.

Each participant was given a short tutorial on the syntax and semantics of regular expressions, and small quiz where they had to enter three positive and three negative examples for a given regular expression to proceed to the rest of the study. Participants were then told about a communication game, and to provide examples that they would use if they were asking another person to write the regular expression for them – as they might on an online forum like StackOverflow.

Participants then went to the annotation screen for the specification creation task (if they took part, if not they proceeded directly to the next stage), where they were shown a regular expression and asked to enter examples consistent with it, in the interface shown in Figure A.3. Consistency was automatically checked, so participants could not enter inconsistent examples. A minimum number of examples between 5 and 7 was randomly picked for each instance (to allow the participants from simply providing the same minimum number for each program), but no limit on the number of examples was placed. Participants created specifications for 40 programs before proceeding to the verification task.

For the verification task, participants are shown a specification and 5 regular expressions, of which the target regular expression (that the specification was written for) is one, as shown in Figure A.4. The others are distractors. To ensure the distractors are not too easy to distinguish from the target, we sample distractors using the LITERAL model. Of the 4 distractors, we attempt to find 2 which are consistent a subset of the specification, and 2 which are inconsistent with the specification. Since providing the entire specification might

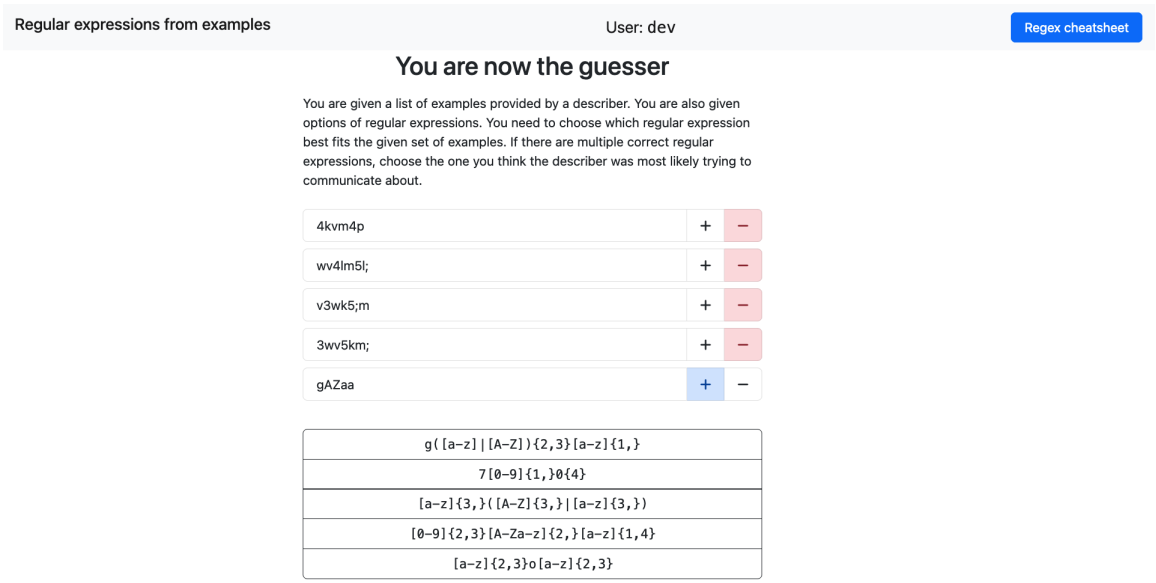


Figure A.4: Interface for verifying a specification written by another human

lead to too few consistent programs remaining, or distractors that are too hard to tell apart from the target, we use only the first example of the specification as input to the LITERAL model to generate distractors. Depending on whether they did the specification creation task, participants verify 40-90 specifications.

The dataset contains 440 examples in total. Of these, 423 have 2 verifications and 17 have one verification. Of the 440, 352 (80%) have two verifications that recover the target. 406 (92%) of the 440 have at least one verification recovering the target. We use 40 of the programs with two verifications recovering the target as the validation set.

A.5 User study

To conduct the user study, we recruited 11 computer science graduate students. Participants spent approximately 1.5 hours on the task and were compensated with $25. Participants were allowed to perform the task online at a location of their choice, and were allowed to perform the task across multiple sittings. Figure A.5 shows the UI.

Each participant was given a short tutorial on the syntax and semantics of regular expressions, and small quiz where they had to enter three positive and three negative examples for a given regular expression to proceed to the rest of the study. Participants were then told about a communication game, and informed that they would be playing the role of the speaker in the game and describing a target program. Each participant interacted with the three models in order – PRAX, LITERAL, and HFT – and cycled through the list in

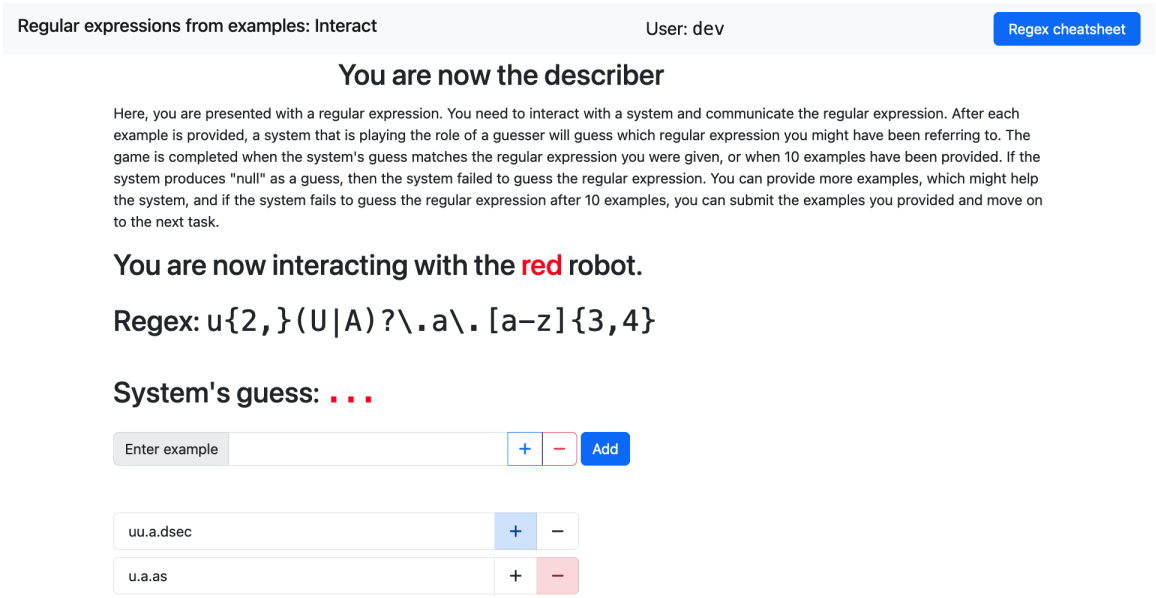


Figure A.5: Screenshot of the user study UI with an example interaction

order, but first model they encountered was chosen randomly. Each model was assigned a color, and the participants were told the color of the model they were interacting with, but not given any other details about it. Participants provided examples one at a time, and observed the synthesizer's highest ranked guess at every step. If the guess matched the target program, the interaction ended. Participants were given the option to move to the next task after providing 10 examples. Each participant completed 31 instances they provided examples for 31 different programs, with one of the interactions from one of the particpicipants being excluded due to an error.

A.6 Prompting an LLM

We prompt the `gpt-3.5-turbo-instruct` variant of GPT-3.5. We provide examples of 5 program-specification pairs in the context of a code snippet, followed by a set of examples. We sample 10 generations with temperature $t = 0.7$ and $p = 0.9$. We sample for a maximum of 64 tokens, or stop tokens that indicate that the regular expression is complete are sampled.

We rank the generated completions using the sum of token log probabilities, and choose the highest scoring program consistent with the given answer as the top-1 guess, and other consistent programs as the top-10 guesses to calculate the TOP-1 and TOP-10 metrics.

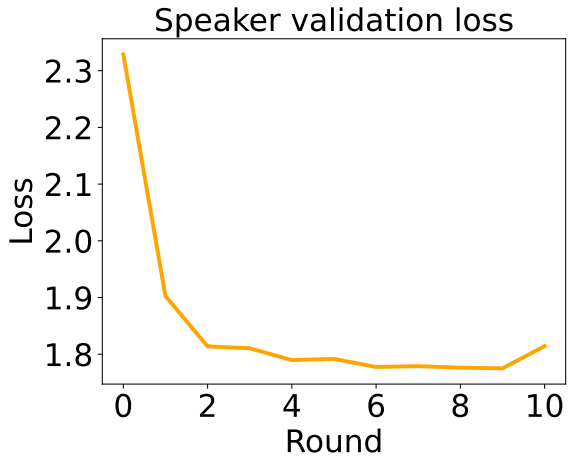


Figure A.6: Speaker loss on the validation set computed over rounds of training

A.7 Speaker quality

One measure of the speaker's quality is the loss evaluated in examples provided by humans. Evaluating loss allows us to see if human provided examples are more likely under a model's distribution, without penalizing the model for the exact example not appearing in a set of examples decoded from the model (since there are many possible examples that are consistent with a program, and multiple possible informative examples too). We see in Figure A.6 that even without being trained on any human-provided examples, the speaker loss on human-provided validation examples reduces.

A.8 Connections to iterated bootstrapped training

Methods such as DreamCoder (Ellis et al., 2021) and Memoized Wake Sleep (Hewitt et al., 2020) use iterated bootstrap training to build a hierarchy of abstractions to solve tasks more effectively over iterations of training. At each iteration of training, they create tasks by sampling a program from a changing library of abstractions, and executing the program on a fixed distribution of inputs to train a synthesizer to work with updated abstractions.

Our method on the other hand is aimed at allowing a synthesizer to reason about how examples are chosen to demonstrate programs from a fixed library. We change the distribution of examples being drawn from a trained speaker model over the course of iterated bootstrapped training towards more informative examples.

Appendix B

Multimodal Interaction for Collaborative Construction

B.1 Analysis

B.1.1 Statistical methods

Our primary statistical method was the linear mixed effects model. For modality experiments, we fit models with fixed effects for round and condition, as well as their interaction, and random intercepts for dyad. We then compared this full model to a series of simpler, nested models with predictors removed. We report results from the most complex model for which AIC substantially dropped, compared to the subsequent simpler model. For the effect of refinement, we we ran paired t-tests between the ***multimodal + refinement*** condition and the ***multimodal + generation only*** condition.

B.1.2 Distance metric

The distance between two designs D, E, are calculated as follows.

```
1  def dist_chamfer_symmetric (D, E):
2    return 0.5 * (dist_asymmetric(D, E) + dist_asymmetric(E, D))
3
4  def dist_asymmetric(D, E):
5    pts_in_D = [sample_points(curve) for curve in D].flatten()
6    pts_dists_to_E = [dist_pt_to_design(pt, E) for pt in pts_in_D]
7
8  def dist_pt_to_design(pt, E):
9    return min([dist_pt_to_curve(pt, curve) for curve in E])
```

The distance between a point and a curve (line 10) is omitted for brevity, for details please consult the code base. The default value for the min operator (line 10) is $\frac{1}{4}$, expressing the idea that beyond half a quadrant away, two points are not likely to be unrelated.

In the analysis, we found a small error in our javascript implementation of the accuracy function that led to a proportion of lower performing trials receiving greater distances than they should have. As accuracy for inclusion of trials was recalculated post-hoc, entries in our dataset were unaffected. However, some participants may have performed extra rounds of modification to meet the target threshold.

B.2 Multimodality and Refinement Ablations

B.2.1 Experimental methods

Crowd source participants were paired and randomly assigned the role of *Designer* or *Maker*. We sampled 100 CADs from the Sketchgraph dataset Seff et al. (2020), and presented these CADs across the set of dyads in each of four conditions:

multimodal + refinement *Designer*'s messages could include text (up to 200 characters) and/or drawing instruction (unlimited). There are 3 rounds total, each round with 30 seconds for the *Designer* to construct a message, and 120 seconds for the *Maker* to manipulate the current CAD in a CAD interface.

text only + refinement Where the message could only contain text but not drawing.

drawing only + refinement Where the message could only contain drawing but not text.

multimodal + generation only Here, rather than having 3 rounds of communication, they are aggregated into a single big round, with 90 seconds for the *Designer* and 360 seconds for the *Maker*.

B.2.2 Results

Refinement improves reconstruction Participants in all **refinement** conditions were able to improve their reconstructions over all three rounds ($b = -0.0335$, $t = -6.91$, $p =< 0.001$) (Fig. 5.3A). While participants in the multimodal refinement condition on round one generated less accurate CADs than the **generation only** condition ($t = 1.47$, $p = 0.145$), by round three their designs were reliably more accurate ($t = -2.15$, $p = 0.0340$), despite having the same total time available. This highlights the value of iterative refinement for creating accurate designs.

Drawings improves reconstruction Participants who could only use text achieved considerably less accurate reconstructions than those in the drawing-only and multimodal conditions ($b = 0.127$, $t = 6.54$, $p =< 0.001$). While text is a relatively poor communicative medium on its own, participants in the multimodal condition still opted to use text in conjunction with drawings, and increased their use of text in later rounds (Fig. 5.3B), suggesting that language is useful for *refining* CADs. The amount of drawing remained consistent in the drawing-only condition, but dropped precipitously after round 1 in the multimodal

condition (Fig. 5.3C). Overall, when given a choice of multimodality, human participants favored more drawing for the generation round, and more texts in the refinement rounds.

Interim discussion Why do participants choose to use multimodal instructions when given a choice? Manually inspecting the data revealed that drawings are primarily used in two ways: Those used for creating a shape, and those used for editing a shape. In the multimodal condition, the *Designer* can disambiguate these with texts such as "make this shape" or "move the shape as shown". However, in the drawing only condition, the *Designer* frequently resorted to redrawing the entire target CAD from scratch — a valid yet inefficient strategy. Data collected from this experiment hint towards a complex interplay between linguistic and graphic communication that differs in generation and refinement settings. The mrCAD dataset, discussed next, aims to provide a large scale dataset of *multimodal generation and refinement instructions* to study these phenomena.

B.3 Data Collection Procedure

B.3.1 mrCAD annotation task

We recruited fluent English speakers from the USA and UK on Prolific. Prolific workers were told that their data was being collected for research and development purposes, presenting findings in academic papers, presentations, or datasets, and gave consent. Our institution did not require IRB approval for this work. We also do not release any personally identifying information for participants in the dataset. Participants were paired, randomly assigned the role of *Designer* or *Maker*, and worked together over a series of rounds to recreate target CADs.

Rather than fixing a number of rounds and the amount of time per round, we instead limited the maximum round number to 10, and time to 9 minutes. Within these limits, participants can choose how many rounds they will take, and to allocate different amount of times for each round. We also lifted the limit on the amount of text characters the participants can send in a message.

B.3.2 Dataset collection interface

Figure B.1 shows screenshots of the user interface used for data collection for both players.

B.3.3 Dataset collection parameters

While the key features of our data collection paradigm were kept consistent between our controlled studies (Section 5.2) and full dataset collection (Section 5.3), we implemented

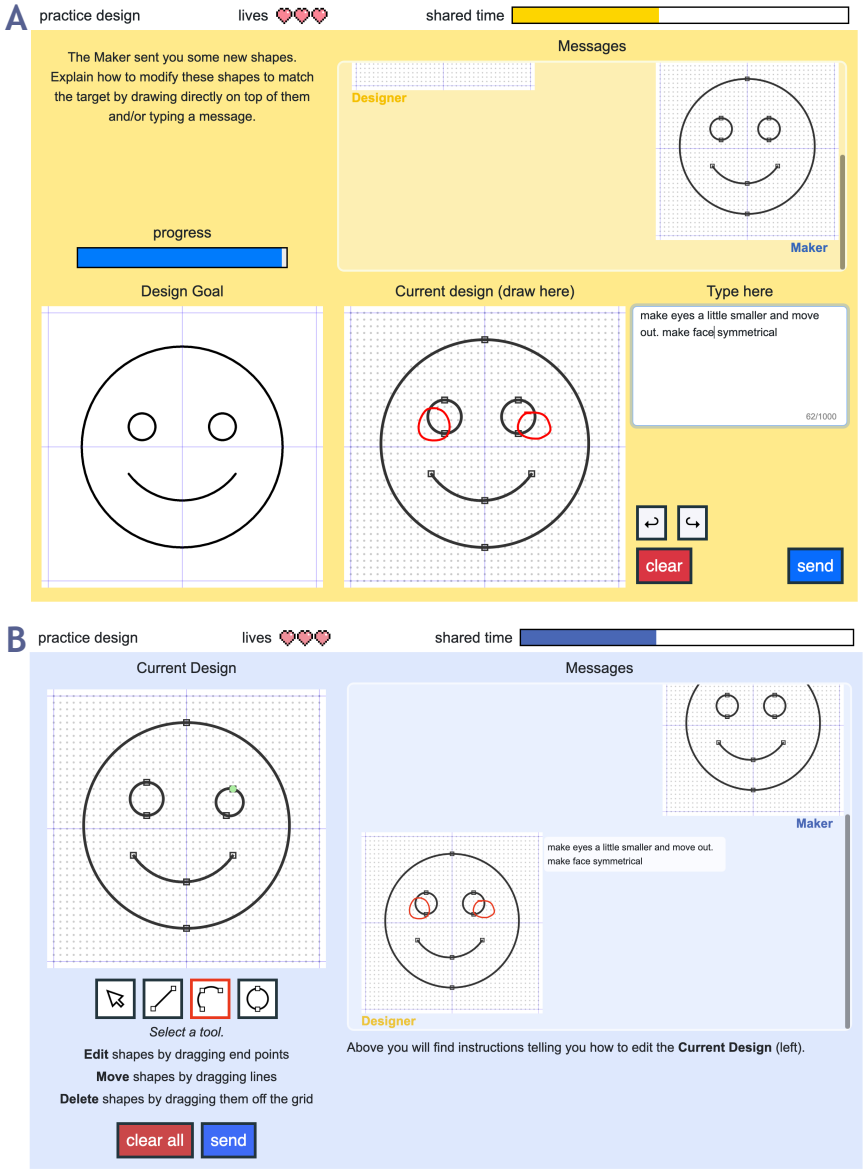


Figure B.1: **A** *Designer* display: target design is shown on the lower leftand instructions are created on the lower right, by drawing on the current CAD and by typing in the text box. History of prior interaction is shown in top right. **B** *Maker* display: on the *Maker*'s turn, the current CAD can be edited in CAD interface on the left, by dragging elements and control points, and by placing new elements by selecting a tool and clicking on grid squares to place control points. Instructions are shown in chat window on right. Both *Designer* and *Maker* can see shared time and number of lives remaining. This target was shown to all participants as a practice trial.

several changes between versions to increase efficiency of data collection.

	studies	**dataset**
Designer turn time	60s (180s)	unlim.
Maker turn time	120s (360s)	unlim.
Trial time limit	unlim.	up to 540s
Base payment	$10	$9.10
Trial bonus	0	$1.00
Lives	N/A	3
Trials per dyad	6	1-13
Rounds per trial	3	1-10
Character limit	200 (600s)	unlim.
Drawing limit	unlim.	unlim.

Table B.1: Parameters for ablation studies (generation-only in parentheses) and primary dataset collection. In experiments, we manipulated access to modalities and refinement rounds (across participants). For dataset collection, participants had up to 540 seconds per trial, but time elapsed at double speed for the *Designer* to discourage precisely drawn copies of the target.

B.3.4 Sampling CAD tasks

We sample tasks from the SketchGraphs dataset Seff et al. (2020), which is released under the permissive MIT license. We first normalize all designs to a 20×20 grid to identify duplicates. We then rescale the designs by a random scaling factor to ensure a variety of design sizes, while ensuring a minimum gap between pairs of elements such as control points, parallel lines, and concentric arcs or circles to ensure that they were far enough apart to be reproducible in the CAD construction interface. We then grouped designs into buckets based on a 'signature' determined by the count of various types of curves in the design (horizontal lines, vertical lines, skewed lines, arcs, and circles). By selecting one design per bucket, we: (1) eliminated most near-duplicates (2) select a more diverse set of designs. Participants were given 12 tasks randomly sampled from the task pool and shown in order of increasing difficulty (increasing number of curves in the design).

B.3.5 Prioritizing motivated annotators

The mrCAD task is challenging. Successful performance requires a person to (1) learn to operate a new CAD interface and (2) be adept at communicating CAD modifications, all within the span of one online session. Furthermore, in a two player game, *both* players must be motivated and capable for a successful reconstruction. How can we prioritize capable

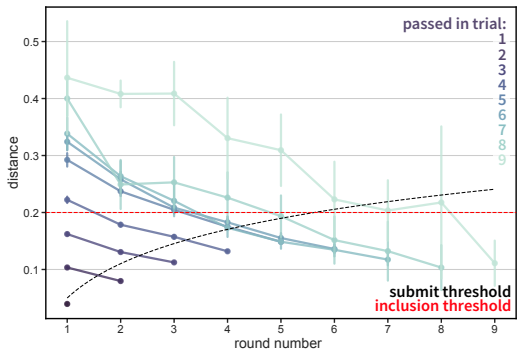


Figure B.2: We implemented a dynamic threshold for submitting designs that became more lenient in later rounds. Participants took a variable number of rounds to reach the threshold. Visualizing distance to the target broken down by round submitted reveals a trend of refinement over time. Red dashed line indicates the fixed threshold for including in analysis. A. Dyads successfully refined CADs over multiple rounds, and could submit their CAD once they reached the threshold, which became more lenient as rounds went on. Only CADs that meet our fixed threshold are included in the full mrCAD dataset.

dyads to provide high-quality annotations, while fairly compensating capable individuals with an unmotivated partner? We devised an incentive scheme as follows.

Prioritizing capable individuals Before participants started performing CAD tasks, they had to pass through three checks: a comprehension quiz, a solo CAD reconstruction task, and a paired practice trial. In the solo CAD reconstruction task, each participant has to recreate a simple CAD consisting of a line, circle, and arc, to our fixed threshold level of accuracy. All participants who passed the solo reconstruction task are deemed capable and motivated, and received a base pay of ($8.70).

Prioritizing capable dyads Once paired, dyads had to successfully recreate one practice trial (a simple smiley face) in order to attempt other CADs. We awarded a $1 bonus per person for every recreation that met the accuracy threshold. If a dyad failed to meet the threshold in 9 minutes, they lost a "life", and were ejected from the study when they lost 3 lives. This limited the number of CADs that unmotivated dyads could attempt, directing study time and compensation towards higher quality reconstructions.

B.3.6 Prioritizing refinement instructions

Our primary goal was to collect a large number of *multimodal refinement instructions* that led to high-quality reconstructions. A simple way of ensuring high-quality reconstructions it to implement a fixed threshold of accuracy for submitting CADs. However, this fixed threshold

does not take into account that some CAD would be easier to reconstruct while others would be more challenging. In a pilot study, we found that with a lenient threshold, unmotivated dyads submitted low quality reconstructions *without refinements*, while with a stringent threshold, motivated dyads were not compensated for decent quality reconstructions on more complex CADs.

Therefore, we implemented a **dynamic submission threshold**, which allowed only highly accurate reconstructions to be submitted in early rounds, but was more permissive in later rounds. Implementing this change allowed us to collect 1,532 additional refinement rounds, an increase of 10.1% compared to using a static threshold. Furthermore, a dynamic threshold kept motivated dyads who had been "stumped" by a difficult CAD in the study longer by not losing a life. The effect of dynamic threshold can be seen in Figure B.2.

B.3.7 Exclusion criteria

Rollouts that contained no CAD actions, entirely empty messages, or had missing rounds were excluded from the dataset for analysis. We also imposed a fixed **inclusion threshold** (Figure 5.4A) on reconstruction accuracy for analysis. In the statistics below, we report only data that meets these criteria, however our full dataset release also includes failures and practice trials.

B.4 Vision-language model experiments

B.4.1 Hardware and software details

Evaluation We queried API models using OpenRouter. For Qwen models, we hosted the model on a local server 1 Nvidia A6000, 6000Ada, L40, or L40S GPU (each of which has 48GB VRAM) using vLLM Kwon et al. (2023). For each instruction in context, we sampled one response from the model. For locally hosted models, each run (through the entire evaluation set with 2324 rollouts) took approximately 19 GPU hours. Evaluation hyperparameters are shown in Table B.2.

Training We implemented finetuning using the supervised finetuning functionality provided by the TRL library von Werra et al. (2020). Each finetuning run (with a max of 10 epochs over the 2684 training rollouts) took 32–48 GPU hours. We used 4 Nvidia L40 or L40S GPUs with 48GB of VRAM for each run. Training hyperparameters are in Table B.3.

B.4.2 Prompt

Figure B.3 shows the structure of the prompts used in experiments.

You are an expert CAD software user playing a game called mrCAD. In this game, there is a designer and a maker. The two players work together to iteratively create a design over a sequence of turns. You will play the role of the maker in this game, and the user will play the role of the designer. In each turn the designer provides an instruction about how to modify the design on the canvas. The instruction may include language instructions, drawings on the canvas, or both. The drawings appear as red strokes on the canvas. The design appears in black strokes on the canvas. Your goal is to follow the designer's instructions. You have to take actions to edit the current state of the design. Each action is taken by calling a tool that performs the action. Each control point is a pair of floating point numbers between -20 and 20 that represent the coordinates of the point on the canvas.

New game:

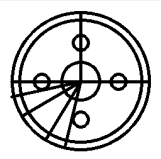
Round 1.

big round circle with a smaller circle inside it sort of resembling a clock

Edit the design based on the designer's instructions using the provided tools. Make sure to follow the instructions carefully.

[{"name": "make_curve", "arguments": {"type": "circle", "control_points": [[0.0, -18.0], [0.0, 18.0]]}}, {"name": "make_curve", "arguments": {"type": "circle", "control_points": [[0.0, -15.0], [0.0, 15.0]]}}, {"name": "move_point", "arguments": {"point": [0.0, -15.0], "new_point": [0.0, -16.0]}}, ...]

The resulting design is:



{"curves": [{"type":"circle","control_points":[[0.0,-18.0],[0.0,18.0]]}, {"type": "circle", "control_points":[[0.0,-16.0],[0.0,16.0]]}, {"type": "circle", "control_points": [[0.0,-12.0],[0.0,-8.0]]}, ...]}

Figure B.3: Structure of the prompt used for API calls as well as SFT models. The prompt includes system messages, templated parts of the instruction presented as user messages, *Designer* instructions, *Maker* responses as assistant messages, and environment feedback presented as user messages. For Qwen models, a description of the tools is presented as part of the system message (not shown here). For other models, the tool descriptions are integrated by the server.

	Qwen-7B(-Instruct and -FT)	others
temperature	0.7	1
top-p	0.95	1

Table B.2: Prompting hyperparameters

	value
LoRA parameters	all linear layers
LoRA rank	128
LoRA α	32
learning rate	10^{-4}
warmup steps	10
batch size	1 / GPU $\times$ 4 GPUs = 4
gradient accumulation steps	4

Table B.3: Fine-tuning hyperparameters for all runs

B.5 Analyzing the number of actions

We studied how agents edit the design by counting the editing actions each agent took. We found (in Table B.4) that in the ablated context (when only the most recent instruction in the sequence of refinement instructions was presented), models took fewer editing actions. By performing fewer bad actions, models are able to achieve better scores. However, the negative value highlights that models don't take a sufficient number of good actions to improve the design. We would also like to note the effect of finetuning here. While more actions results in worse performance for off-the-shelf models, the finetuned model is able to take more actions while also improving performance. While the overall performance is still poor, this change highlights the potential for approaches that better leverage the training data to improve multi-turn, multimodal refinement abilities of models in this setting.

Model	# generation actions	# refinement actions
Human	17.59	8.31
GPT-4o	4.70	4.13
−context	–	2.63
GPT-4o-mini	3.90	5.35
−context	–	3.21
Claude-3.7	2.02	4.29
Qwen2.5-7B	1.39	1.11
Qwen2.5-7B-FT	7.32	6.40

Table B.4: Average number of actions taken by models in generation and refinement stages

Appendix C

Learning to Form Conventions from Simulated Interactions

C.1 Base models

C.1.1 COCO base models

Speaker model For base models in COCO games, we use the pretrained versions of Gemma-3-12B (Gemma Team, 2025) and Pixtral-12B (Agrawal et al., 2024). We prompt these models to with one demonstration game, and the context of the game so far. For each game, we present the images once, and then reference the images using a label. So, we show the model images as `Image A: <image> Image B: <image>`, and then prompt the model to produce a description of an image by referencing its label. For the demonstration game, we choose a set of 4 COCO training images to create a context, and choose corresponding captions from the COCO dataset to use as utterances for these images. We note that these descriptions are written by people who do not have access to the other images in the context, nor to any prior conversation turns about that image, ensuring they do not reflect properties of convention formation. These captions serve as a guide for the model about the kind of descriptions we want it to produce – without it, the model produces very long utterances that sometimes do not reference the content of the image. For the demonstration game, we also use different labels (`M`, `N`, `O`, `P`) from the main game (`A`, `B`, `C`, `D`) to ensure the model does not confuse which image is being referenced.

In addition to these steps, we had to take additional measures to ensure the Pixtral model produced utterances of appropriate length. With just the measures above, we still found the Pixtral model producing excessively long descriptions, and failing to generate an end-of-sequence token that stopped generation. As a workaround, we used a different end-of-sequence marker (`<EOM>`) which we appended to the demonstration utterances, and used as a stop string while decoding from the model. This did not correspond to a special token in the model's vocabulary, and was tokenized as a string that was part of the template to generate prompts. We found that the model was able to learn to use this token in context, and produced utterances of appropriate length.

Listener model For the speaker model, we presented the images once, and referred to them only using labels. If we did the same for the listener model, it would be able to simply answer with the label associated with an image in a previous trial, without reasoning about

the image at all. To ensure the listener cannot exploit associations between labels and descriptions from previous trials, we present the images to the listener twice. The second presentation is before the most recent trial presented to the listener (for which the model has to make a guess), and includes the images in a shuffled order. In initial experiments we found that shuffling the images on second presentation led to poor listener accuracy. To address this, we finetuned the listener on games in this format.

We trained a Gemma-3-12B base model to serve as a listener. Since we finetuned the model, we did not include a demonstration game. We finetuned the model to predict the label of an image based on the context of the game so far, including the utterance for the most recent trial. We construct synthetic games to train the listener using COCO descriptions. We randomly sample a set of COCO training images to create a context, and then sample a sequence of trials with that context. In each trial, we sample one of the images, a corresponding caption from the COCO dataset, and train the model to predict the label of the image given the prompt. Again, we note that since these are descriptions from the COCO dataset, they are written out of context and do not reflect properties of convention formation.

We synthesize 250 games (5000 trials) for finetuning. We finetune all linear layers of the model with LoRA (Hu et al., 2022) with $r = 16$ and $\alpha = 32$. We use a batch size of 2 with 8 gradient accumulation steps (effective batch size of 16). We finetuned the model for a maximum of 5 epochs, and chose 10^{-4} as the learning rate warmed up over 1% of training and linearly decayed over the remaining steps, choosing between 10^{-4} and 10^{-5} based on validation loss on a held out set of 10 games. We choose the checkpoint with the lowest validation loss among checkpoints after each epoch. We use the `SFTTrainer` in the HuggingFace TRL library for training (von Werra et al., 2020).

C.1.2 Tangram base models

Speaker model For tangrams, we finetune the vision encoder of the Gemma-3-12B base model. We do this to avoid modifying the in-context learning abilities of the base model, while still adapting it to the new visual domain. We construct synthetic games for images drawn from the pretrain set (a subset of the training set; Section C.2) of Kilogram images. For descriptions, we use descriptions from human-human reference games over tangram reference games collected by Gul and Artzi (2024). They conduct multiple rounds of games, and release multiple descriptions of each Kilogram image, allowing us to create a dataset to train a model in the same way as for COCO images. Again, we note that even though these descriptions are from reference games, we use them out of their original context (we have 4 image reference games, while Gul and Artzi (2024) have 10 image reference games), and are single-turn interactions, ensuring they do not reflect properties of convention formation.

Additionally, for the speaker, we sample the listener's guess at random (so the speaker isn't biased to observing only correct guesses from the listener).

We finetune the full vision encoder of a Gemma-3-12B base model on 500 games (10000 trials). We use a batch size of 1/GPU on 4 GPUs, with 4 gradient accumulation steps (effective batch size of 16). We finetuned the model for a maximum of 10 epochs, with a learning rate of 10^{-5} warmed up over 1% of training and linearly decayed over the remaining steps. We choose the checkpoint with the lowest loss on a validation set of 25 games among checkpoints after each epoch.

Listener model For the listener model, we finetune the vision encoder first (like for the speaker model), and then finetune all the linear layers (with LoRA) to adapt the model to image shuffling (which requires more than adapting to the visual domain). We first finetune the vision encoder on 250 games (5000 trials) generated in a similar way as for training the COCO listener model. At this stage, we don't shuffle the images on the second presentation and maintain the same order. We use a batch size of 1/GPU on 4 GPUs, with 4 gradient accumulation steps (effective batch size of 16). We finetuned the model for a maximum of 10 epochs, with a learning rate of 10^{-5} warmed up over 1% of training and linearly decayed over the remaining steps. We choose the checkpoint with the lowest loss on a validation set of 25 games among checkpoints after each epoch.

Then, we use the same games to finetune all the linear layers with LoRA, using $r = 16$ and $\alpha = 32$. We use a batch size of 1/GPU on 4 GPUs, with 4 gradient accumulation steps (effective batch size of 16). We finetuned the model for a maximum of 10 epochs, with a learning rate of 10^{-5} warmed up over 1% of training and linearly decayed over the remaining steps. We choose the checkpoint with the lowest loss on a validation set of 25 games among checkpoints after each epoch.

C.2 Sampling contexts of images

We construct image contexts based on image similarity. The effects of convention formation are known to be context-dependent (Hawkins et al., 2023), so the choice of contexts is important. Images that are more similar are hard to discriminate in the same context, while dissimilar images are easier to tell apart. We determine image similarity using pretrained CLIP-style visual encoders.

For COCO, we create contexts of images from the COCO 2014 train set as the image bank. We embed all images using the `siglip2-base-patch16-512` (Tschannen et al., 2025) image encoder. For each context, we sample a seed image at random, and compute the cosine similarity of that image's embedding to all images in the bank (except itself). We

then divide the similarities by a temperature τ and apply the softmax function to get a categorical distribution over images. We then sample the other images without replacement from this distribution. We use 0.01, 0.02, 0.03, 0.04, and 0.05 as temperatures for sampling contexts, with 0.01 being the hardest contexts (most similar images), and 0.05 being the easiest (least similar images). We sample 100 contexts at each temperature for training. For evaluation we apply the same procedure to the COCO validation set. We sample 50 contexts (10 for each temperature) for validation, and another 75 contexts for testing (15 for each temperature).

For tangrams, we use the original splits from the Kilogram dataset (Ji et al., 2022b). We further partition the training set into two, with one part used for pretraining (finetuning the speaker's vision encoder and the listener), and the other part used for simulating games to produce preference training data. We use the CLIP encoders released by Ji et al. (2022b) (for whole, black images, following Gul and Artzi, 2024) to compute image similarity, and sample contexts at temperature 0.125, 0.25, 0.5, and 1.0. We sample 100 contexts at each temperature for training. We also apply the same procedure to the validation split of Kilogram images to construct the validation set (10 contexts at each temperature). For the test set, we use the test split of Kilogram images, and sample 40 contexts each at temperatures 0.125, 0.25.

C.3 Preference optimization

For COCO we simulate 500 games (10000 trials), and construct preference optimization pairs using the criteria described in Section 6.2. For tangrams, we simulate 400 games (8000 trials). We use top-p sampling with decoding temperature 1.0 and $p = 0.95$ to sample utterances. Given that the preference pairs are constructed from samples drawn from the model, the exact number of preference pairs varies based on the criteria used to construct them. We use a fixed number of games, and use all the preference pairs constructed from these games to train the model. During preference optimization, we use the same demonstration game used while simulating games.

We adapt all linear layers of the model the model with LoRA adapters ($r = 32$ and $\alpha = 32$), using IPO as the preference optimization algorithm. We train for 3 epochs. We use a batch size of 1/GPU on 4 GPUs, with 8 gradient accumulation steps (effective batch size of 32). We choose a learning rate from 10^{-6} and 10^{-5} and a KL regularization parameter β between 0.1, 0.2, 0.3 and 0.4 using a validation procedure (described below). We choose 10^{-5} as the learning rate and $\beta = 0.3$ for the Gemma model on COCO images and use the same parameters for all other Gemma experiments (COCO and tangram models, all utility functions). For Pixtral models (all utility functions), we use a learning rate of 10^{-6}

and $\beta = 0.3$, without searching over different values of these parameters. For evaluation of COCO models, we sweep over decoding temperature 0 (greedy decoding), 0.3, 0.7 and 1.0. Based on the validation procedure, we chose greedy decoding for Gemma and 0.7 for Pixtral.

For validation, we construct a set of images contexts from a held out set of images. We simulate games for context, generating 4 utterances for each trial (like we do when simulating games for training). We then use average listener accuracy across all samples at all trials (this can be viewed as the area under the curve of accuracy over repetitions of images) as the validation criterion.

C.4 Metrics

To evaluate message length, we use the spaCy package (`en_core_web_sm`, Honnibal et al. (2020)) to tokenize the utterance and count all the tokens. For WNR, we first compute the word novelty distance (WND). We follow Hua and Artzi (2024) in selecting the content words (with POS tag `NOUN`, `ADJ`, `VERB`, `ADV`, `PROPN`, `NUM`, `PRON`, or `ADP`), lemmatizing them with spaCy, and computing the edit distance between the previous utterance used to describe an image and the current utterance with no penalties for deletions. We then compute the WNR as the WND divided by the length of the previous utterance.

We also map POS tags to part-of-speech types (for Figure 6.7) as follows:

C.5 Human study

We recruit participants through Prolific. Based on pilot experiments, we found that two model speaker–human listener games take 15 minutes to complete, and one human speaker–human listener game takes 15 minutes to complete (human speakers take longer to type a message than a model takes to produce one). Based on these estimates, we compensated participants with US$1.50 for each model speaker-human listener game, and US$3.00 for each human speaker–human listener game. This amounts to an hourly rate of US$12.00. We also incentivized participants with a US$0.04 bonus for every correct trial (for a listener, this was choosing the correct image, for a speaker, this was producing an utterance that led to the listener choosing the correct image). We recruit participants who are located in the United States, are fluent in English, have at least 1000 approved Prolific submissions, and an approval rate of at least 99% on Prolific. Our study was deemed exempt from review by our Institutional Review Board.

We conducted experiments separately for model-as-a-speaker games and human-as-a-speaker games. For the model-as-a-speaker setting, the games proceeded as follows:

POS Tag	Part-of-speech type
ADJ	adjectives
ADP	prepositions
ADV	adverbs
AUX	other
CCONJ	conjunctions
DET	determiners
INTJ	other
NOUN	nouns
NUM	other
PART	other
PRON	pronouns
PROPN	nouns
PUNCT	other
SYM	other
VERB	verbs
X	other
SCONJ	conjunctions

Table C.1: Mapping from POS tags to part-of-speech types used in Figure 6.7.

- The participant signs a consent form (agreeing to their data being used for this research study, and declaring that they are at least 18 years of age, and are located in the US), and read instructions for the task.
- The participant proceeds to play a game with the first model. This model is chosen by deciding with probability 0.5 whether it will be SUCCESS+COST model or one of the baseline models. If it is one of the baseline models, one of the two baseline models is chosen with probability 0.5.
- The participant answers questions about the first game they played.
- The participant proceeds to play a game with the second model. If the SUCCESS+COST model was chosen in the first game, the second model one of the baseline models chosen with probability 0.5. If it is one of the baseline models, then the second model is the SUCCESS+COST model.
- The participant answers questions about the second game they played.
- The participant is given a set of questions to answer comparing both the models they interacted with.

For the human-as-a-speaker setting, participants signed a consent form, read instructions

	Speaker	#
COCO	SUCCESS+COST	63
	Gemma-3-12B-Instruct	34
	SUCCESS	29
	Human	30
Tangrams	SUCCESS+COST	60
	Gemma-Tangrams-Base	25
	SUCCESS	35
	Human	12

Table C.2: The number of games with each speaker.

for the task, played a game, and then answered questions about the game if they were a listener.

For model-as-a-speaker games, we excluded all data from participants who did not complete both games. For the human-as-a-speaker setting, we excluded incomplete games (due to issues such as disconnection). A bug in our human-human study interface also led to some participants re-entering the pool and getting paired to play another game. We excluded all games where either participant was not playing their first game. The number of games with each speaker is shown in Table C.2. For tangrams, we also excluded data from one human speaker–human listener pair where the human speaker did not send informative messages, leading to the listener getting chance accuracy.

C.5.1 Interface

We conducted our study on a web interface, adapted from the one released by Hawkins et al. (2020a). For model-as-a-speaker games, we used colors to refer to the models, so as to not reveal their identity/nature to the participant. We always referred to the SUCCESS+COST model as the blue robot, and the SUCCESS model as the yellow robot, and the Gemma-Tangrams-Base/Gemma-3-12B-Instruct model as the orange robot.

The listener interface is shown in Figure C.1. In addition to this interface, we recorded the participant's response time for each trial, ensuring to keep the clock running only while the participant's tab was active. The images are shuffled after each trial.

For human-as-a-speaker games, we used the same interface as the model-as-a-speaker games for the listener. The speaker is shown the images as in Figure C.2, and are instructed to describe the highlighted image.

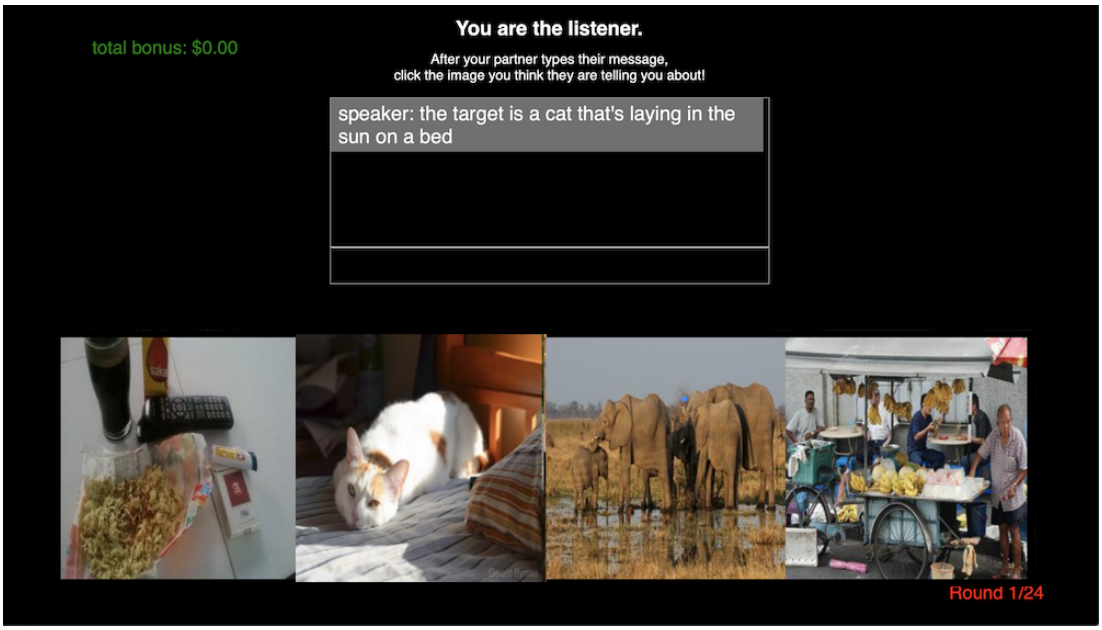


Figure C.1: The listener's interface. While the speaker is typing, or the model is generating a response, the box indicates that the other player is typing. Once their message is sent, it is displayed in the box.

C.5.2 Questions

We asked participants questions at different stages, each time after they had completed a game. After playing a game with a model, they answered the following questions, asking whether they agreed with the following statements on a 5 point scale from 1 (strongly disagree) to 5 (strongly agree).

- **Mental Demand**: It was easy to understand the speaker's descriptions.
- **Temporal Demand**: I was able to quickly understand the speaker's descriptions.
- **Performance**: I was able to successfully interpret the speaker's descriptions.

After completing both games, they answered the following questions, phrased based on whether they played with the yellow or orange model. The responses are on a 5 point scale from 1 (strongly disagree) to 5 (strongly agree).

- **Mental Demand**: I found it easier to understand the blue robot's descriptions than the (orange/yellow) robot's descriptions.
- **Temporal Demand**: I was able to more quickly understand the blue robot's descriptions than the (orange/yellow) robot's descriptions.

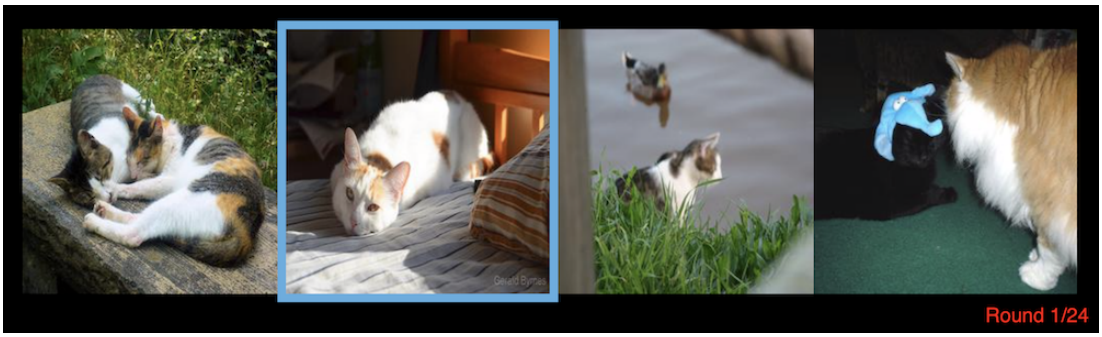


Figure C.2: The speaker's view of images. The highlighted image is the one that the speaker is describing.

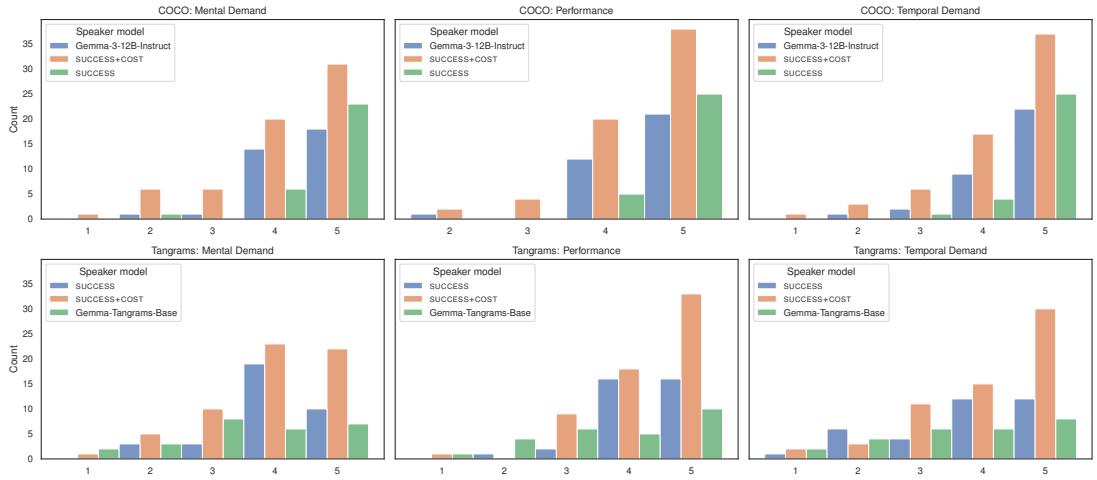


Figure C.3: Questions asked to participants about the model with which they interacted.

- **Performance**: I was more successful at interpreting the blue robot's descriptions than the (orange/yellow) robot's descriptions.

For COCO games, people generally gave the models high ratings on all parameters (Figure C.3, top row), and did not have any clear preferences between the models (Figure C.4, top row). In tangram games, people showed a preference for the SUCCESS+COST model over Gemma-Tangrams-Base (Figure C.4, bottom row), and generally had lower ratings for the Gemma-Tangrams-Base model (Figure C.3, bottom row).

C.5.3 Characterizing the consistency of human speakers

Like Hua et al. (2025), we found that not all human speakers showed the kind of adaptation behavior observed in previous work. While the reasons for this are not known, we follow Hua et al. (2025) in dividing speakers into two groups: high consistency and low consistency

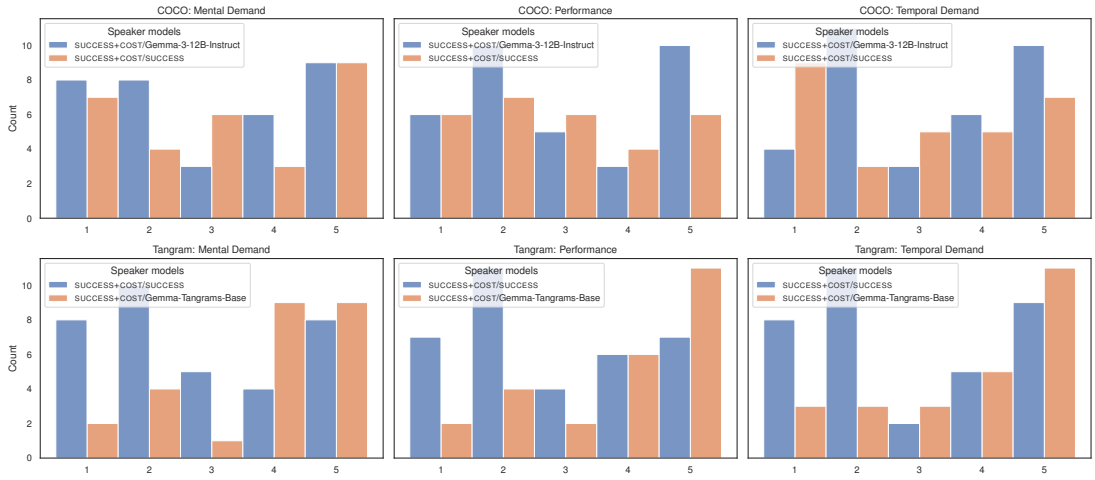


Figure C.4: Questions asked to participants comparing different models.

by clustering speakers based on word novelty. For each human speaker, we compute the word novelty distance (WND; which is the WNR without normalizing by the length of the previous utterance) in the last two blocks of the game. By that point in the game, we expect stable conventions to have formed. We see that not all speakers have low WND at this stage, with some maintaining a very high WND even late into the game.

We follow Hua et al. (2025) in fitting a Gaussian mixture model (GMM) to the average WND in the last two blocks of the game. For COCO games, we speaker over 1–8 components, and 25 random seeds to find a model with the lowest Bayesian Information Criterion (BIC). The best model has 7 components, with means [0.134, 1.187, 2.655, 5.625, 9.875, 4.625, 3.750]. The first two clusters have low average WND, indicating high consistency speakers. The remaining clusters have high average WND, indicating low consistency speakers. We have 21 high consistency speakers and 9 low consistency speakers.

For tangram games, given that we have fewer games, we searched over 1–4 components and 25 random seeds. The best model has 4 components, with means [0.417, 2.667, 5.417, 3.750]. The first cluster has low average WND, indicating high consistency speakers. The remaining clusters have high average WND, indicating low consistency speakers. We are left with 6 low consistency speakers and 6 high consistency speakers.

C.6 Tangrams model-as-a-listener results

We conduct experiments on tangram images with a model listener. To ensure we have a strong model for the visual domain, we use the same listener model that we use for generating feedback during training. We perform inference the same was as for human listener experiments. Results are shown in Figure C.5, and reflect similar trends to human

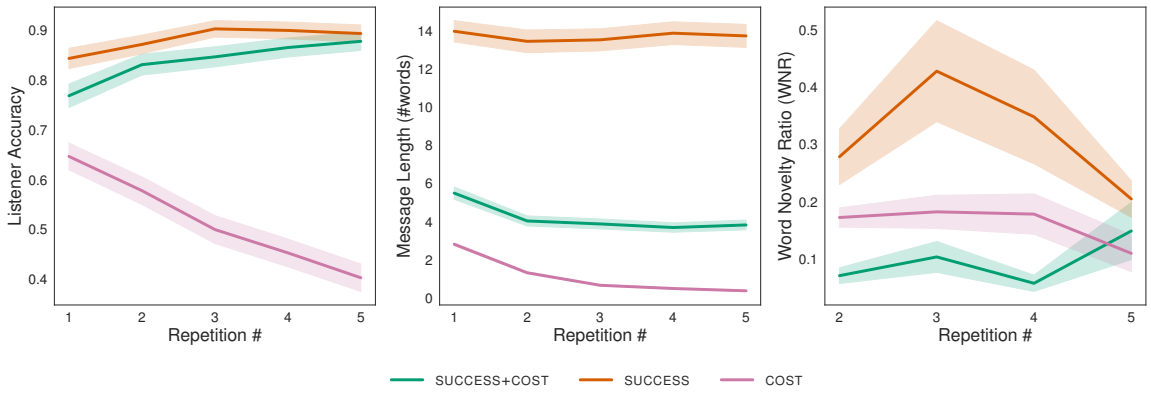


Figure C.5: In tangram experiments with a model listener, the SUCCESS+COST and SUCCESS speakers show similar trends to human listener experiments. When training with COST alone, we see that the listener accuracy decreases, even though the messages are shorter.

listener experiments, and to COCO games for the COST speaker.

Bibliography

Pravesh Agrawal, Szymon Antoniak, Emma Bou Hanna, Baptiste Bout, Devendra Chaplot, Jessica Chudnovsky, Diogo Costa, Baudouin De Monicault, Saurabh Garg, Theophile Gervet, Soham Ghosh, Amélie Héliou, Paul Jacob, Albert Q. Jiang, Kartik Khandelwal, Timothée Lacroix, Guillaume Lample, Diego Las Casas, Thibaut Lavril, and 23 others. 2024. Pixtral 12b. *Preprint*, arXiv:2410.07073.

Jacob Andreas and Dan Klein. 2016. Reasoning about pragmatics with neural listeners and speakers. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1173–1182, Austin, Texas. Association for Computational Linguistics.

Anthropic. 2025. Claude 3.7 sonnet system card. Technical report, Anthropic. Accessed: 2025-03-01.

Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, and 8 others. 2025. Qwen2.5-vl technical report. *Preprint*, arXiv:2502.13923.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, and 1 others. 2022. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*.

Matej Balog, Alexander L. Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. 2017. Deepcoder: Learning to write programs. In *International Conference on Learning Representations*.

Jonathan Berant, Maximillian Chen, Adam Fisch, Reza Aghajani, Fantine Huot, Mirella Lapata, and Jacob Eisenstein. 2025. Learning steerable clarification policies with collaborative self-play. *arXiv preprint arXiv:2512.04068*.

Leon Bergen, R. Levy, and Noah D. Goodman. 2016. Pragmatic reasoning through semantic inference. *Semantics and Pragmatics*, 9.

Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault de Chezelles, Quentin Cappart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. 2024. WorkArena++: Towards compositional planning and reasoning-based common knowledge work tasks. In *Advances in Neural Information Processing Systems*.

M Akmal Butt and Petros Maragos. 1998. Optimum design of chamfer distance transforms. *IEEE Transactions on Image Processing*, 7(10):1477–1484.

Federico Cassano, Luisa Li, Akul Sethi, Noah Shinn, Abby Brennan-Jones, Jacob Ginesin, Edward Berman, George Chakhnashvili, Anton Lozhkov, Carolyn Jane Anderson, and 1 others. 2023. Can it edit? evaluating the ability of large language models to follow code editing instructions. *arXiv preprint arXiv:2312.12450*.

Kranti Chalamalasetti, Jana Götze, Sherzod Hakimov, Brielen Madureira, Philipp Sadler, and David Schlangen. 2023. clembench: Using game play to evaluate chat-optimized language models as conversational agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 11174–11219, Singapore. Association for Computational Linguistics.

Bingsen Chen, Boyan Li, Ping Nie, Yuyu Zhang, Xi Ye, and Chen Zhao. 2026. Beyond single-shot writing: Deep research agents are unreliable at multi-turn report revision. *arXiv preprint arXiv:2601.13217*.

Hansheng Chen, Ruoxi Shi, Yulin Liu, Bokui Shen, Jiayuan Gu, Gordon Wetzstein, Hao Su, and Leonidas Guibas. 2024. Generic 3d diffusion adapter using controlled multi-view editing. *arXiv preprint arXiv:2403.12032*.

Valerie Chen, Ameet Talwalkar, Robert Brennan, and Graham Neubig. 2025a. Code with me or for me? how increasing ai automation transforms developer workflows. *ArXiv*, abs/2507.08149.

Xinyun Chen, Petros Maniatis, Rishabh Singh, Charles Sutton, Hanjun Dai, Max Lin, and Denny Zhou. 2021. Spreadsheetcoder: Formula prediction from semi-structured context. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 1661–1672. PMLR.

Zizhao Chen, Mustafa Omer Gul, Yiwei Chen, Gloria Geng, Anne Wu, and Yoav Artzi. 2025b. Retrospective learning from interactions. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24580–24606, Vienna, Austria. Association for Computational Linguistics.

Wayne Chi, Valerie Chen, Ryan Shar, Aditya Mittal, Jenny Liang, Wei-Lin Chiang, Anastasios Nikolas Angelopoulos, Ion Stoica, Graham Neubig, Ameet Talwalkar, and Chris Donahue. 2026. EditBench: Evaluating LLM abilities to perform real-world instructed code edits. In *International Conference on Learning Representations*.

Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, and 1 others. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53.

Herbert H. Clark. 1996. *Using Language*. Cambridge University Press, Cambridge.

Herbert H. Clark and Deanna Wilkes-Gibbs. 1986. Referring as a collaborative process. *Cognition*, 22(1):1–39.

Reuben Cohn-Gordon and Noah Goodman. 2019. Lost in machine translation: A method to reduce meaning loss. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 437–441, Minneapolis, Minnesota. Association for Computational Linguistics.

Reuben Cohn-Gordon, Noah Goodman, and Christopher Potts. 2018. Pragmatically informative image captioning with character-level inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 439–443, New Orleans, Louisiana. Association for Computational Linguistics.

Jonathan Cook, Diego Antognini, Martin Klissarov, Claudiu Musat, and Edward Grefenstette. 2026. Learning to learn from language feedback with social meta-learning. *Preprint*, arXiv:2602.16488.

Jacob Devlin, Jonathan Uesato, Surya Bhupatiraju, Rishabh Singh, Abdel rahman Mohamed, and Pushmeet Kohli. 2017. RobustFill: Neural program learning under noisy I/O. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 990–998. PMLR.

Mark Dingemanse and Nick J Enfield. 2024. Interactive repair and the foundations of language. *Trends in Cognitive Sciences*, 28(1):30–42.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. 2024a. WorkArena: How capable are web agents at solving common knowledge work tasks? In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11642–11662. PMLR.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. 2024b. WorkArena: How capable are web agents at solving common knowledge work tasks?

In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11642–11662. PMLR.

Yann Dubois, Percy Liang, and Tatsunori Hashimoto. 2024. Length-controlled alpacaeval: A simple debiasing of automatic evaluators. In *First Conference on Language Modeling*.

Jacob Eisenstein, Reza Aghajani, Adam Fisch, Dheeru Dua, Fantine Huot, Mirella Lapata, Vicky Zayats, and Jonathan Berant. 2025. Don't lie to your friends: Learning what you know from collaborative self-play. In *Proceedings of the Conference on Language Modeling*.

Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. 2021. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2021, page 835–850, New York, NY, USA. Association for Computing Machinery.

FAIR and 1 others. 2022. Human-level play in the game of Diplomacy by combining language models with strategic reasoning. *Science*, 378(6624):1067–1074.

Judith E Fan, Wilma A Bainbridge, Rebecca Chamberlain, and Jeffrey D Wammes. 2023. Drawing as a versatile cognitive tool. *Nature Reviews Psychology*, 2(9):556–568.

Yu Feng, Ruben Martins, Osbert Bastani, and Isil Dillig. 2018. Program synthesis using conflict-driven learning. *ACM SIGPLAN Notices*, 53(4):420–435.

Margarida Ferreira, Miguel Terra-Neves, Miguel Ventura, Inês Lynce, and Ruben Martins. 2021. Forest: An interactive multi-tree synthesizer for regular expressions. In *Tools and Algorithms for the Construction and Analysis of Systems*, pages 152–169, Cham. Springer International Publishing.

Michael C. Frank and Noah D. Goodman. 2012a. Predicting pragmatic reasoning in language games. *Science*, 336(6084):998–998.

Michael C. Frank and Noah D. Goodman. 2012b. Predicting pragmatic reasoning in language games. *Science*, 336(6084):998–998.

Daniel Fried, Jacob Andreas, and Dan Klein. 2018a. Unified pragmatic models for generating and following instructions. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 1951–1963, New Orleans, Louisiana. Association for Computational Linguistics.

Daniel Fried, Ronghang Hu, Volkan Cirik, Anna Rohrbach, Jacob Andreas, Louis-Philippe Morency, Taylor Berg-Kirkpatrick, Kate Saenko, Dan Klein, and Trevor Darrell. 2018b. Speaker-follower models for vision-and-language navigation. In *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc.

Tsu-Jui Fu, Wenze Hu, Xianzhi Du, William Yang Wang, Yinfei Yang, and Zhe Gan. 2024. Guiding instruction-based image editing via multimodal large language models. In *International Conference on Learning Representations*.

Ge Gao, Alexey Taymanov, Eduardo Salinas, Paul Mineiro, and Dipendra Misra. 2024. Aligning LLM agents by learning latent preference from user edits. In *Advances in Neural Information Processing Systems*.

Jiaxin Ge, Zora Zhiruo Wang, Xuhui Zhou, Yi-Hao Peng, Sanjay Subramanian, Qinyue Tan, Maarten Sap, Alane Suhr, Daniel Fried, Graham Neubig, and 1 others. 2025. Autopresent: Designing structured visuals from scratch. *arXiv preprint arXiv:2501.00912*.

Gemini Team, Google. 2025. Gemini 3 flash model card. Technical report, Google DeepMind.

Gemma Team. 2025. Gemma 3 technical report. *Preprint*, arXiv:2503.19786.

Mohammad Gheshlaghi Azar, Zhaohan Daniel Guo, Bilal Piot, Remi Munos, Mark Rowland, Michal Valko, and Daniele Calandriello. 2024. A general theoretical paradigm to understand learning from human preferences. In *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*, volume 238 of *Proceedings of Machine Learning Research*, pages 4447–4455. PMLR.

Nelson Goodman. 1976. *Languages of art: An approach to a theory of symbols*. Hackett publishing.

Herbert P Grice. 1975. Logic and conversation. In *Speech acts*, pages 41–58. Brill.

Mustafa Omer Gul and Yoav Artzi. 2024. Cogen: Learning from feedback with coupled comprehension and generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 12966–12982, Miami, Florida, USA. Association for Computational Linguistics.

Sumit Gulwani. 2011. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '11, page 317–330, New York, NY, USA. Association for Computing Machinery.

Janosch Haber, Tim Baumgärtner, Ece Takmaz, Lieke Gelderloos, Elia Bruni, and Raquel Fernández. 2019. The PhotoBook dataset: Building common ground through visually-grounded dialogue. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1895–1910, Florence, Italy. Association for Computational Linguistics.

Robert Hawkins, Minae Kwon, Dorsa Sadigh, and Noah Goodman. 2020a. Continual adaptation for efficient machine communication. In *Proceedings of the 24th Conference on Computational Natural Language Learning*, pages 408–419, Online. Association for Computational Linguistics.

Robert D. Hawkins, Michael C. Frank, and Noah D. Goodman. 2020b. Characterizing the dynamics of learning in repeated reference games. *Cognitive Science*, 44(6):e12845.

Robert D. Hawkins, Michael Franke, Michael C. Frank, Adele E. Goldberg, Kenny Smith, Thomas L. Griffiths, and Noah D. Goodman. 2023. From partners to populations: A hierarchical bayesian account of coordination and convention. *Psychological Review*, 130(4):977–1016.

Luke Hewitt, Tuan Anh Le, and Joshua Tenenbaum. 2020. Learning to learn generative programs with memoised wake-sleep. In *Proceedings of the 36th Conference on Uncertainty in Artificial Intelligence (UAI)*, volume 124 of *Proceedings of Machine Learning Research*, pages 1278–1287. PMLR.

Mark K Ho, Michael Littman, James MacGlashan, Fiery Cushman, and Joseph L Austerweil. 2016. Showing versus doing: Teaching by demonstration. In *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. The curious case of neural text degeneration. In *International Conference on Learning Representations*.

Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. 2020. spaCy: Industrial-strength Natural Language Processing in Python.

Nicola Horst, Davide Mazzaccara, Antonia Schmidt, Michael Sullivan, Filippo Momentè, Luca Franceschetti, Philipp Sadler, Sherzod Hakimov, Alberto Testoni, Raffaella Bernardi, Raquel Fernández, Alexander Koller, Oliver Lemon, David Schlangen, Mario Giulianelli, and Alessandro Suglia. 2025. Playpen: An environment for exploring learning through conversational interaction. *Preprint*, arXiv:2504.08590.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Yilun Hua and Yoav Artzi. 2024. Talk less, interact better: Evaluating in-context conversational adaptation in multimodal LLMs. In *First Conference on Language Modeling*.

Yilun Hua, Evan Wang, and Yoav Artzi. 2025. Post-training for efficient communication via convention formation. In *Second Conference on Language Modeling*.

Holly Huey, Caren M Walker, and Judith E Fan. 2021. How do the semantic properties of visual explanations guide causal inference? In *Proceedings of the annual meeting of the cognitive science society*, volume 43.

Anya Ji, Noriyuki Kojima, Noah Rush, Alane Suhr, Wai Keen Vong, Robert D Hawkins, and Yoav Artzi. 2022a. Abstract visual reasoning with tangram shapes. *arXiv preprint arXiv:2211.16492*.

Anya Ji, Noriyuki Kojima, Noah Rush, Alane Suhr, Wai Keen Vong, Robert D. Hawkins, and Yoav Artzi. 2022b. Abstract visual reasoning with tangram shapes. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 582–601, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Jin-Hwa Kim, Nikita Kitaev, Xinlei Chen, Marcus Rohrbach, Byoung-Tak Zhang, Yuandong Tian, Dhruv Batra, and Devi Parikh. 2019. CoDraw: Collaborative drawing as a testbed for grounded goal-driven communication. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 6495–6513, Florence, Italy. Association for Computational Linguistics.

Ksenia Konyushkova, Christos Kaplanis, Serkan Cabi, and Misha Denil. 2025. Vision-language model dialog games for self-improvement. *Preprint*, arXiv:2502.02740.

Robert M. Krauss and Sidney Weinheimer. 1964. Changes in reference phrases as a function of frequency of usage in social interaction: a preliminary study. *Psychonomic Science*, 1:113–114.

Alexander Ku, Peter Anderson, Roma Patel, Eugene Ie, and Jason Baldridge. 2020. Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4392–4412, Online. Association for Computational Linguistics.

Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, and 1 others. 2025. Measuring AI ability to complete long software tasks. In *Advances in Neural Information Processing Systems*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. 2025. Llms get lost in multi-turn conversation. *Preprint*, arXiv:2505.06120.

Royi Lachmy, Valentina Pyatkin, Avshalom Manevich, and Reut Tsarfaty. 2022. Draw me a flower: Processing and grounding abstraction in natural language. *Transactions of the Association for Computational Linguistics*, 10:1341–1356.

Cassidy Laidlaw, Eli Bronstein, Timothy Guo, Dylan Feng, Lukas Berglund, Justin Svegliato, Stuart Russell, and Anca Dragan. 2025. AssistanceZero: Scalably solving assistance games. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 32278–32305. PMLR.

Bryan Lawson. 2006. *How designers think*. Routledge.

Angeliki Lazaridou, Anna Potapenko, and Olivier Tieleman. 2020. Multi-agent communication meets natural language: Synergies between functional and structural language learning. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7663–7674, Online. Association for Computational Linguistics.

Mina Lee, Tatsunori B. Hashimoto, and Percy Liang. 2019. Learning autocomplete systems as a communication game. *Preprint*, arXiv:1911.06964.

David K. Lewis. 1969. *Convention: A Philosophical Study*. Harvard University Press, Cambridge, MA.

Mike Lewis, Denis Yarats, Yann Dauphin, Devi Parikh, and Dhruv Batra. 2017. Deal or no deal? End-to-end learning of negotiation dialogues. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*.

Jessy Lin, Daniel Fried, Dan Klein, and Anca Dragan. 2022. Inferring rewards from language in context. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8546–8560, Dublin, Ireland. Association for Computational Linguistics.

Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. 2014. Microsoft coco: Common objects in context. In *Computer Vision – ECCV 2014*, pages 740–755, Cham. Springer International Publishing.

Xiao Liu, Tianjie Zhang, Yu Gu, Iat Long Iong, Yifan Xu, Xixuan Song, Shudan Zhang, Hanyu Lai, Xinyi Liu, Hanlin Zhao, and 1 others. 2024. VisualAgentBench: Towards large multimodal models as visual foundation agents. *arXiv preprint arXiv:2408.06327*.

William P McCarthy, Justin Matejka, Karl DD Willis, Judith E Fan, and Yewen Pu. 2024. Communicating design intent using drawing and text. In *Proceedings of the 16th Conference on Creativity & Cognition*, pages 512–519.

William P McCarthy*, **Saujas Vaduguru***, Karl D.D. Willis, Justin Matejka, Judith E Fan, Daniel Fried, and Yewen Pu. 2025. mrCAD: Multimodal communication to refine computer-aided designs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*.

Bill McDowell and Noah Goodman. 2019. Learning from omission. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 619–628, Florence, Italy. Association for Computational Linguistics.

Will Monroe and Christopher Potts. 2015. Learning in the rational speech acts model. In *20 th Amsterdam Colloquium*.

Anjali Narayan-Chen, Prashant Jayannavar, and Julia Hockenmaier. 2019. Collaborative dialogue in Minecraft. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 5405–5415, Florence, Italy. Association for Computational Linguistics.

Ansong Ni, Ruta Desai, Yang Li, Xinjie Lei, Dong Wang, Jiemin Zhang, Jane Yu, Ramya Raghavendra, Gargi Ghosh, Shang-Wen Li, and Asli Celikyilmaz. 2025. Collaborative reasoner: Self-improving social agents with synthetic conversations. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

OpenAI. 2024. Gpt-4o system card. *Preprint*, arXiv:2410.21276.

OpenAI. 2024. Introducing SWE-bench verified. `https://openai.com/index/introducing-swe-bench-verified/`.

OpenAI. 2025a. Gpt-4o mini: Advancing cost-efficient intelligence. Accessed: 2025-03-01.

OpenAI. 2025b. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*.

OpenAI. 2025c. Operator system card. Technical report, OpenAI.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, and 1 others. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations*.

Tomislav Pejsa, Julian Kantor, Hrvoje Benko, Eyal Ofek, and Andrew Wilson. 2016. Room2room: Enabling life-size telepresence in a projected augmented reality environment. In *Proceedings of the 19th ACM conference on computer-supported cooperative work & social computing*, pages 1716–1725.

Elizaveta Pertseva, Mark Barbone, Joey Rudek, and Nadia Polikarpova. 2022. Regex+: Synthesizing regular expressions from positive examples. *11TH Workshop on Synthesis*.

Yewen Pu, Kevin Ellis, Marta Kryven, Josh Tenenbaum, and Armando Solar-Lezama. 2020. Program synthesis with pragmatic communication. In *Advances in Neural Information Processing Systems*, volume 33, pages 13249–13259. Curran Associates, Inc.

Yewen Pu, Saujas Vaduguru, Priyan Vaithilingam, Elena Glassman, and Daniel Fried. 2023. Amortizing pragmatic program synthesis with rankings. *Preprint*, arXiv:2309.03225.

Peng Qi, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D. Manning. 2020. Stanza: A Python natural language processing toolkit for many human languages. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, and 1 others. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Sudha Rao and Hal Daumé III. 2018. Learning to ask good questions: Ranking clarification questions using neural expected value of perfect information. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2737–2746, Melbourne, Australia. Association for Computational Linguistics.

Aditya Sanghi, Hang Chu, Joseph G Lambourne, Ye Wang, Chin-Yi Cheng, Marco Fumero, and Kamal Rahimi Malekshan. 2022. Clip-forge: Towards zero-shot text-to-shape generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18603–18613.

Ari Seff, Yaniv Ovadia, Wenda Zhou, and Ryan P Adams. 2020. Sketchgraphs: A large-scale dataset for modeling relational geometry in computer-aided design. *arXiv preprint arXiv:2007.08506*.

Ari Seff, Wenda Zhou, Nick Richardson, and Ryan P Adams. 2021. Vitruvion: A generative model of parametric cad sketches. *arXiv preprint arXiv:2109.14124*.

Omar Shaikh, Kristina Gligoric, Ashna Khetan, Matthias Gerstgrasser, Diyi Yang, and Dan Jurafsky. 2024. Grounding gaps in language model generations. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6279–6296, Mexico City, Mexico. Association for Computational Linguistics.

Shannon Zejiang Shen, Valerie Chen, Ken Gu, Alexis Ross, Zixian Ma, Jillian Ross, Alex Gu, Chenglei Si, Wayne Chi, Andi Peng, Jocelyn J Shen, Ameet Talwalkar, Tongshuang Wu, and David Sontag. 2025. Completion $\neq$ collaboration: Scaling collaborative effort with agents.

Chenglei Si, Yanzhe Zhang, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2024. Design2code: How far are we from automating front-end engineering? *Preprint*, arXiv:2403.03163.

Prasann Singhal, Tanya Goyal, Jiacheng Xu, and Greg Durrett. 2024. A long way to go: Investigating length correlations in RLHF. In *First Conference on Language Modeling*.

Weiwei Sun, Xuhui Zhou, Weihua Du, Xingyao Wang, Sean Welleck, Graham Neubig, Maarten Sap, and Yiming Yang. 2025. Training proactive and personalized llm agents. *arXiv preprint arXiv:2511.02208*.

Ece Takmaz, Mario Giulianelli, Sandro Pezzelle, Arabella Sinclair, and Raquel Fernández. 2020. Refer, Reuse, Reduce: Generating Subsequent References in Visual and Conversational Contexts. In *Proceedings of the 2020 Conference on Empirical Methods in Natural*

Language Processing (EMNLP), pages 4350–4368, Online. Association for Computational Linguistics.

Saujas Vaduguru, Daniel Fried, and Yewen Pu. 2024. Generating pragmatic examples to train neural program synthesizers. In *The Twelfth International Conference on Learning Representations*.

Saujas Vaduguru, Yilun Hua, Yoav Artzi, and Daniel Fried. 2026. Success and cost elicit convention formation for efficient communication. To appear at the *64th Annual Meeting of the Association for Computational Linguistics*.

Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia, Basil Mustafa, Olivier Hénaff, Jeremiah Harmsen, Andreas Steiner, and Xiaohua Zhai. 2025. Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *Preprint*, arXiv:2502.14786.

Takuma Udagawa and Akiko Aizawa. 2019. A natural language corpus of common grounding under continuous and partially-observable context. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 7120–7127.

Priyan Vaithilingam, Yewen Pu, and Elena L. Glassman. 2023. The usability of pragmatic communication in regular expression synthesis. *Preprint*, arXiv:2308.06656.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. 2020. Trl: Transformer reinforcement learning. `https://github.com/huggingface/trl`.

Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. 2025a. Inducing programmatic skills for agentic tasks. In *Proceedings of the Conference on Language Modeling*.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025b. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911. PMLR.

Julia White, Jesse Mu, and Noah D Goodman. 2020. Learning to refer informatively by amortizing pragmatic reasoning. In *CogSci*.

Anthony Williams and Robert Cowdroy. 2002. How designers communicate ideas to each other in design meetings. In *DS 30: Proceedings of DESIGN 2002, the 7th International Design Conference, Dubrovnik*.

Shirley Wu, Michel Galley, Baolin Peng, Hao Cheng, Gavin Li, Yao Dou, Weixin Cai, James Zou, Jure Leskovec, and Jianfeng Gao. 2025. CollabLLM: From passive responders to active collaborators. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 67260–67283. PMLR.

Tianhao Wu, Weizhe Yuan, Olga Golovneva, Jing Xu, Yuandong Tian, Jiantao Jiao, Jason E. Weston, and Sainbayar Sukhbaatar. 2024. Meta-rewarding language models: Self-improving alignment with llm-as-a-meta-judge. *ArXiv*, abs/2407.19594.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. 2020. Benchmarking multimodal regex synthesis with complex structures. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 6081–6094, Online. Association for Computational Linguistics.

Lili Yu, Howard Chen, Sida I. Wang, Tao Lei, and Yoav Artzi. 2020. Interactive classification by asking informative questions. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Online. Association for Computational Linguistics.

Simon Yu, Gang Li, Weiyan Shi, and Peng Qi. 2026. PolySkill: Learning generalizable skills through polymorphic abstraction for continual learning. In *International Conference on Learning Representations*.

Kai Zhang, Lingbo Mo, Wenhu Chen, Huan Sun, and Yu Su. 2023a. MagicBrush: A manually annotated dataset for instruction-guided image editing. In *Advances in Neural Information Processing Systems*.

Michael JQ Zhang and Eunsol Choi. 2025. Clarify when necessary: Resolving ambiguity through interaction with LMs. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 5541–5558, Albuquerque, New Mexico. Association for Computational Linguistics.

Michael JQ Zhang, W. Bradley Knox, and Eunsol Choi. 2025. Modeling future conversation turns to teach LLMs to ask clarifying questions. In *International Conference on Learning Representations*.

Tianyi Zhang, Tao Yu, Tatsunori Hashimoto, Mike Lewis, Wen-Tau Yih, Daniel Fried, and Sida Wang. 2023b. Coder reviewer reranking for code generation. In *Proceedings of the 40th International Conference on Machine Learning*.

Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. 2025. SkillWeaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*.

Xuhui Zhou, Weiwei Sun, Qianou Ma, Yiqing Xie, Jiarui Liu, Weihua Du, Sean Welleck, Yiming Yang, Graham Neubig, Sherry Tongshuang Wu, and Maarten Sap. 2026. Mind the sim2real gap in user simulation for agentic tasks. *Preprint*, arXiv:2603.11245.

Yifei Zhou, Song Jiang, Yuandong Tian, Jason Weston, Sergey Levine, Sainbayar Sukhbaatar, and Xian Li. 2025a. SWEET-RL: Training multi-turn LLM agents on collaborative reasoning tasks. *arXiv preprint arXiv:2503.15478*.

Yifei Zhou, Song Jiang, Yuandong Tian, Jason Weston, Sergey Levine, Sainbayar Sukhbaatar, and Xian Li. 2025b. Sweet-rl: Training multi-turn llm agents on collaborative reasoning tasks. *Preprint*, arXiv:2503.15478.

Hao Zhu, Graham Neubig, and Yonatan Bisk. 2021. Few-shot language coordination by modeling theory of mind. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 12901–12911. PMLR.

C Lawrence Zitnick and Devi Parikh. 2013. Bringing semantics into focus using visual abstraction. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3009–3016.